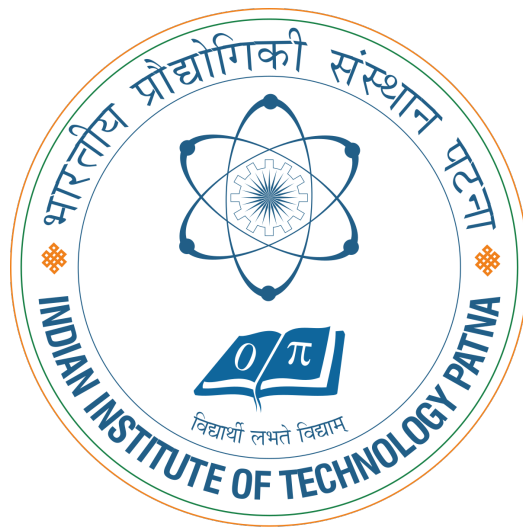


Multi-class classification using Fully Connected Neural Network



Ritik Yadav (2621MA03)

Sahnil Siddique (2621MA02)

Suman Acharya (2621MA11)

Department of Mathematics
Indian Institute of Technology Patna

1. Introduction

The assignment requires a fully connected neural network for multi-class classification on two synthetic datasets. The experiments compare several hidden-layer configurations and use a 60%/20%/20% class-wise training/validation/test split. The model is trained with stochastic gradient descent and squared error as the instantaneous loss. The report focuses on the experimental results present in the supplied notebook and organizes them according to the assignment requirements.

2. Objectives

The objective of this task are:

- Generate a three-class Gaussian cluster dataset and a three-class nonlinear spiral dataset, with 500 examples per class.
- Split each dataset class-wise into 60% training, 20% validation, and 20% test data.
- Implement an FCNN with manual forward propagation, squared-error loss, backpropagation, and SGD updates.
- Compare multiple hidden-layer architectures and select the best architecture using validation accuracy.
- Report training/validation error curves, decision regions, confusion matrices, output-node behavior, and classification accuracy.
- Compare the FCNN against a single-neuron baseline and interpret the observed results.

3. Dataset Generation and Preparation

3.1 Three-Class Gaussian Clusters

The Gaussian dataset contains 1,500 two-dimensional samples: 500 samples for each of three classes. The notebook generates the classes using different means and different standard deviations. The class centers used in the implementation are approximately $(-3, -1)$, $(3, -1)$, and $(0, 3.5)$. The resulting clusters are compact and well separated in the plane.

3.2 Three-Class Spiral Dataset

The spiral dataset also contains 1,500 samples, with 500 samples per class. Each class is generated along a spiral using a radial variable and a class-dependent angular offset. Noise is added to the angular coordinate and to the Cartesian coordinates. The three classes form interleaving arms, making the problem strongly nonlinear.

3.3 Train/Validation/Test Split

Dataset	Training	Validation	Test	Total
Gaussian	900 (60%)	300 (20%)	300 (20%)	1500
Spiral	900 (60%)	300 (20%)	300 (20%)	1500

The split is performed separately within each class, preserving equal class counts in all three subsets: 300 training, 100 validation, and 100 test samples per class.

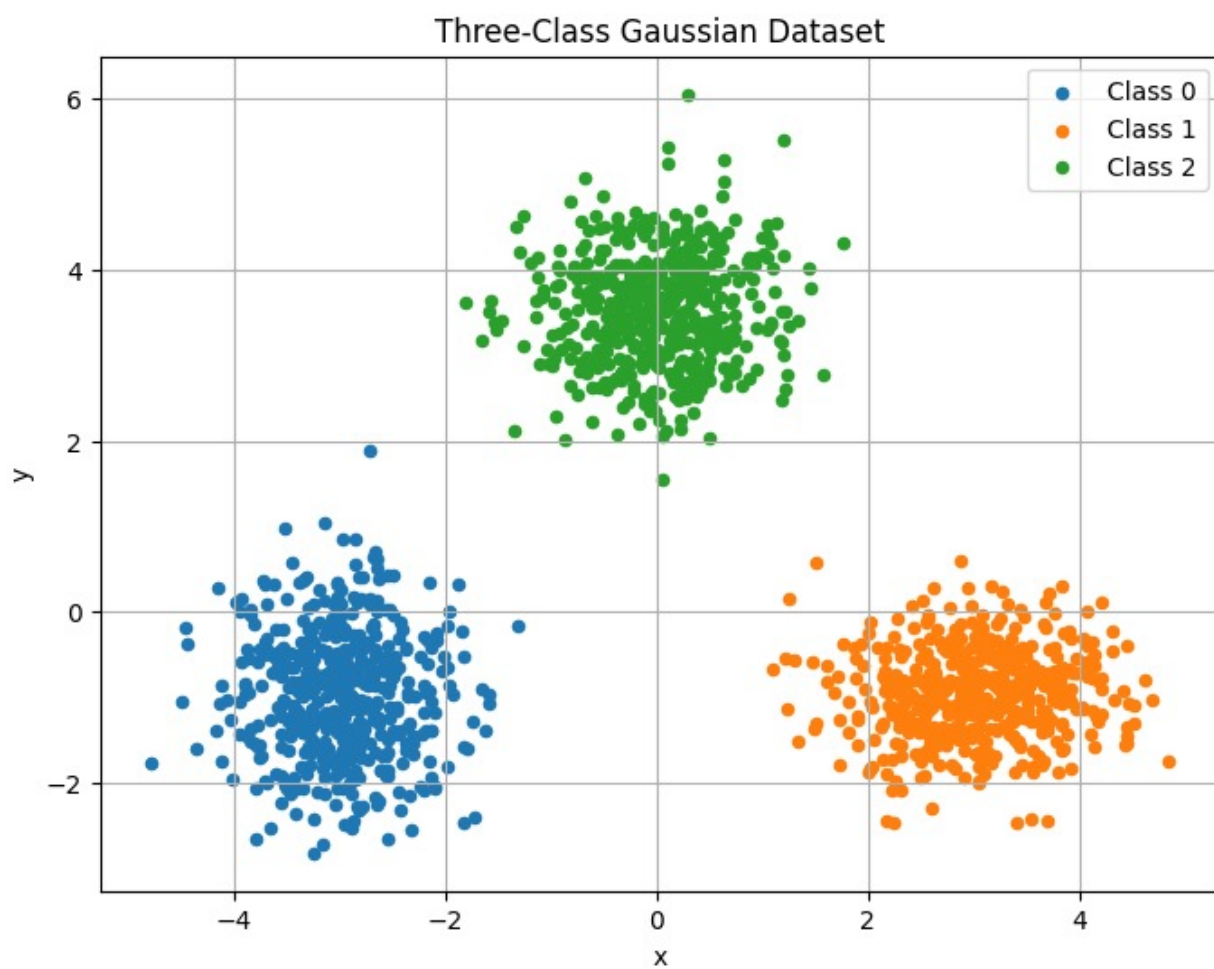


Figure 1: Three-class Gaussian dataset generated by the supplied notebook.

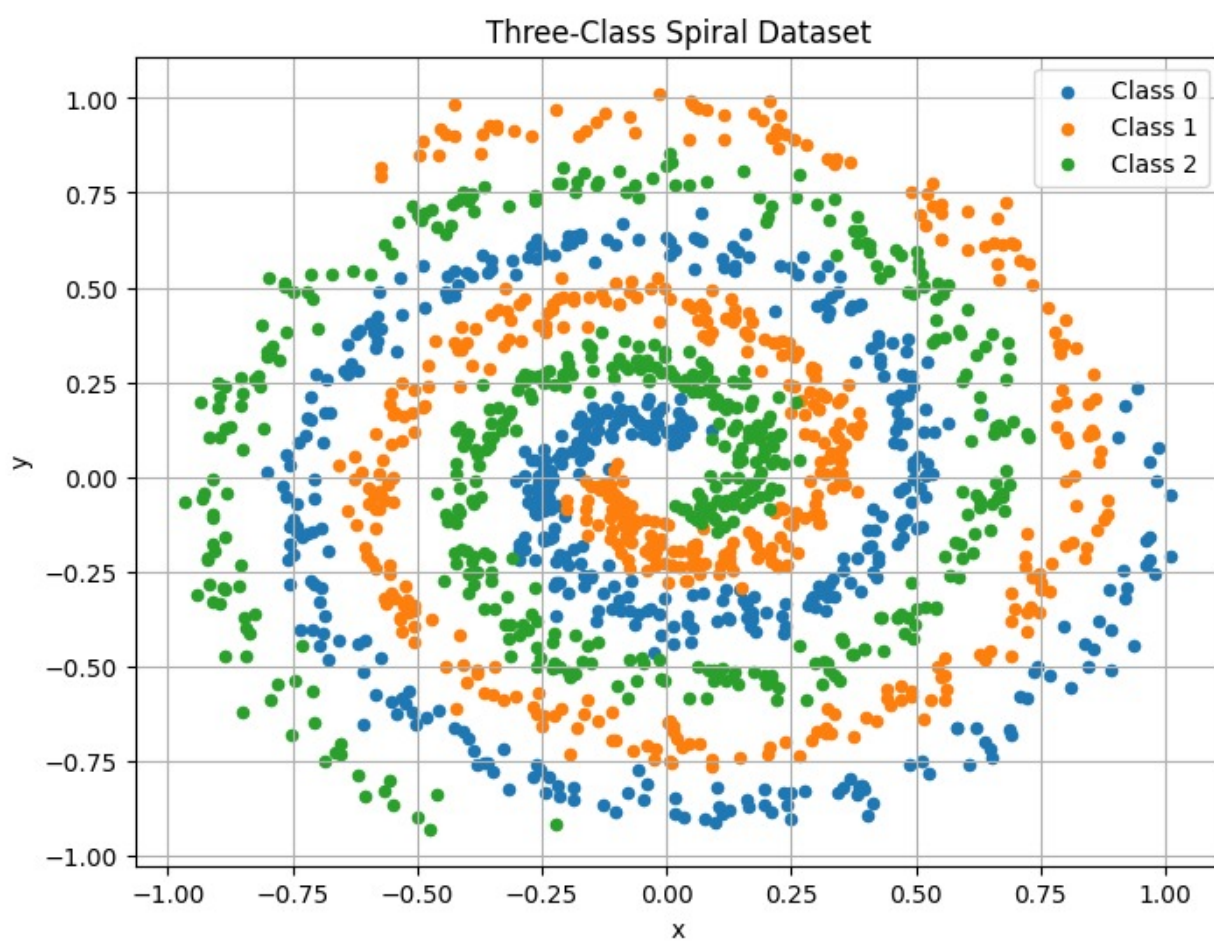


Figure 2: Three-class nonlinear spiral dataset.

4. FCNN Architecture and Mathematical Formulation

4.1 Network Structure

The FCNN accepts two input variables, corresponding to the x and y coordinates. The hidden layers use the hyperbolic tangent activation function. The final layer has three output nodes and uses softmax so that the outputs form class probabilities.

Forward Propagation:

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^l$$

$$A^{(l)} = \tanh(Z^{(l)}) \text{ for hidden layers}$$

$$A^{(L)} = \text{softmax}(Z^{(L)}) \text{ for the output layer}$$

For a vector $z = (z_n, z_n, z_n)$, the softmax output is $p_n = \exp(z_n) / \sum \exp(z_n)$.

The notebook subtracts the maximum logit before exponentiation, which improves numerical stability.

4.2 Squared-Error Loss

$E = \text{mean}((y_{\text{pred}} - y_{\text{true}})^2)$ The assignment specifically asks for squared error as the instantaneous loss. The notebook represents class labels using one-hot vectors, so each training example has a three-dimensional target vector.

4.3 Backpropagation and SGD

For the output layer, the implementation starts with the derivative of the mean squared error with respect to the output values. The error signal is

then propagated backward through the weight matrices. For hidden layers, the derivative of tanh is applied using $\tanh'(z) = 1 - \tanh^2(z)$. The gradients are used immediately to update the weights and biases, one training example at a time, which implements stochastic gradient descent.

$$W \leftarrow W - learning_rate * dW$$

$$b \leftarrow b - learning_rate * db$$

4.4 Initialization

The notebook uses a Xavier-style uniform initialization with limit $\sqrt{6/(\text{fan_in} + \text{fan_out})}$ for each weight matrix, while biases are initialized to zero.

5. Experimental Setup

Parameter	Gaussian Experiment	Spiral Experiment
Input dimension	2	2
Output classes	3	3
Candidate architectures	[4], [8], [16], [32]	[8,8], [16,16], [32,16], [32,32]
Epochs	500	1000
Learning rate	0.02	0.02
Loss	Squared error	Squared error
Hidden activation	tanh	tanh
Output activation	softmax	softmax
Optimization	SGD, one example/update	SGD, one example/update

6. Architecture Selection Using Validation Data

6.1 Gaussian Dataset

Architecture	Final validation error	Validation accuracy
(4,)	0.000000	100.00%
(8,)	0.000000	100.00%
(16,)	0.000000	100.00%
(32,)	0.000000	100.00%

All four Gaussian architectures reached 100.00% validation accuracy and a displayed final validation error of 0.000000. The notebook's selection routine chooses the first architecture among ties, giving (4,) as the selected best Gaussian architecture.

6.2 Spiral Dataset

Architecture	Final validation error	Validation accuracy
(8,8)	0.000000	100.00%
(16,16)	0.000000	100.00%
(32,16)	0.000000	100.00%
(32,32)	0.000000	100.00%

The spiral results show a clear benefit from increased network capacity. The (8,8) network reaches only 63.00% validation accuracy, whereas (32,32) reaches 94.33% and has the smallest final validation error among the tested architectures. Consequently, (32,32) is selected as the best spiral architecture.

7. Training and Validation Error

7.1 Best Gaussian Architecture

The Gaussian error curve rapidly approaches zero. Both training and validation error are displayed as essentially zero throughout the later epochs, consistent with the 100% validation classification accuracy.

7.2 Best Spiral Architecture

The spiral curve remains comparatively high during early training and begins a substantial reduction after several hundred epochs. The curve is noisier than the Gaussian curve, which is consistent with the greater difficulty of learning a nonlinear spiral boundary using SGD.

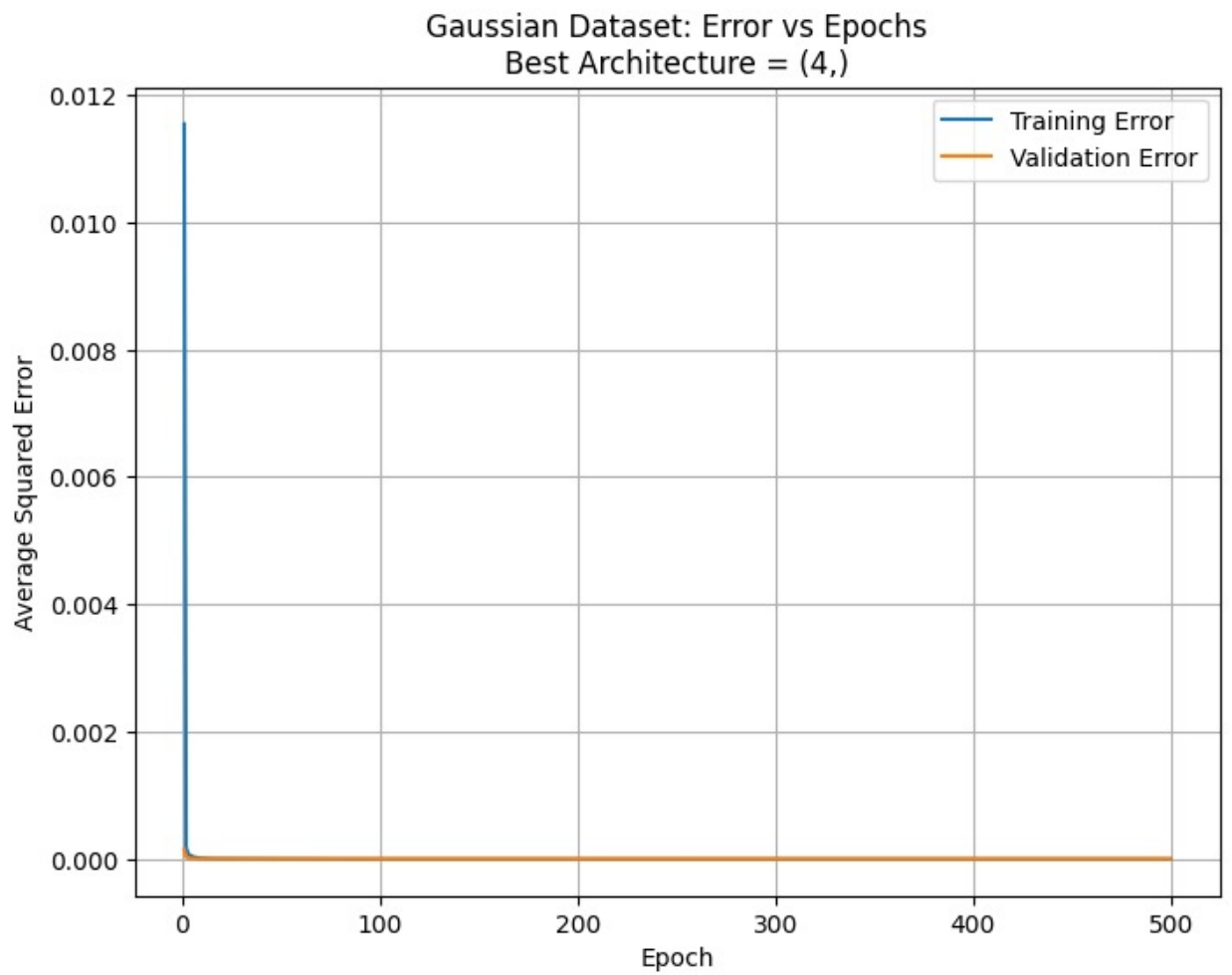


Figure 3: Average squared error versus epoch for the selected Gaussian architecture (4 hidden neurons).

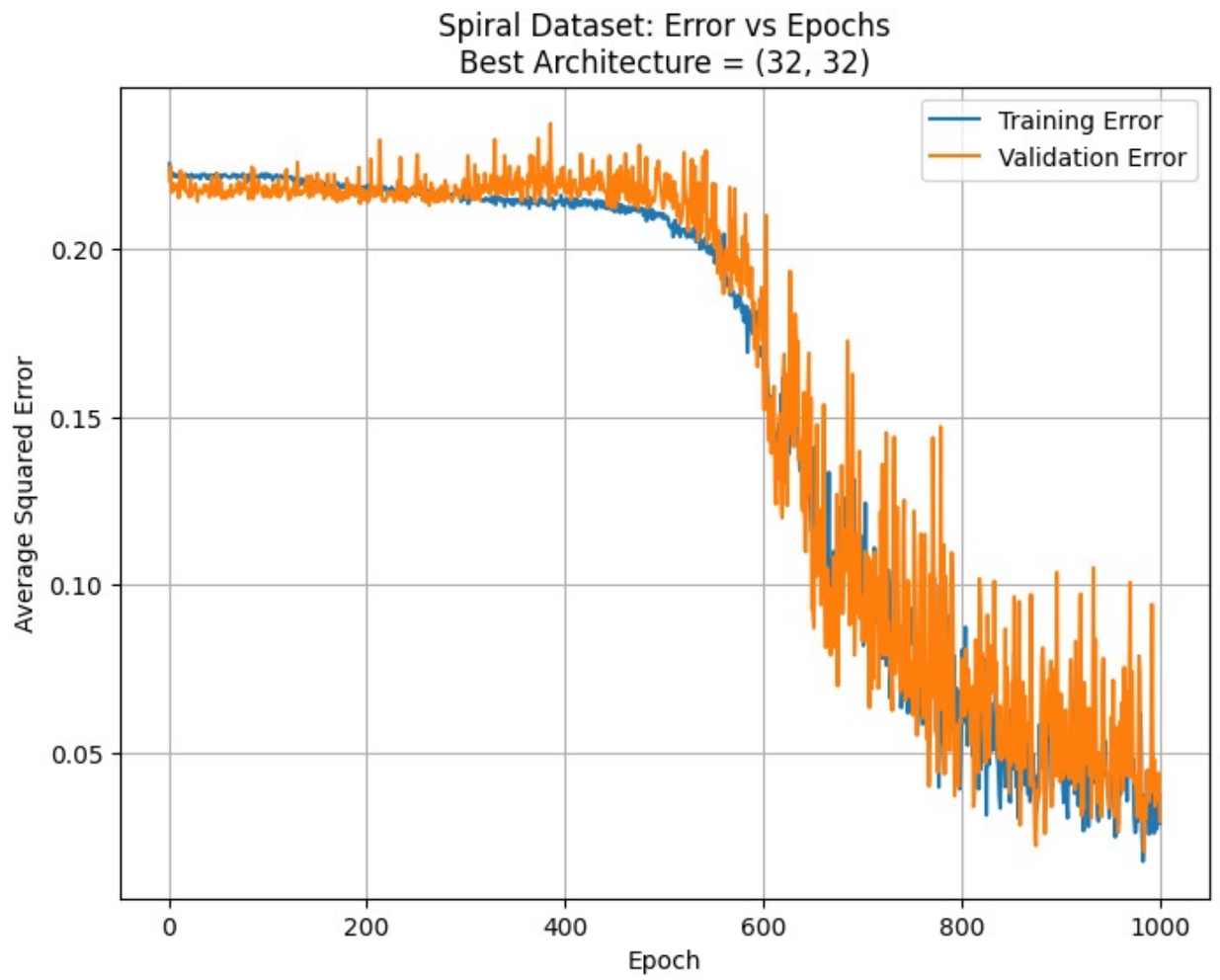


Figure 4: Average squared error versus epoch for the selected spiral architecture (32,32).

8. Decision Regions

8.1 Gaussian Dataset

The learned regions are cleanly separated into three areas corresponding to the three compact clusters. This agrees with the perfect validation and test classification results.

8.2 Spiral Dataset

The (32,32) network produces a substantially more complex nonlinear partition than the single-neuron baseline. The decision regions follow the spiral structure sufficiently well to obtain 92.33% test accuracy, although some points near the interleaving class boundaries remain difficult to classify.

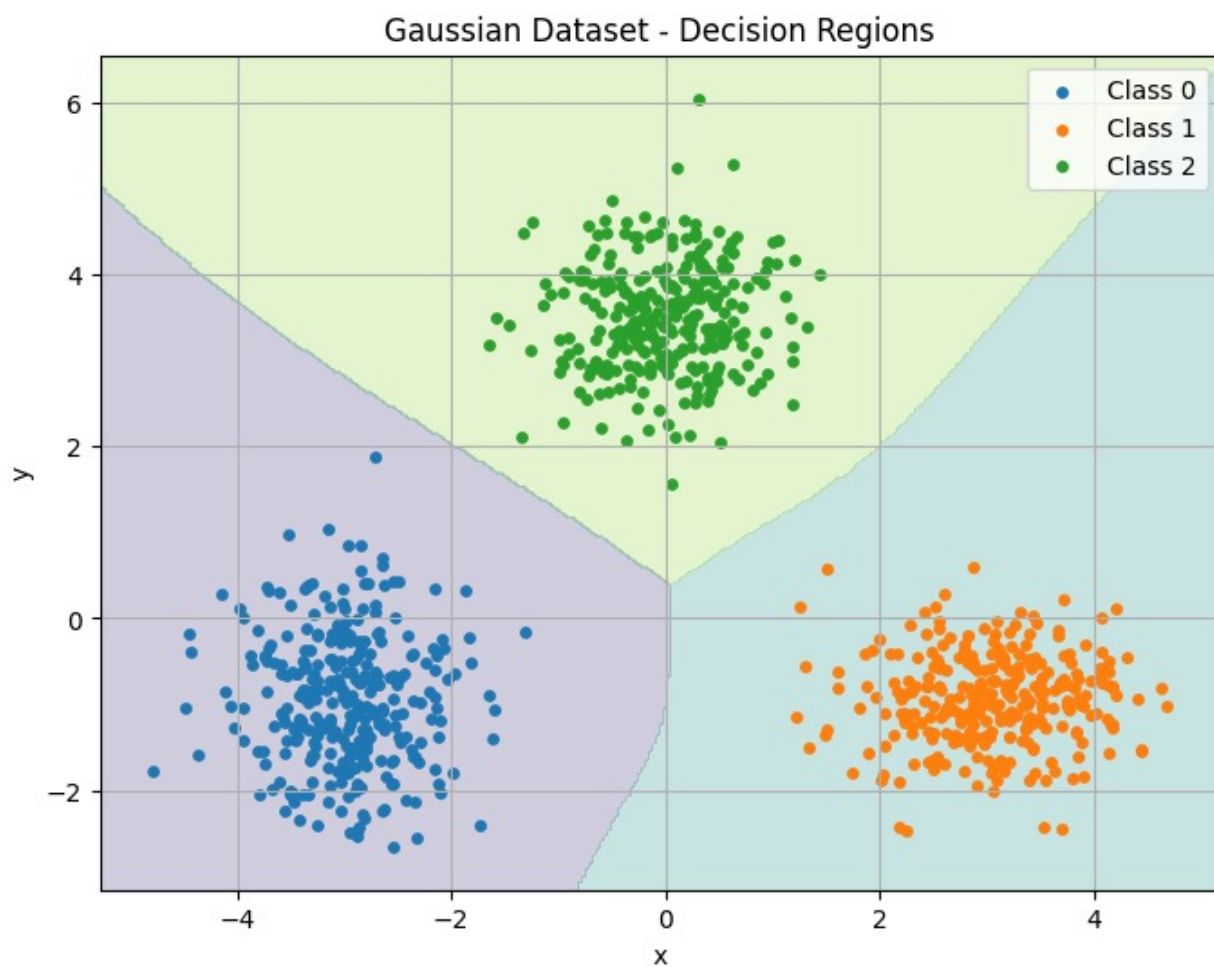


Figure 5: Decision regions of the selected Gaussian FCNN on the training data.

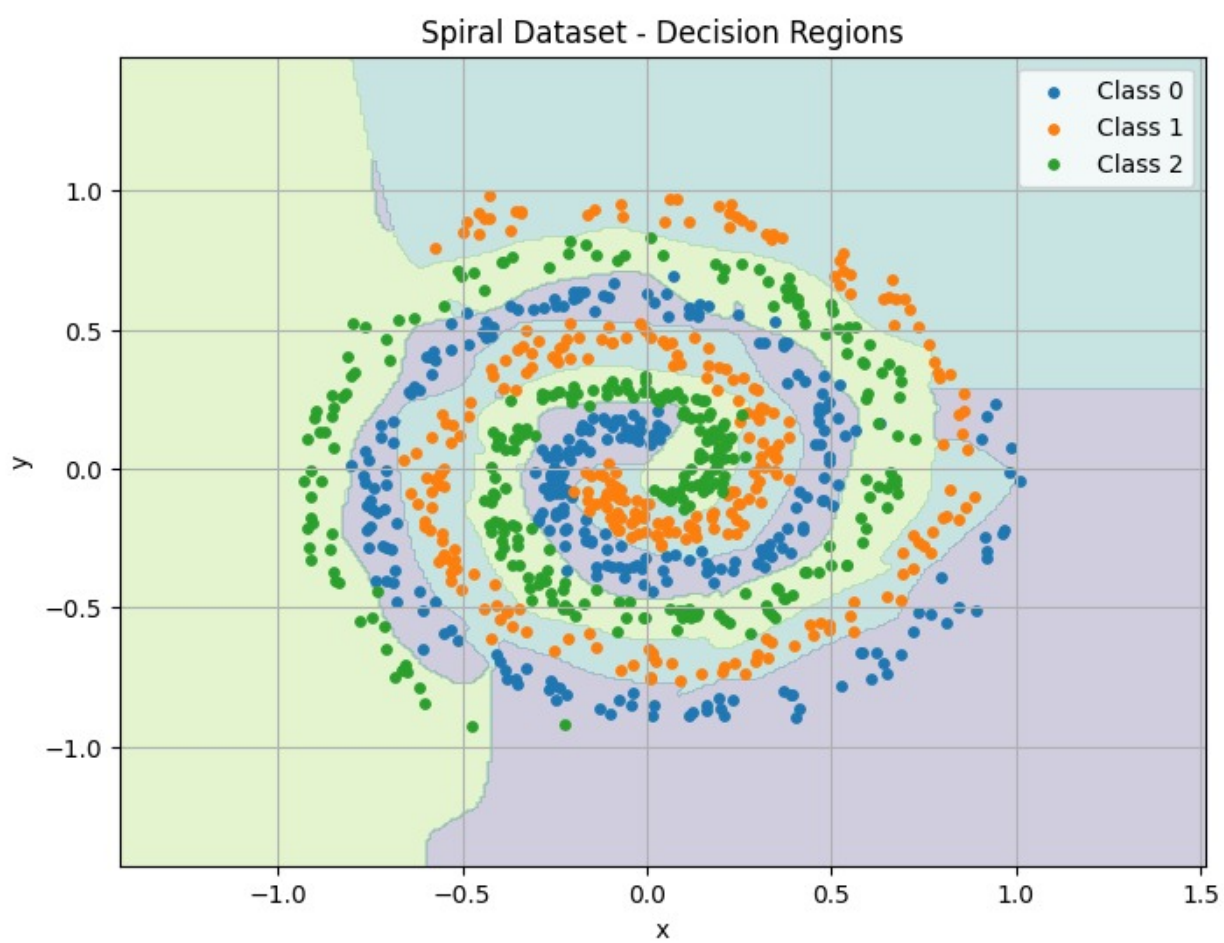


Figure 6: Decision regions of the selected spiral FCNN on the training data.

9. Confusion Matrices and Classification Accuracy

9.1 Validation Results for All Architectures

Gaussian validation confusion matrices:

Architecture	Confusion matrix	Accuracy
(4,)	[[100,0,0],[0,100,0],[0,0,100]]	100.00%
(8,)	[[100,0,0],[0,100,0],[0,0,100]]	100.00%
(16,)	[[100,0,0],[0,100,0],[0,0,100]]	100.00%
(32,)	[[100,0,0],[0,100,0],[0,0,100]]	100.00%

Spiral validation confusion matrices:

Architecture	Confusion matrix	Accuracy
(8,8)	[[80,8,12],[46,49,5],[27,13,60]]	63.00%
(16,16)	[[95,2,3],[9,88,3],[4,7,89]]	90.67%
(32,16)	[[93,1,6],[6,92,2],[3,3,94]]	93.00%
(32,32)	[[93,1,6],[5,94,1],[2,2,96]]	94.33%

For the spiral validation set, the diagonal entries increase markedly as the network becomes more expressive. For the selected (32,32) model, the diagonal contains 93, 94, and 96 correct predictions for Classes 0, 1, and 2 respectively.

9.2 Test-Set Results for the Selected Models

- Gaussian test confusion matrix: $[[100,0,0],[0,100,0],[0,0,100]]$, giving 100.00% test accuracy.
- Spiral test confusion matrix: $[[90,4,6],[6,90,4],[1,2,97]]$, giving 92.33% test accuracy.



Figure 7: Gaussian test confusion matrix for the selected (4,) model.

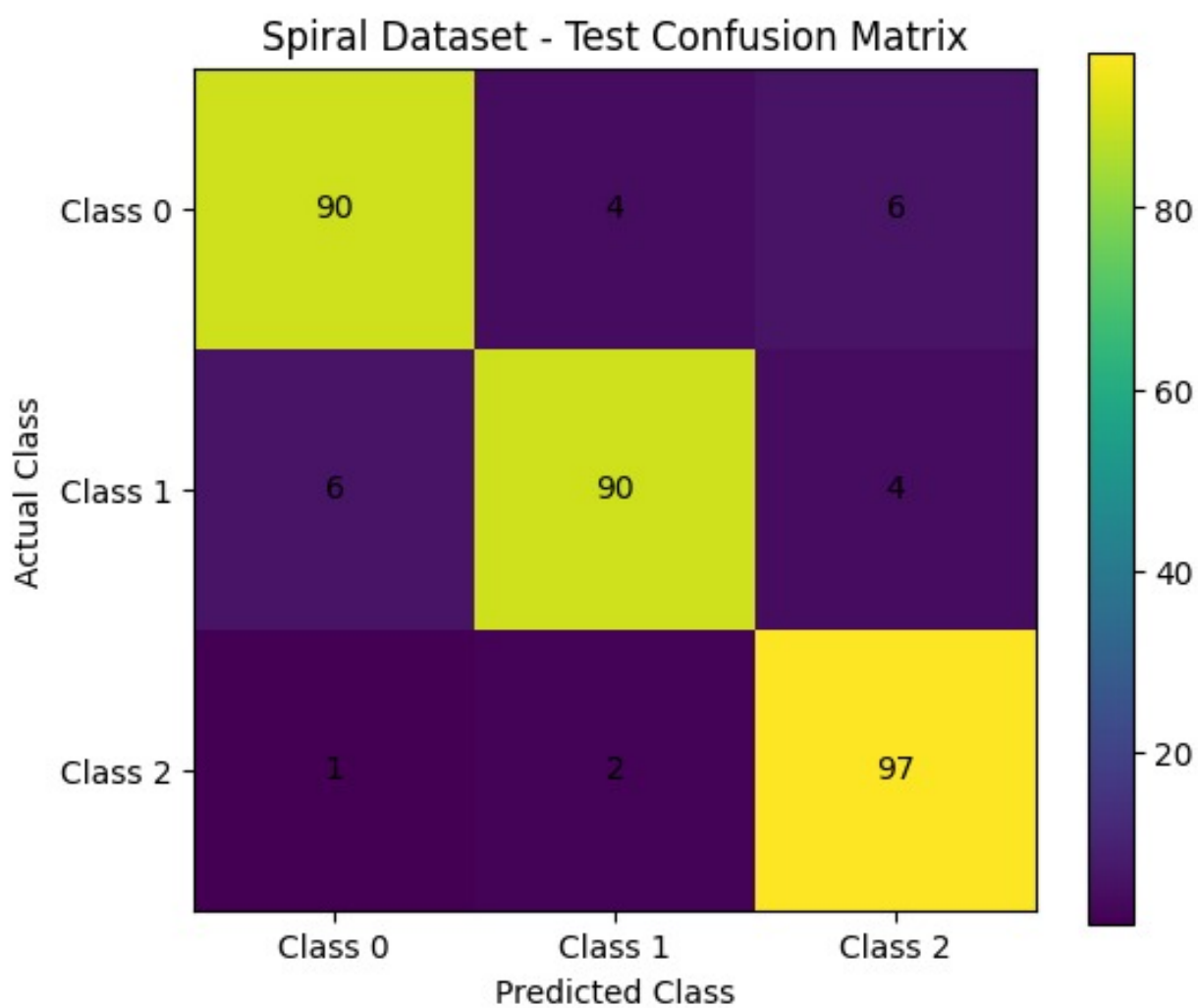


Figure 8: Spiral test confusion matrix for the selected (32,32) model.

10. Hidden-Layer and Output-Node Behavior

The assignment asks for plots of the values produced by hidden nodes and output nodes after training. For the selected Gaussian model, the four hidden nodes and three output nodes are plotted in the notebook. For the selected spiral model, the notebook visualizes the hidden layers and the three output nodes.

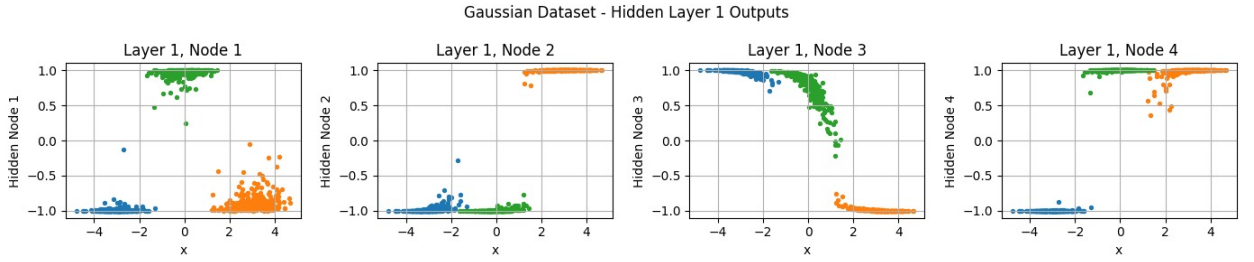


Figure 9: Gaussian selected model: outputs of the four hidden nodes.

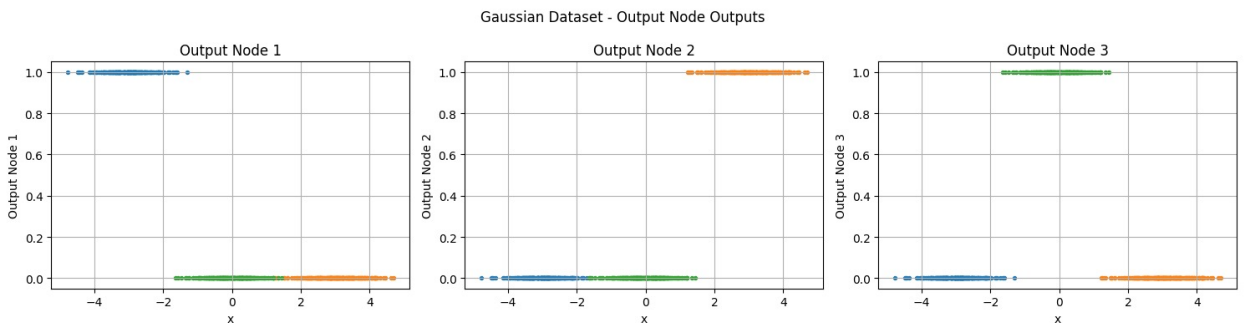


Figure 10: Gaussian selected model: outputs of the three output nodes.

- The Gaussian hidden-node plots show that different neurons respond differently across the spatial clusters. The final output nodes provide class-specific responses that are strongly separated for the three classes.
- The spiral hidden layers contain many more neurons than the Gaussian model. Their responses collectively transform the original two-dimensional spiral geometry into representations that make the three classes more separable at the output layer.

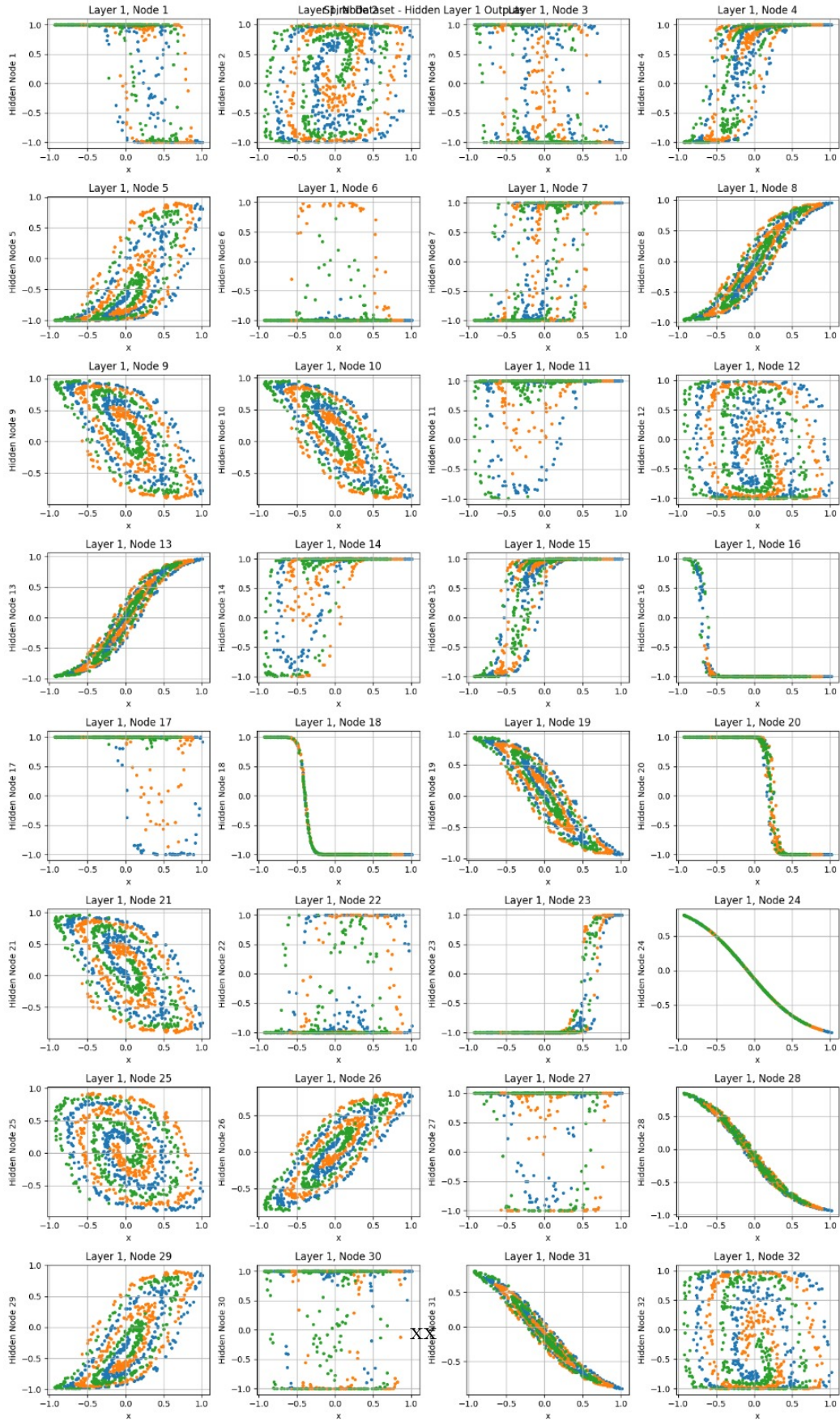


Figure 11: Spiral selected model: hidden-layer 1 node outputs.

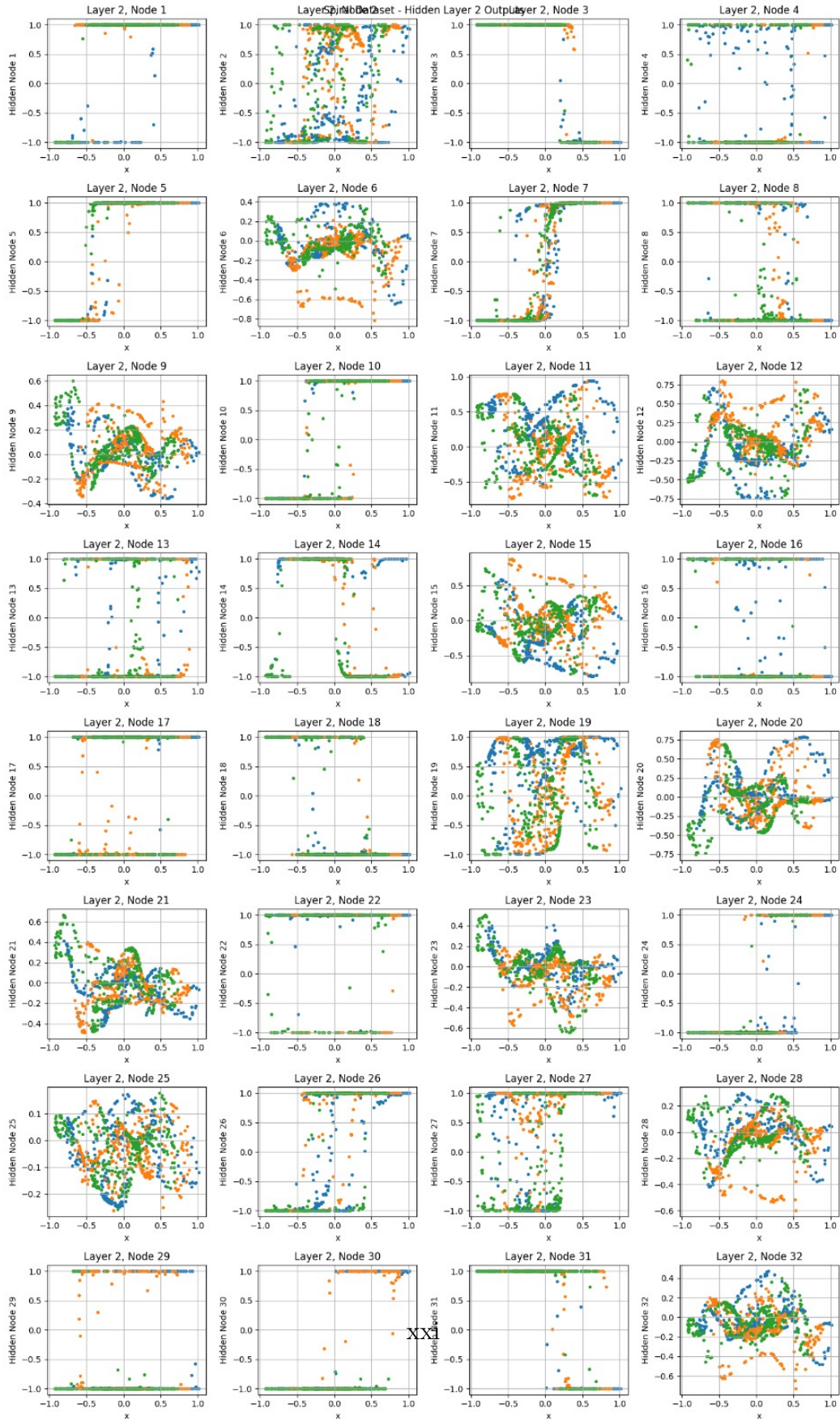


Figure 12: Spiral selected model: hidden-layer 2 node outputs.

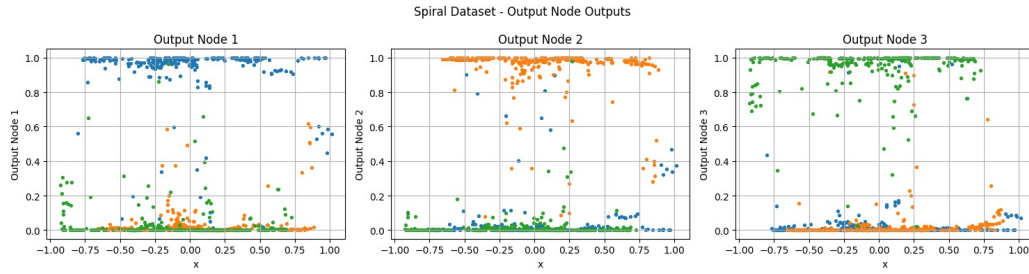


Figure 13: Spiral selected model: three output-node responses.

11. Hidden-Node Activation Surfaces

We also visualize the activation of individual hidden nodes as a function of the two input coordinates. These plots provide an intuitive view of how neurons partition the input space.

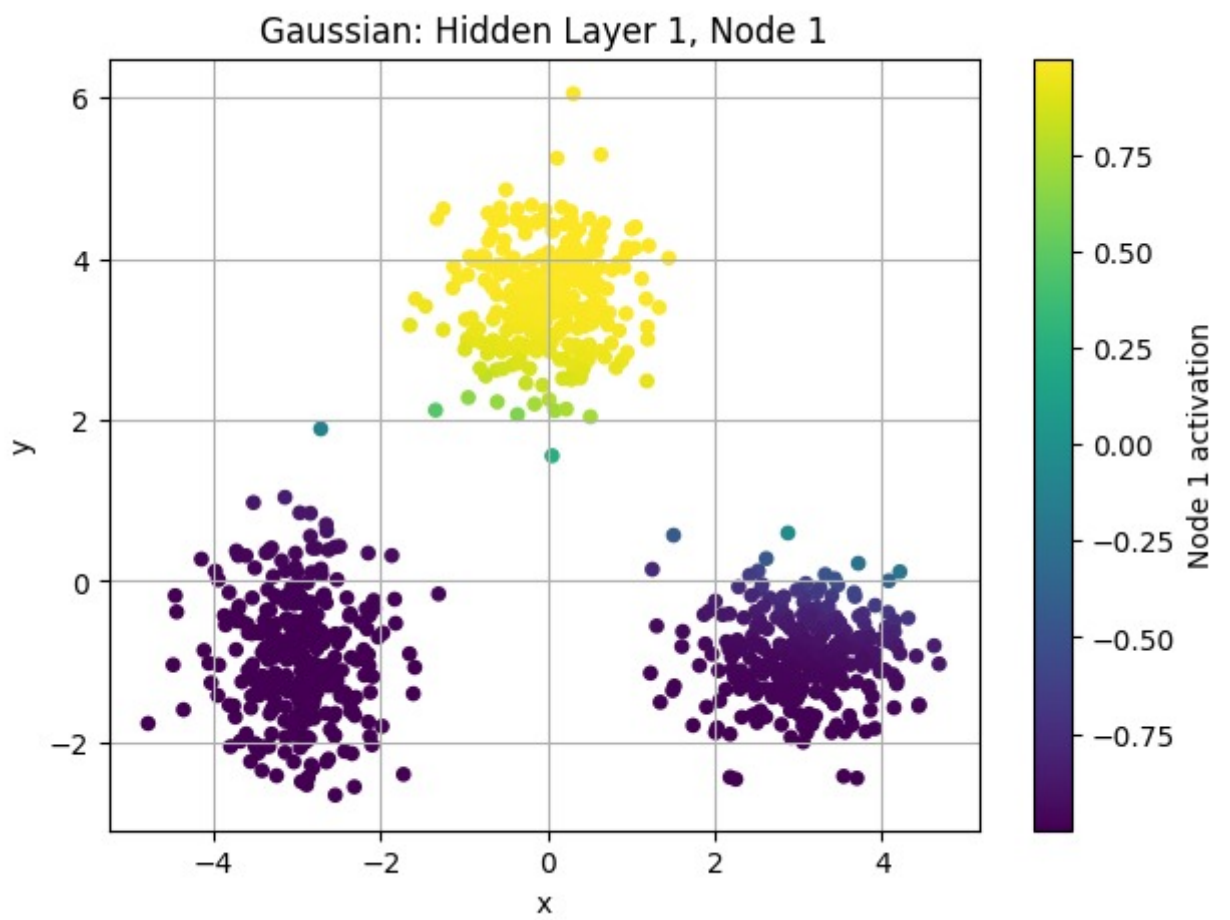


Figure 14: Gaussian hidden layer 1, node 1 activation.

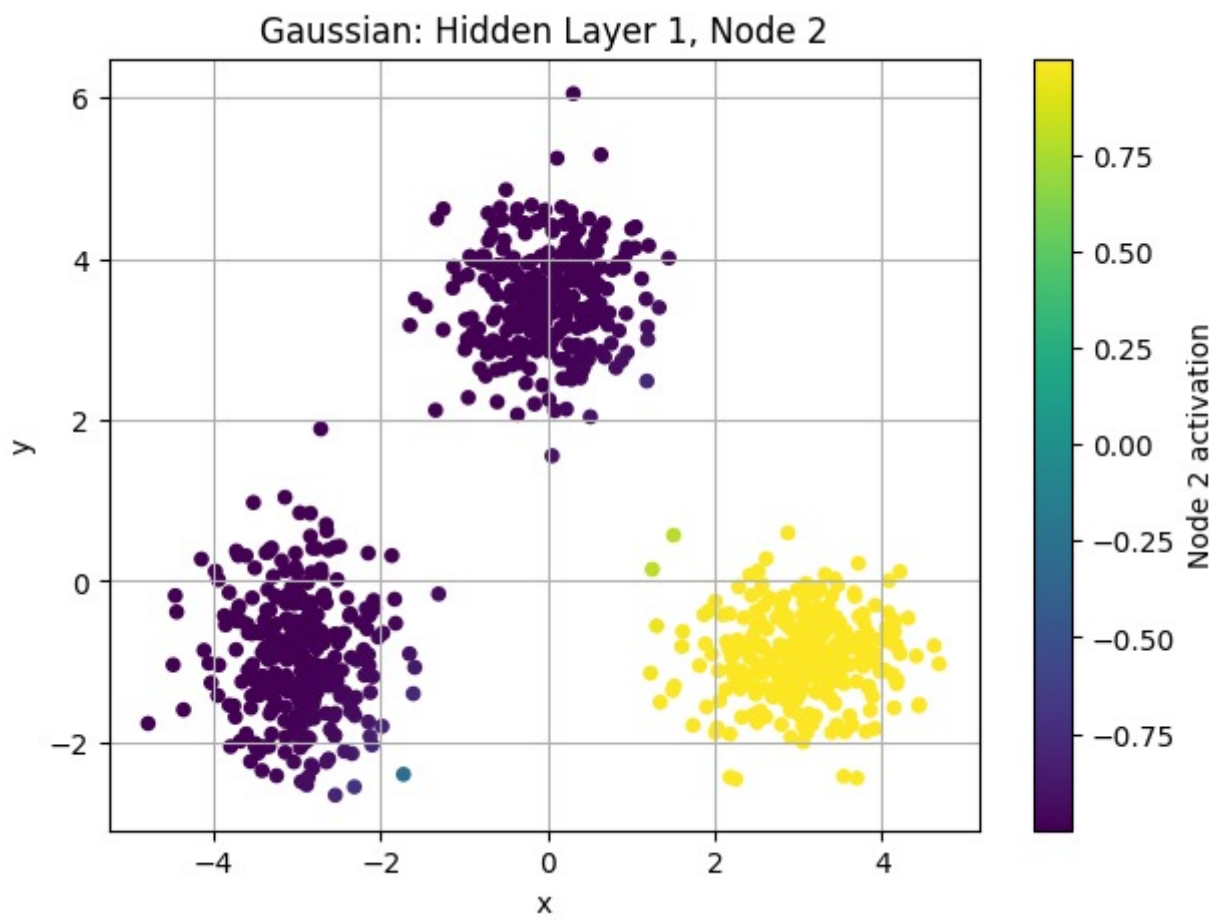


Figure 15: Gaussian hidden layer 1, node 2 activation.

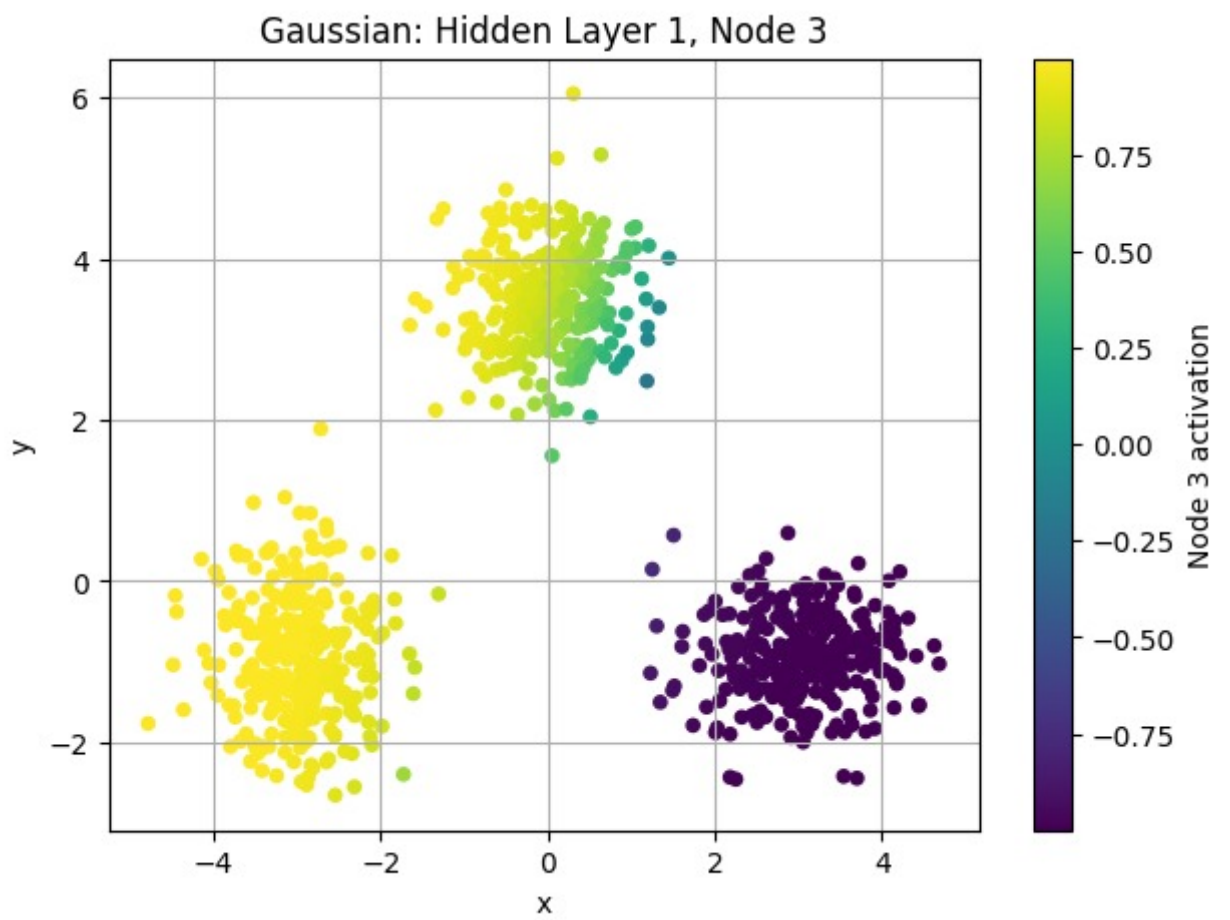


Figure 16: Gaussian hidden layer 1, node 3 activation.

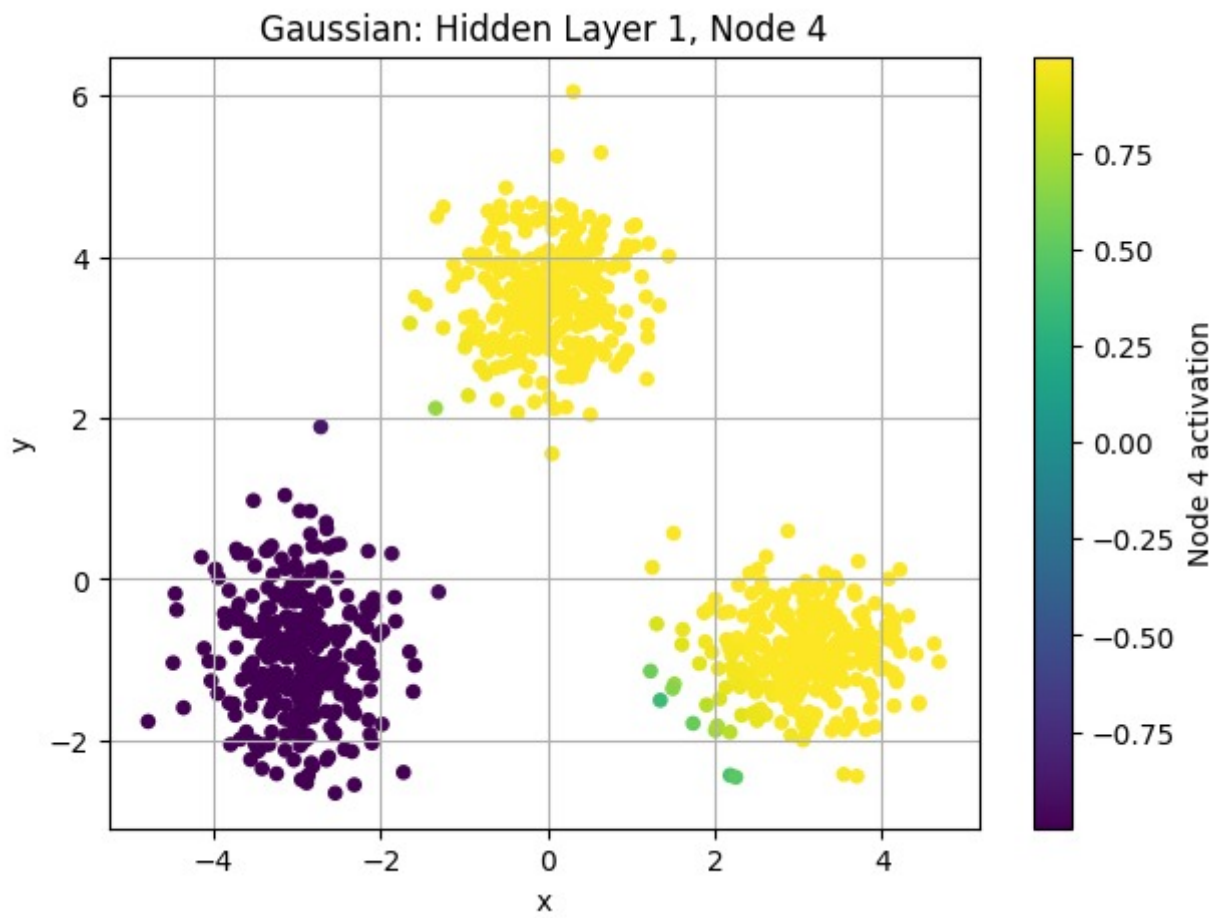


Figure 17: Gaussian hidden layer 1, node 4 activation.

12. Comparison with a Single-Neuron Model

A single-neuron baseline was trained separately for each dataset using SGD and squared error. The scalar target was $y/2$, and the learned scalar output was converted into three classes using thresholds at 0.333 and 0.666. Because a single linear neuron cannot form complex nonlinear decision boundaries, this baseline provides a useful comparison with the FCNN.

Dataset	Single neuron	Best FCNN	Improvement
Gaussian	96.33%	100.00% — (4,)	+3.67 percentage points
Spiral	33.33%	92.33% — (32,32)	+59.00 percentage points

On the Gaussian data, the single neuron performs well because the clusters are approximately linearly separable; nevertheless, the FCNN improves test accuracy from 96.33% to 100.00%. On the spiral data, the single neuron achieves only 33.33%, which is close to the level expected when three classes cannot be represented by a single linear scalar decision rule. The FCNN raises performance to 92.33%, demonstrating the practical value of multiple nonlinear hidden layers.

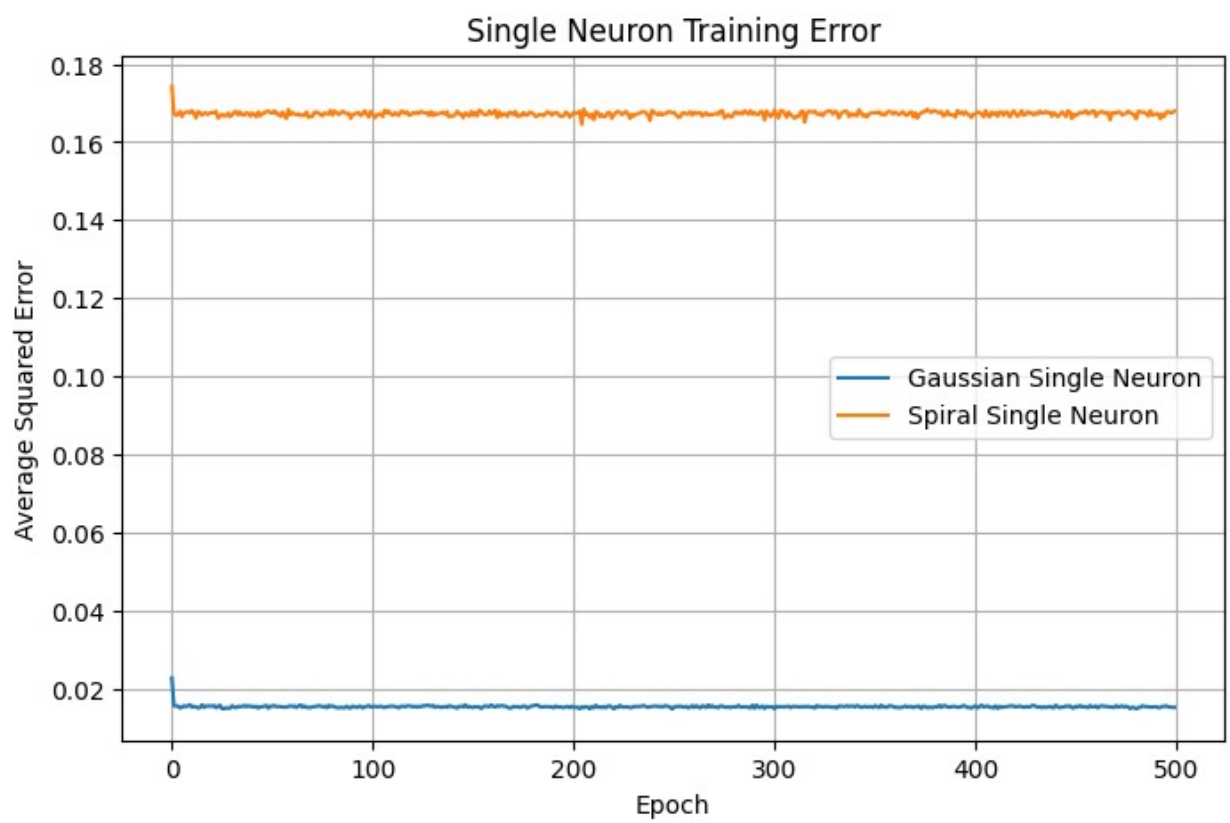


Figure 18: Single-neuron training error for the Gaussian and spiral datasets.

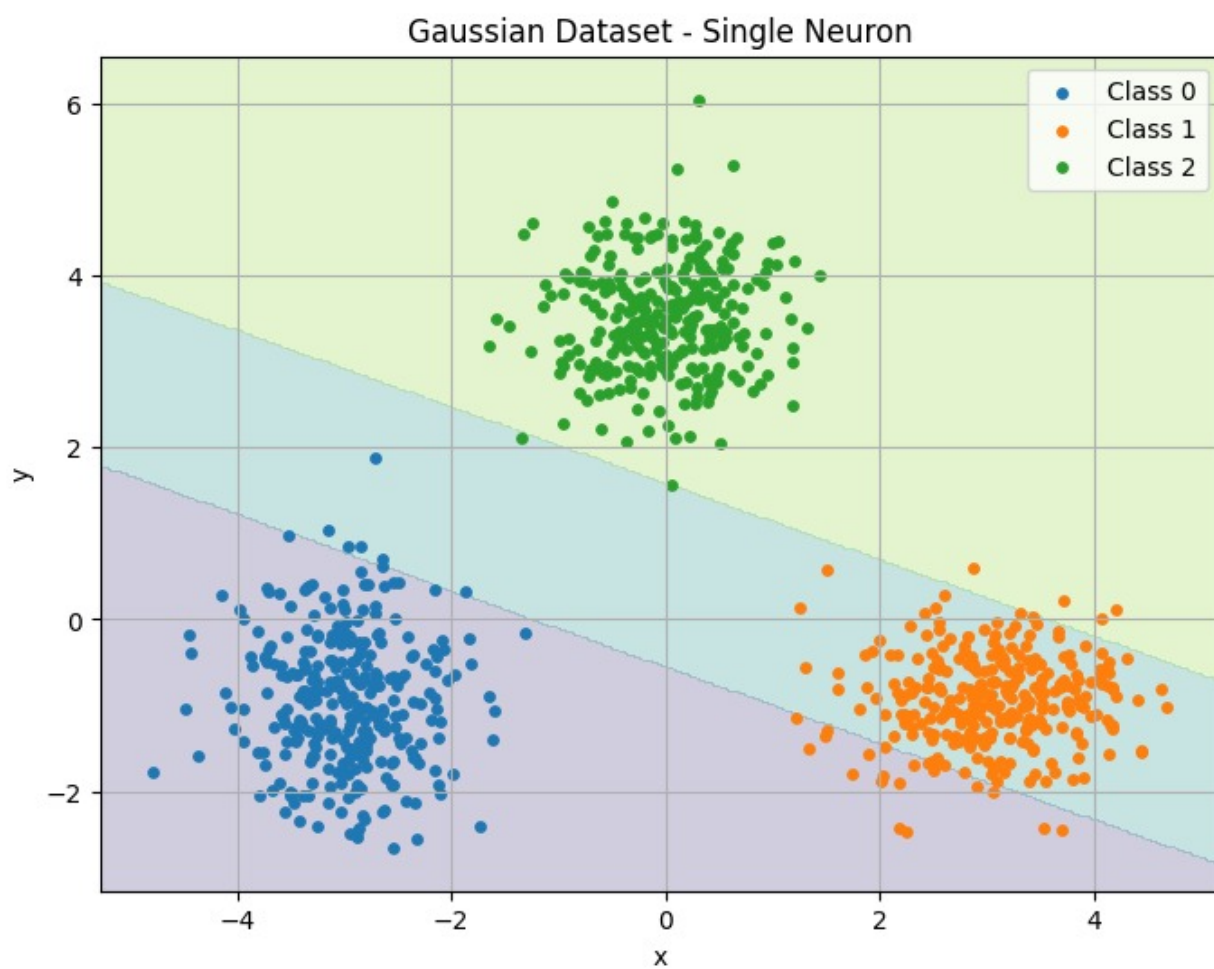


Figure 19: Gaussian single-neuron decision regions.

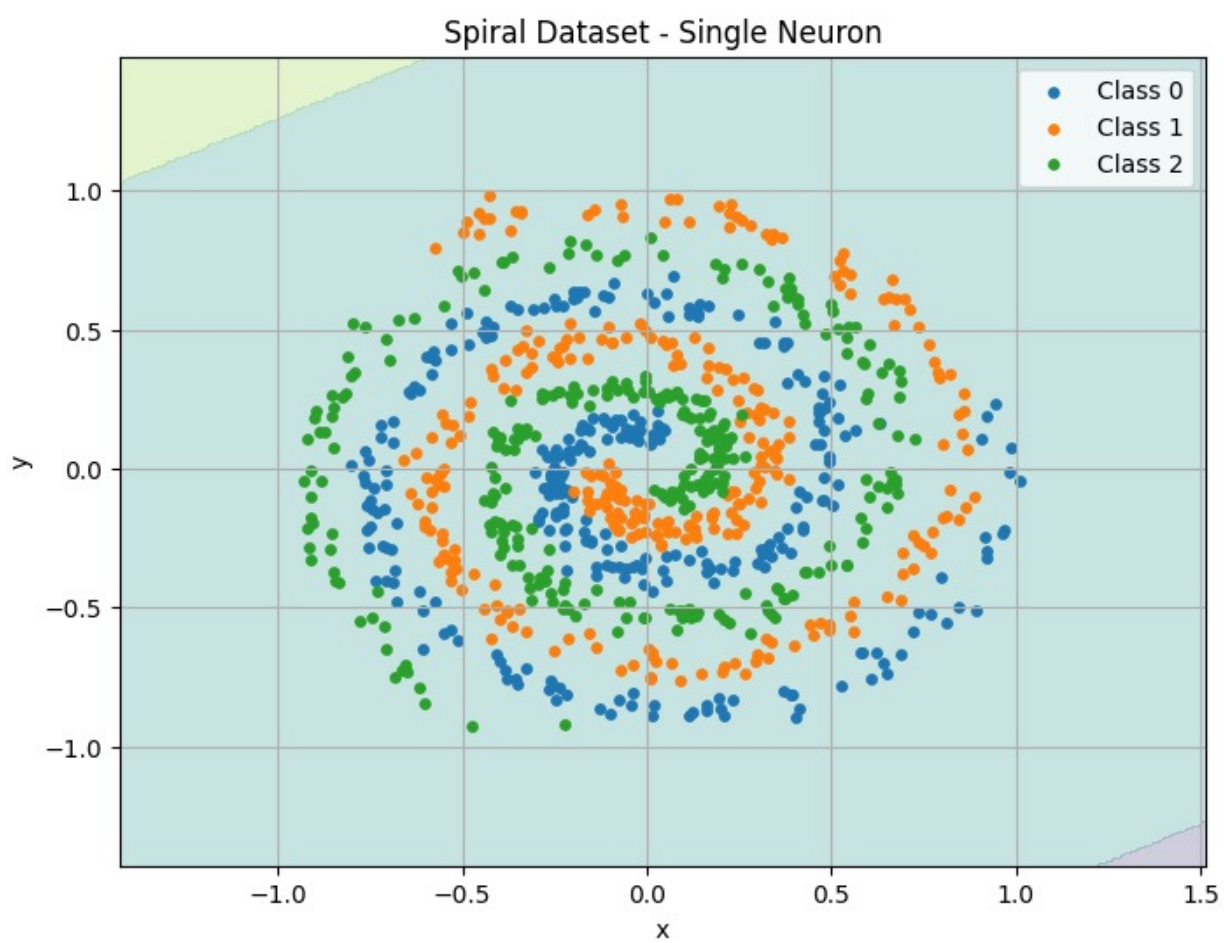


Figure 20: Spiral single-neuron decision regions.

13. Overall Results

Dataset	Best architecture	Validation accuracy	Test accuracy
Gaussian	(4,)	100.00%	100.00%
Spiral	(32,32)	94.33%	92.33%

The main performance observations are:

- Gaussian classification is comparatively easy: every tested one-hidden-layer architecture achieved 100% validation accuracy, and the selected (4,) network achieved 100% test accuracy.
- Spiral classification is substantially harder: increasing hidden-layer capacity improves validation accuracy from 63.00% to 94.33%.
- The selected spiral model generalizes well but shows a small validation-to-test decrease, from 94.33% to 92.33%.
- The FCNN is particularly valuable for nonlinear structure: its advantage over the single-neuron baseline is 59.00 percentage points on the spiral dataset.

14. Inferences and Discussion

Gaussian dataset: The data consist of compact, well-separated clusters. A small nonlinear network is sufficient to learn a near-perfect partition. The fact that all tested architectures obtain 100% validation accuracy indicates that additional hidden neurons do not provide a measurable classification benefit for this particular dataset.

Spiral dataset: The interleaving geometry requires a nonlinear decision function. The results show a strong relationship between model capacity

and classification performance: the smallest tested architecture (8,8) is inadequate, while the larger (32,32) network achieves the best validation and test results.

Role of hidden layers: The hidden layers transform the input coordinates through tanh activations. The resulting internal representations allow the output layer to combine nonlinear features and construct a much more flexible decision boundary than a single neuron can provide.

Optimization behavior: The Gaussian problem converges extremely quickly, while the spiral problem requires substantially more epochs and exhibits noisier error trajectories. This is consistent with the greater geometric complexity of the spiral classes.

Implementation observation: The notebook’s final inference helper contains a wording inconsistency: its spiral inference text says “The Gaussian clusters are approximately linearly separable.” That sentence does not match the spiral dataset. The earlier experiment and decision-region analysis correctly treat the spiral dataset as nonlinearly separable, and that is the interpretation used in this report.

15. Limitations and Possible Improvements

- The architecture search is limited to the configurations explicitly tested in the notebook.
- The FCNN uses full manual SGD updates without mini-batching, momentum, Adam, or learning-rate scheduling.
- Squared error is used because it is required by the assignment, although other losses could behave differently for classification.
- The notebook defines a standardization routine but does not use standard-

ized features in the reported experiments.

- Results are based on a fixed random seed (42). Repeating experiments with several seeds would help assess robustness.
- For the spiral problem, further tuning of depth, width, learning rate, initialization, or number of epochs could potentially improve generalization.

16. Conclusion

A fully connected neural network was implemented from scratch using PyTorch tensors for the required multi-class classification task. The experiments demonstrate two different regimes. For the approximately linearly separable Gaussian clusters, even a small FCNN with one hidden layer of four neurons is sufficient to achieve 100.00% test accuracy. For the nonlinear spiral dataset, a deeper and wider network is necessary; the selected (32,32) architecture reaches 94.33% validation accuracy and 92.33% test accuracy. The comparison with the single-neuron baseline further emphasizes the importance of nonlinear hidden layers, with the FCNN improving spiral test accuracy by 59.00 percentage points. Overall, the results satisfy the main experimental requirements of Task 1 and illustrate how neural-network architecture affects the ability to model complex decision boundaries.

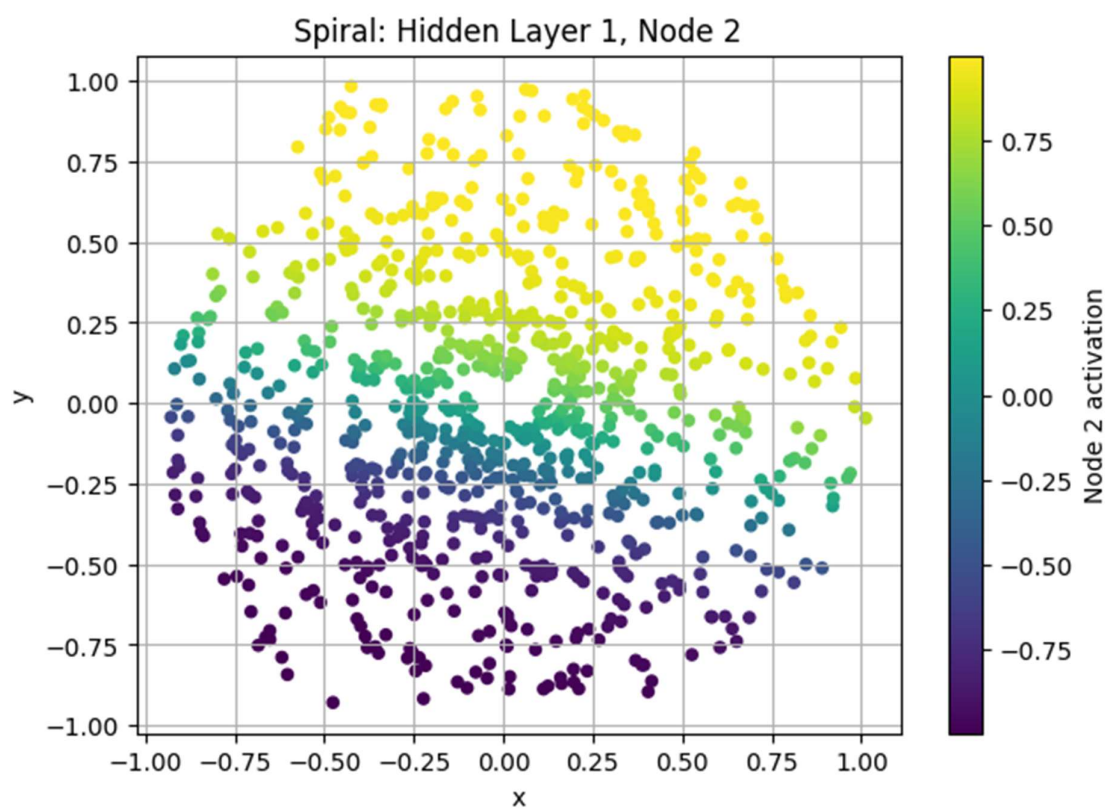
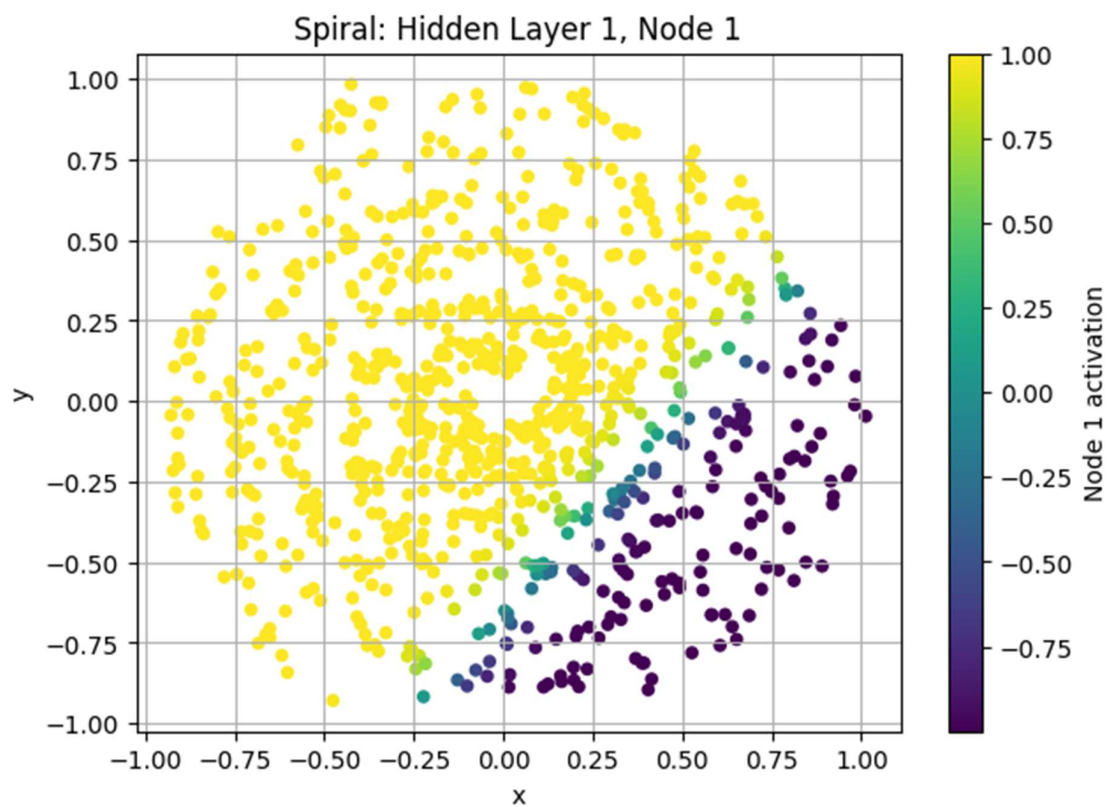
Appendix A — Key Implementation Logic

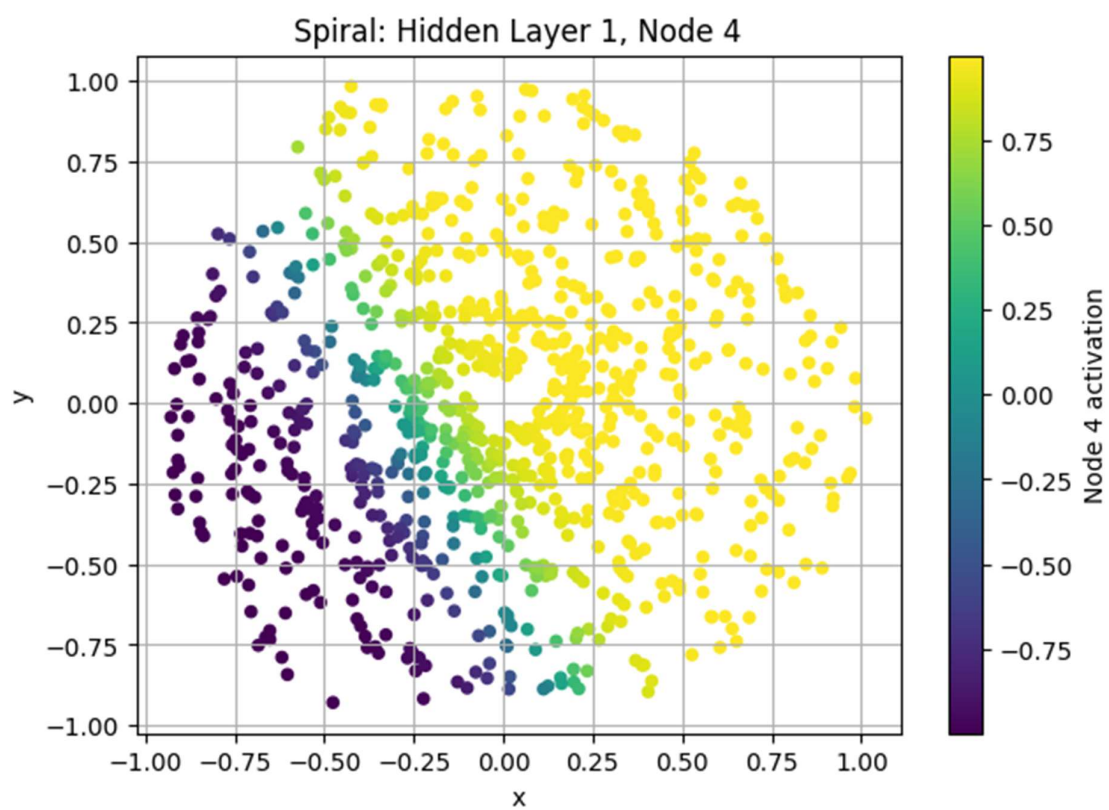
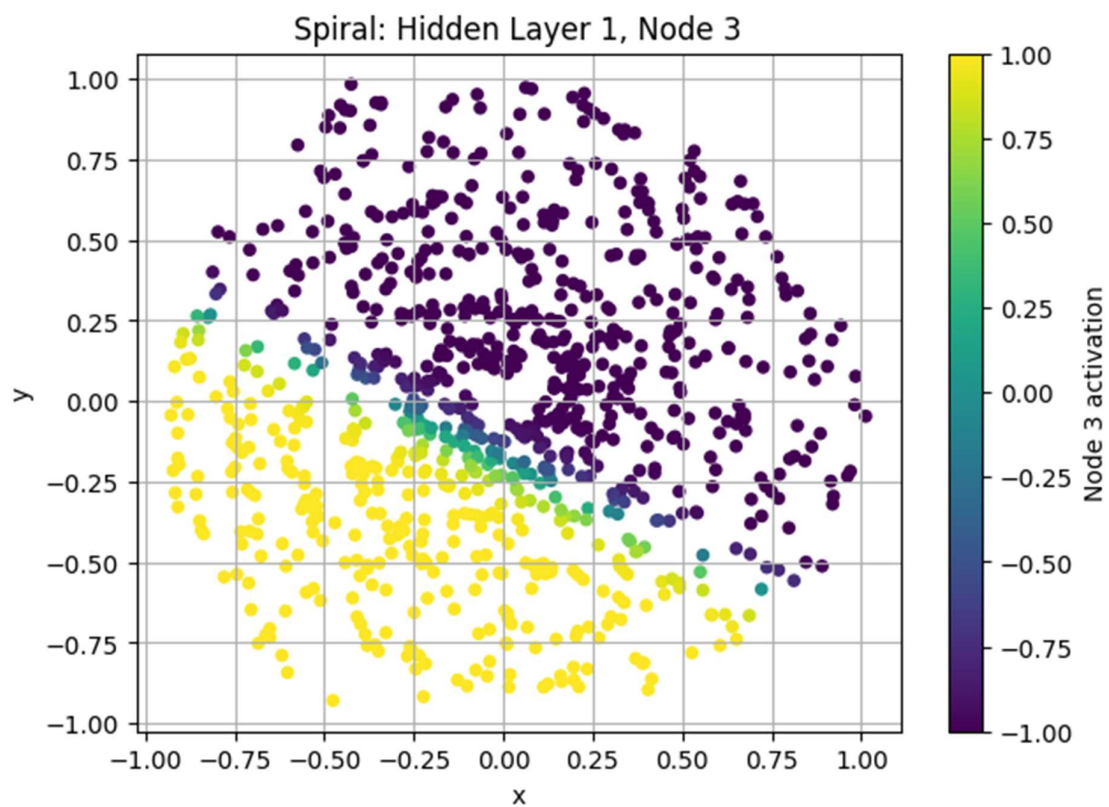
The core implementation follows the sequence below:

1. Generate X and y for each synthetic dataset.
2. Perform a class-wise 60/20/20 train-validation-test split.
3. Build FCNN weights and biases using Xavier-style initialization.
4. Forward pass: affine transformation $\rightarrow$ tanh hidden layers $\rightarrow$ softmax output.
5. Compute mean squared error against one-hot targets.
6. Backpropagate the error manually through all layers.
7. Update weights and biases using SGD.
8. Repeat for the requested number of epochs.
9. Evaluate validation accuracy for every candidate architecture.
10. Select the architecture with maximum validation accuracy.
11. Evaluate the selected model on the test set.
12. Plot errors, decision regions, confusion matrices and node outputs.

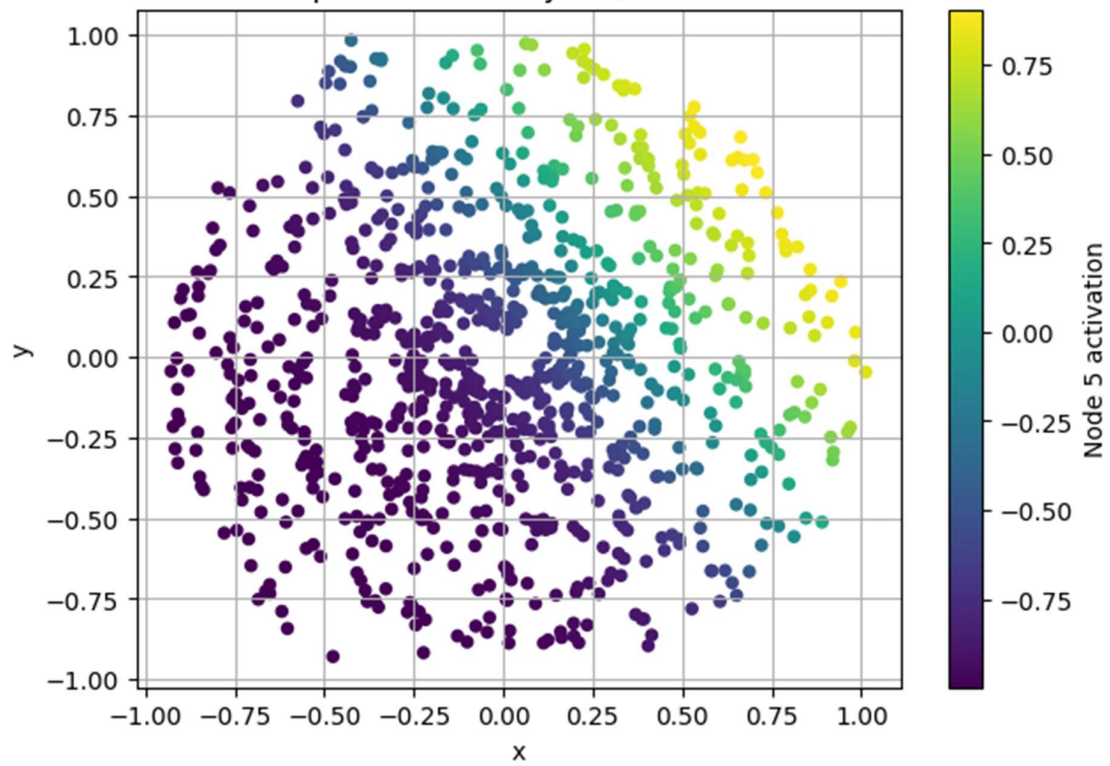
Appendix B — Final Performance Table

Dataset	Model	Accuracy
Gaussian	Single Neuron	96.33%
Gaussian	FCNN (4,)	100.00%
Spiral	Single Neuron	33.33%
Spiral	FCNN (32,32)	92.33%

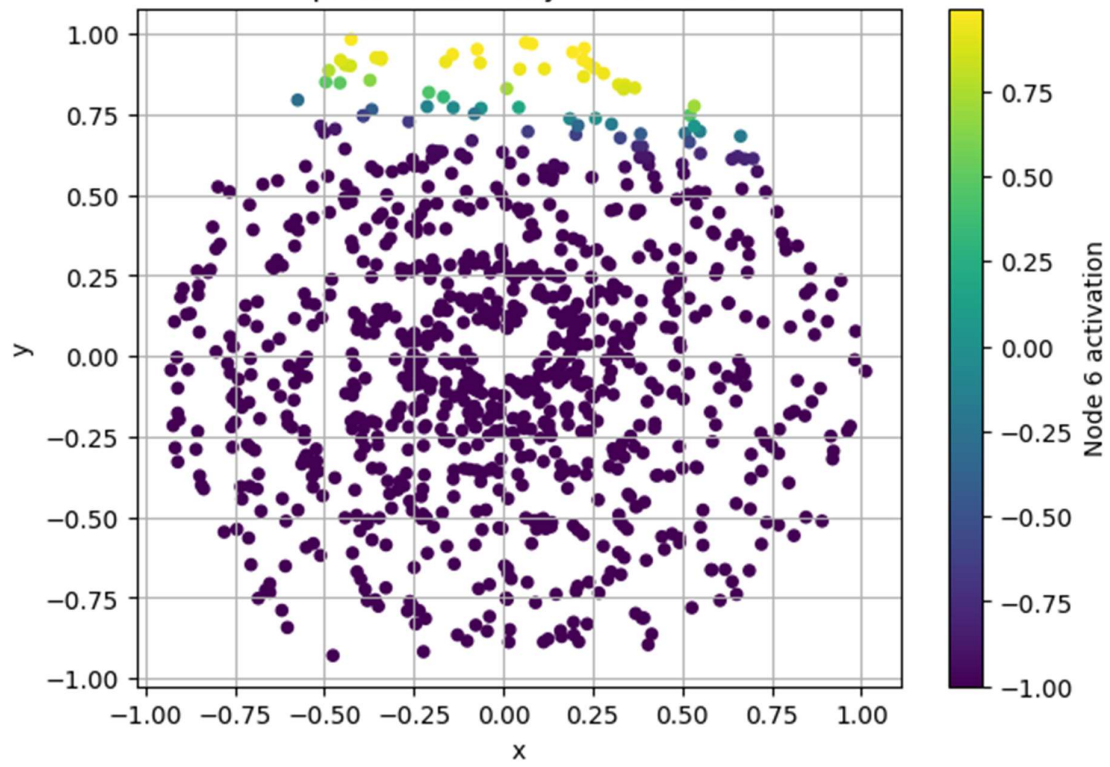




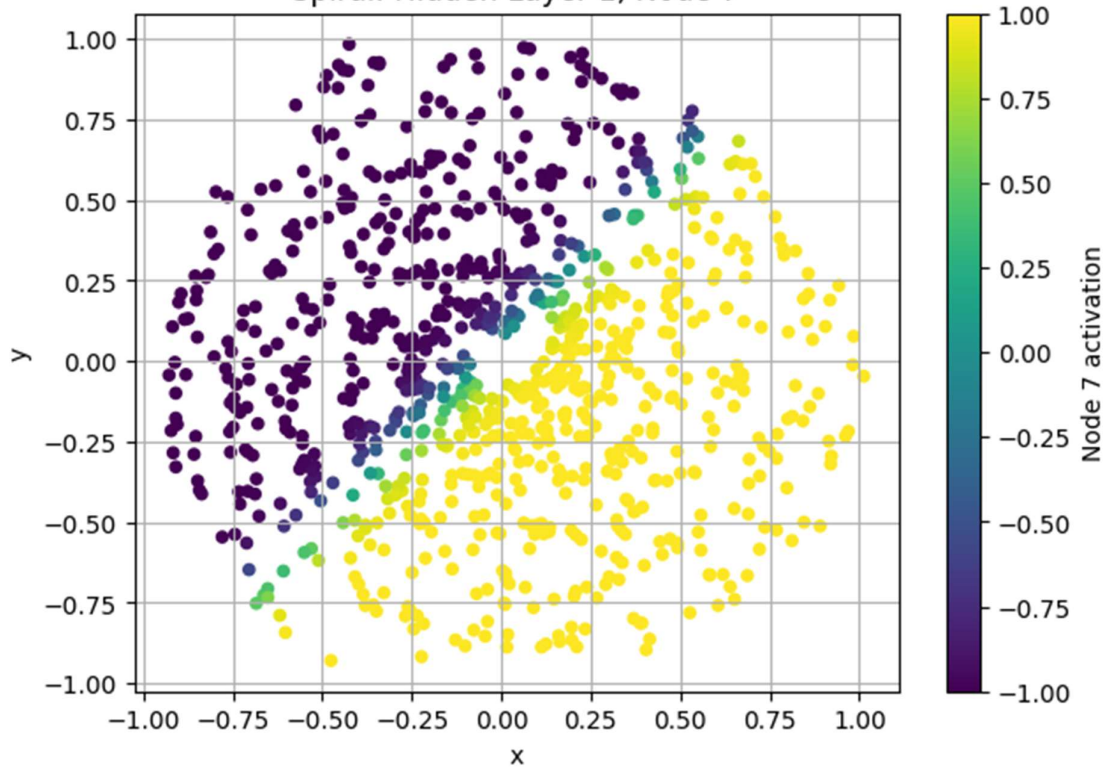
Spiral: Hidden Layer 1, Node 5



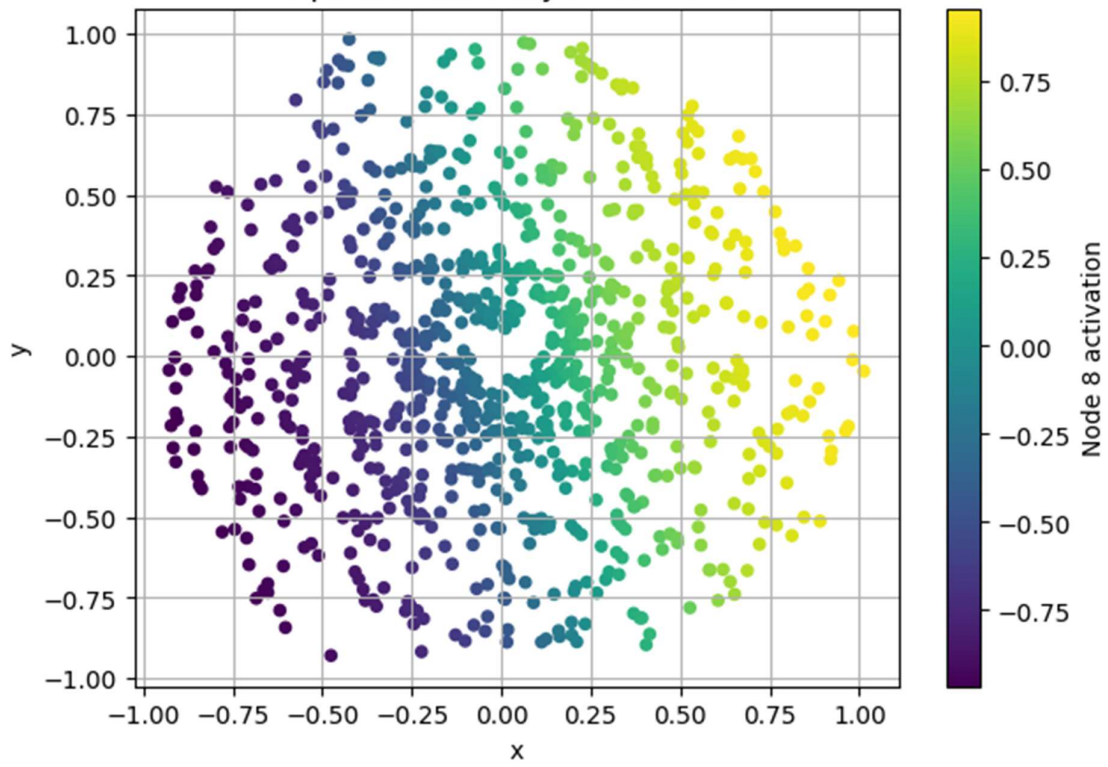
Spiral: Hidden Layer 1, Node 6



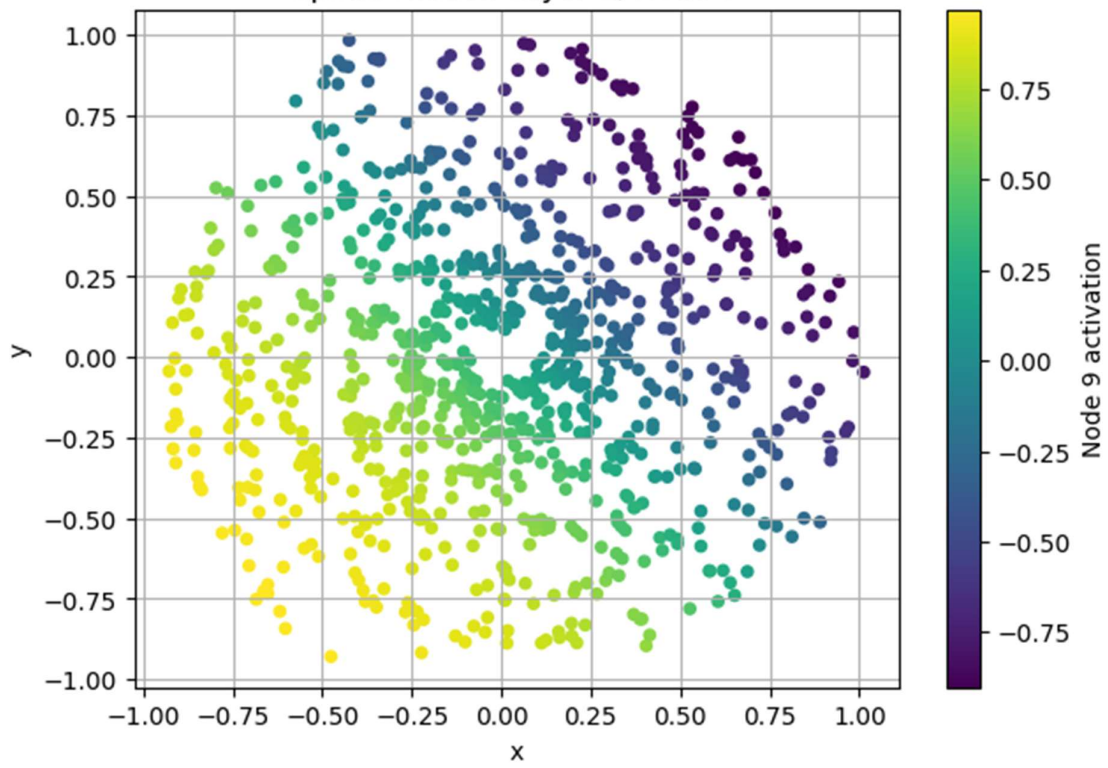
Spiral: Hidden Layer 1, Node 7



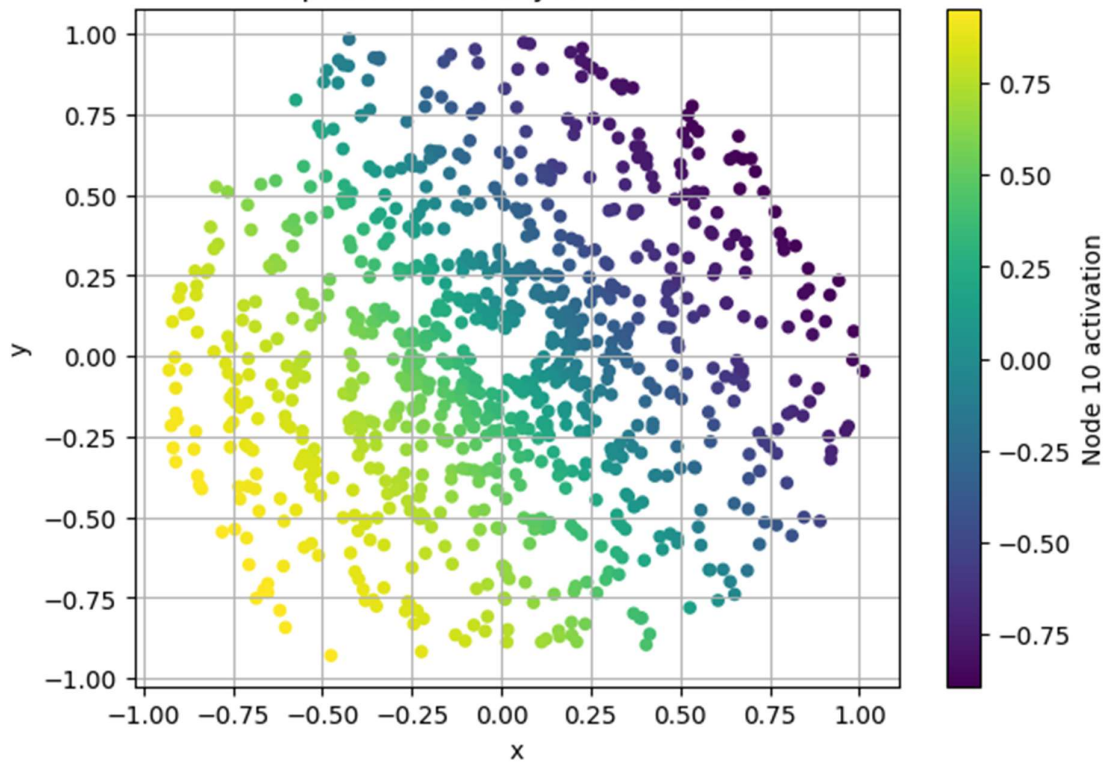
Spiral: Hidden Layer 1, Node 8

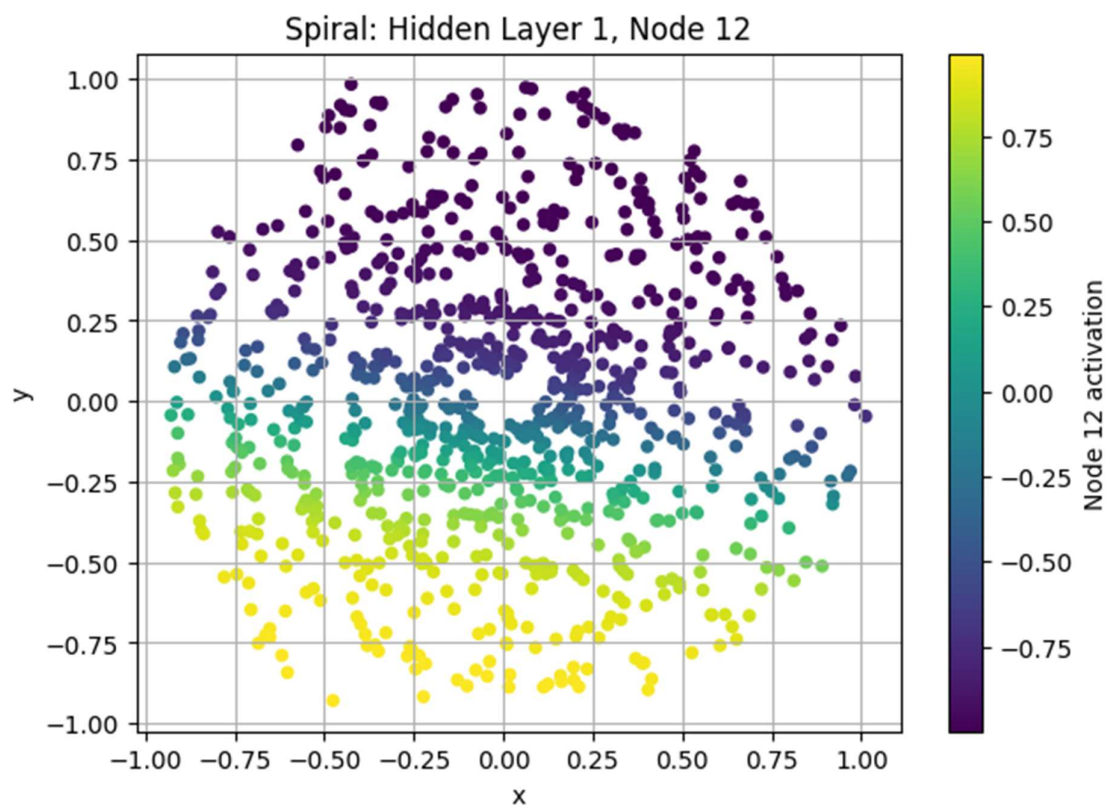
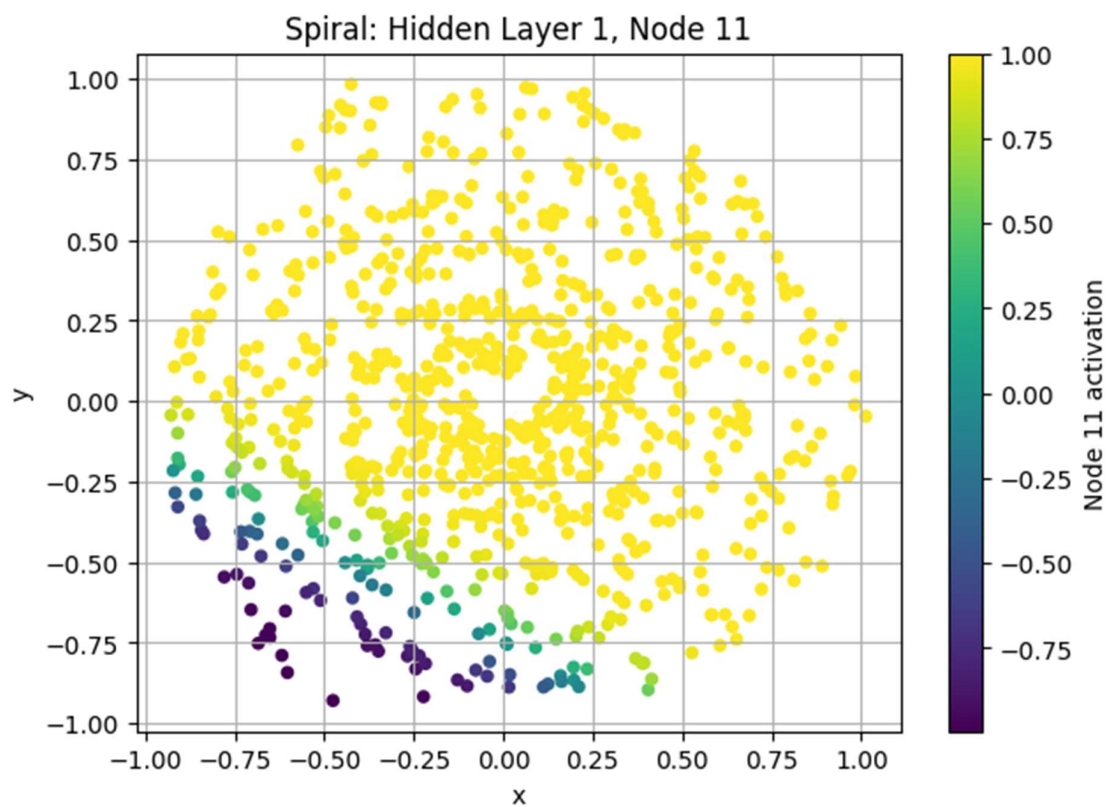


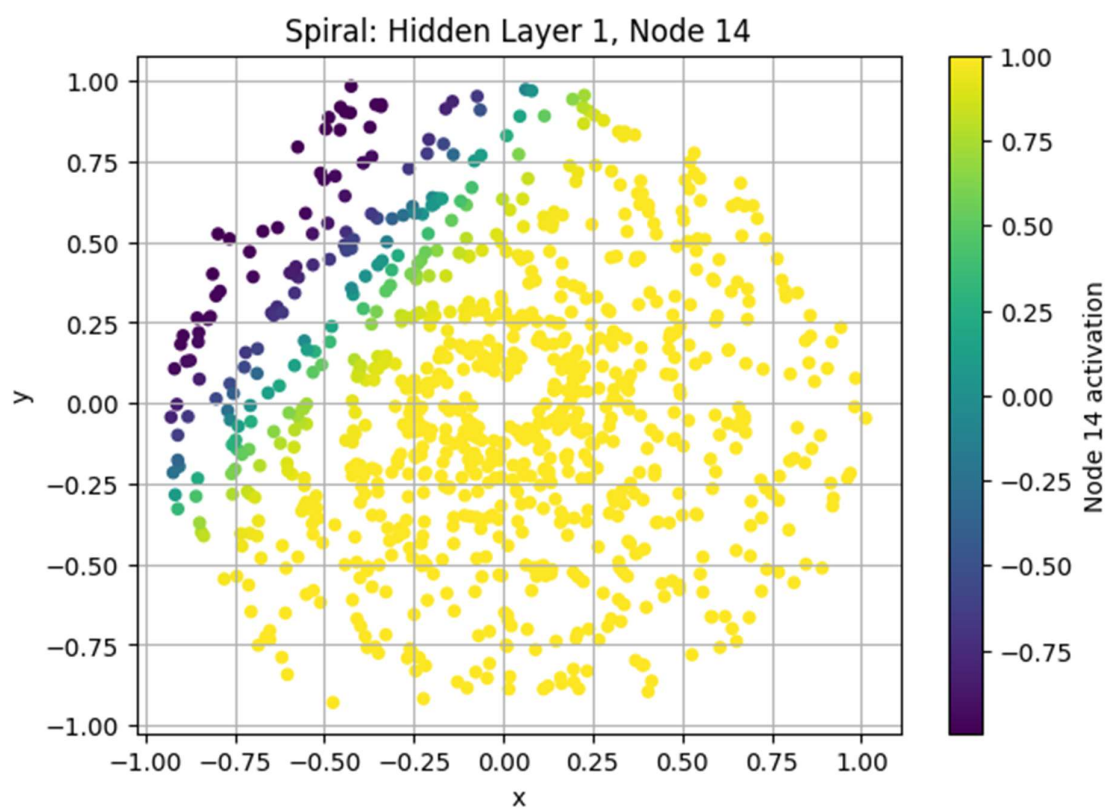
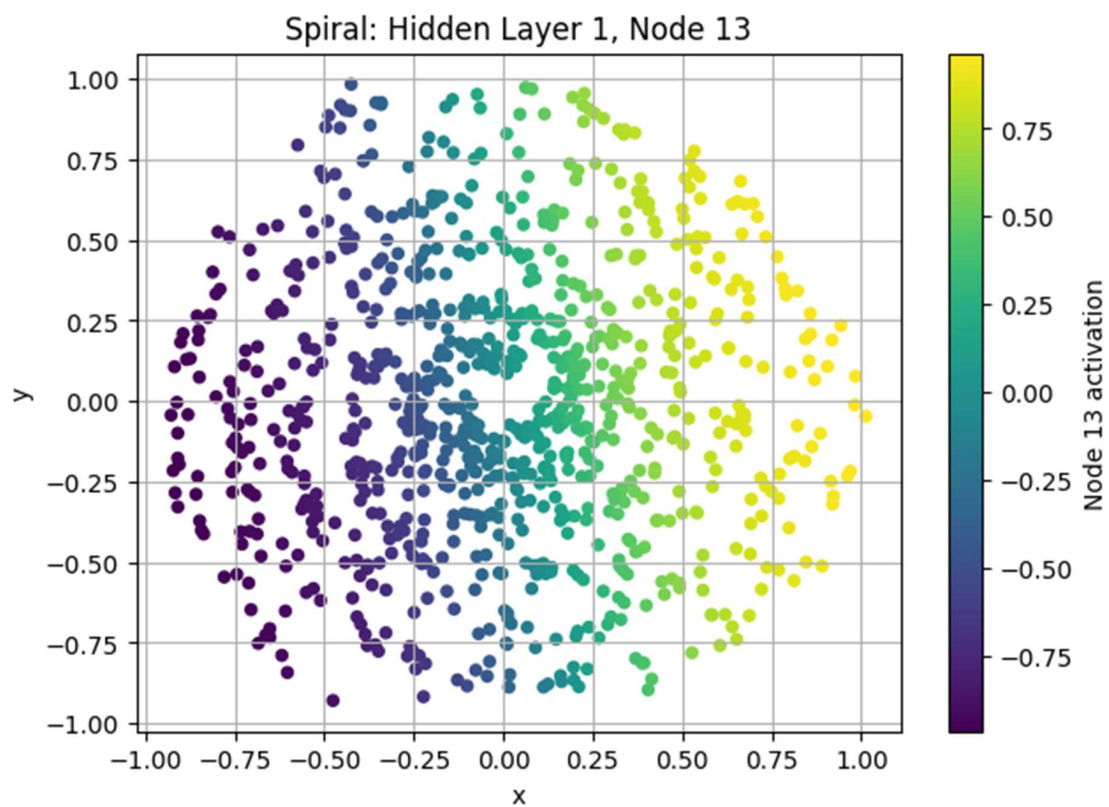
Spiral: Hidden Layer 1, Node 9

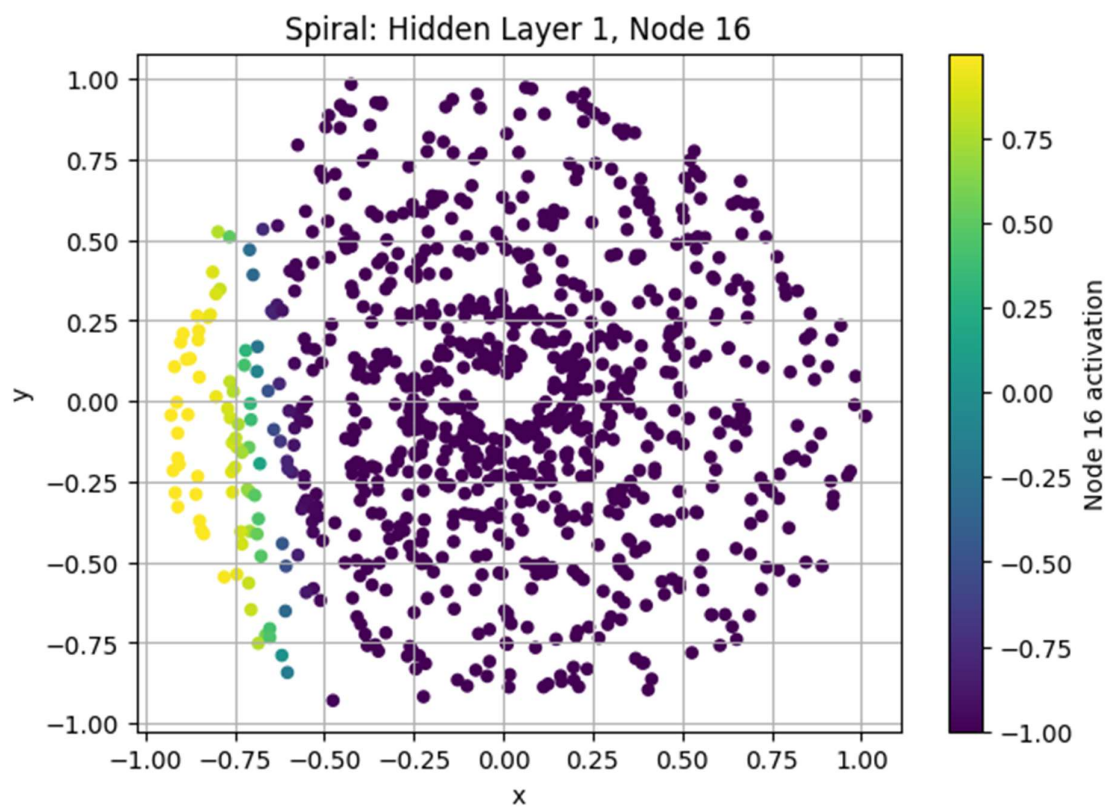
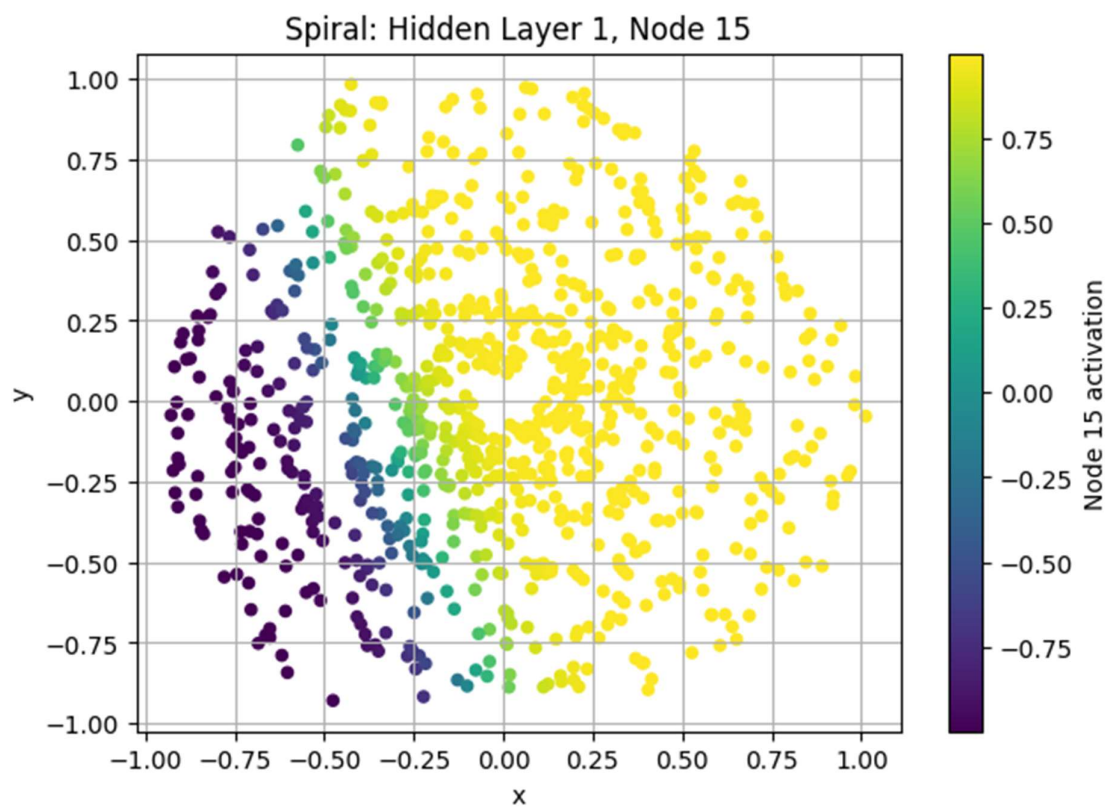


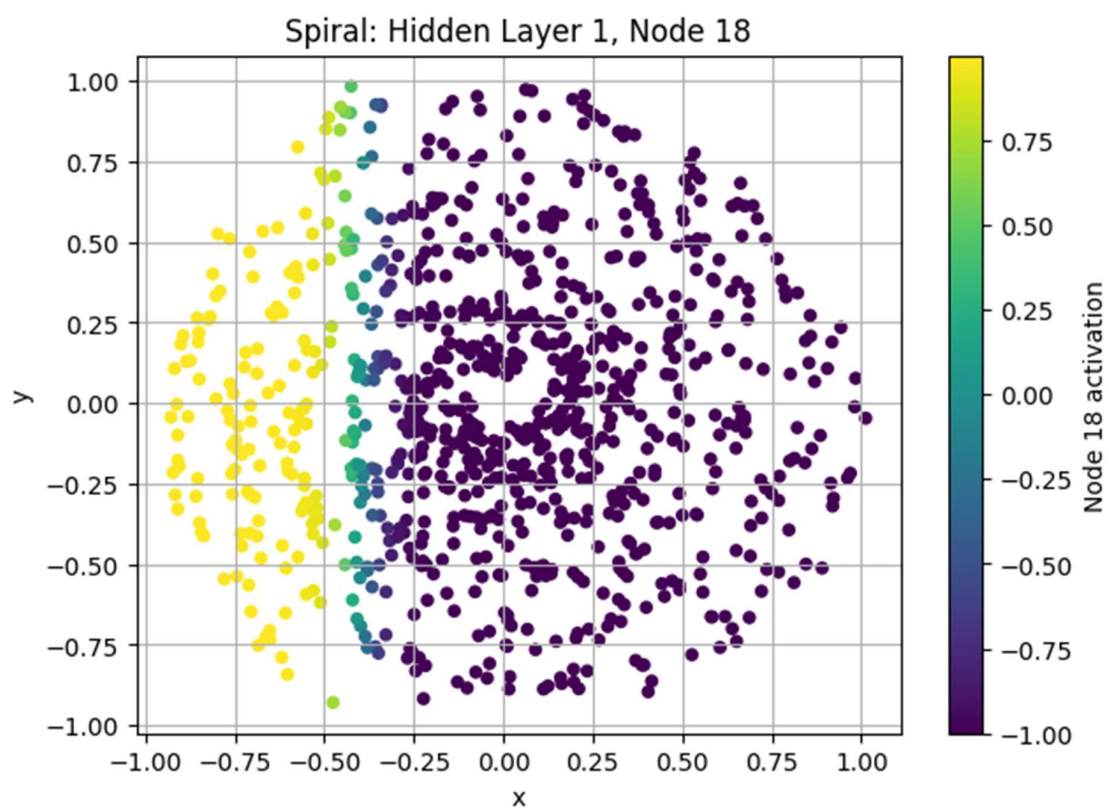
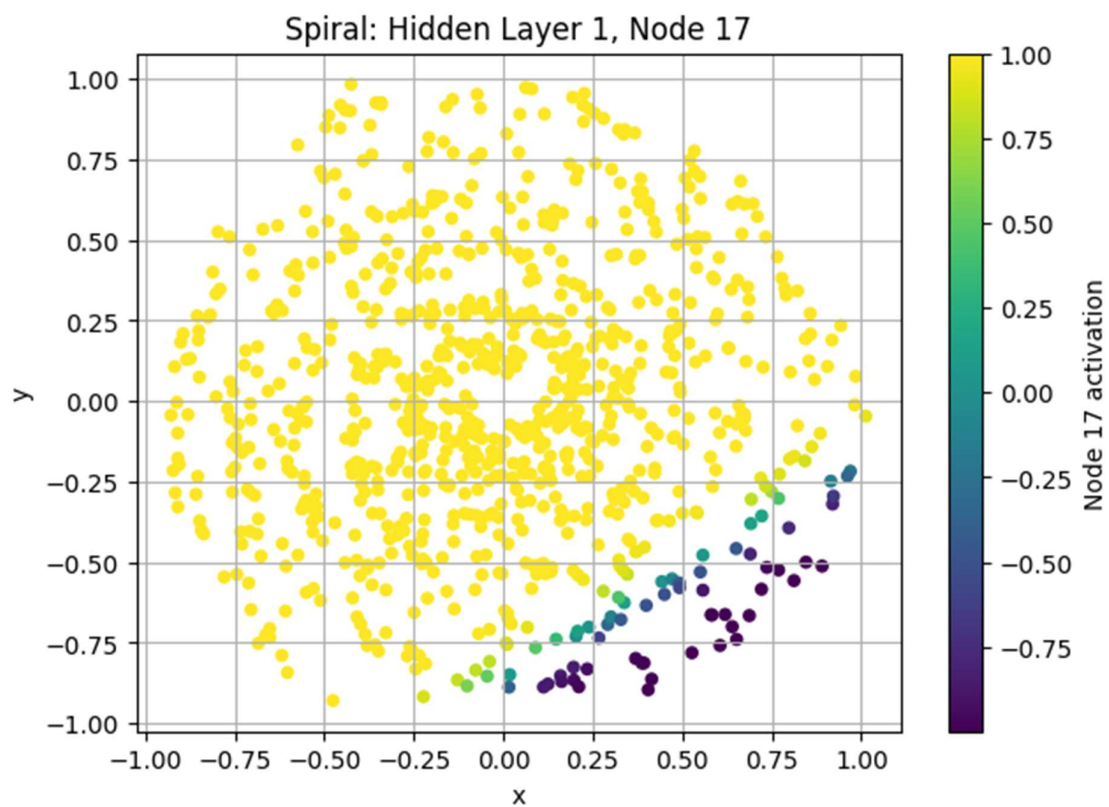
Spiral: Hidden Layer 1, Node 10

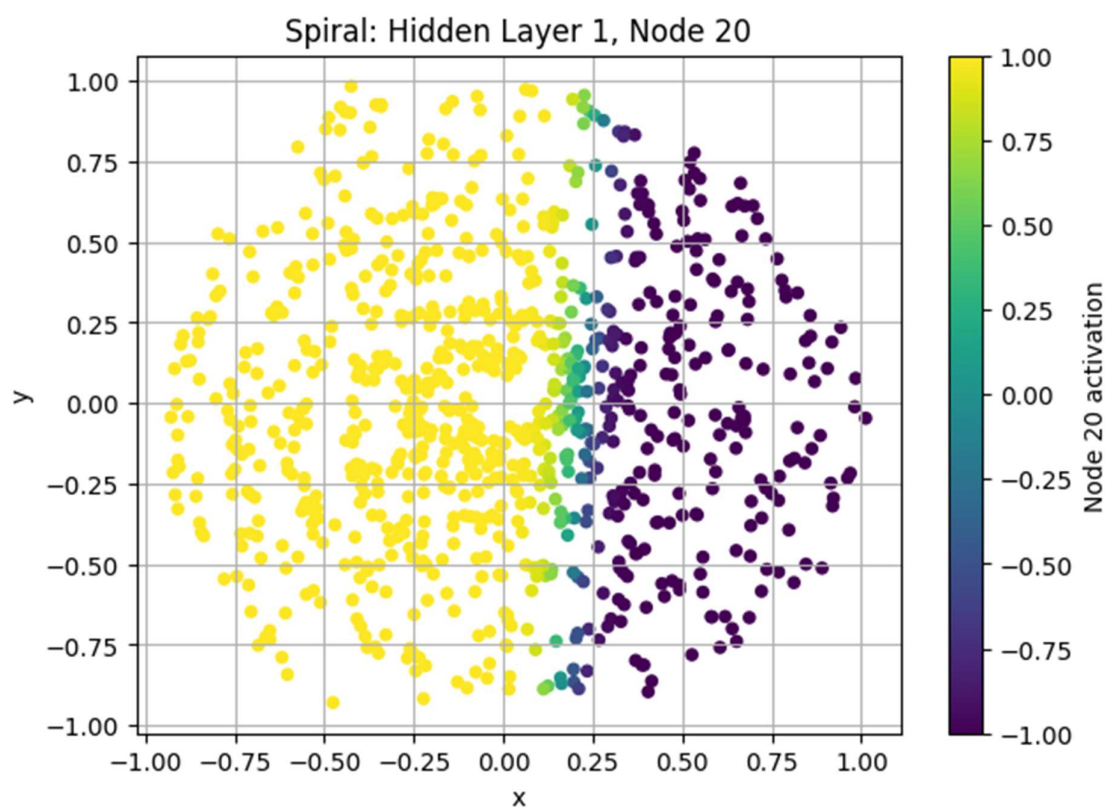
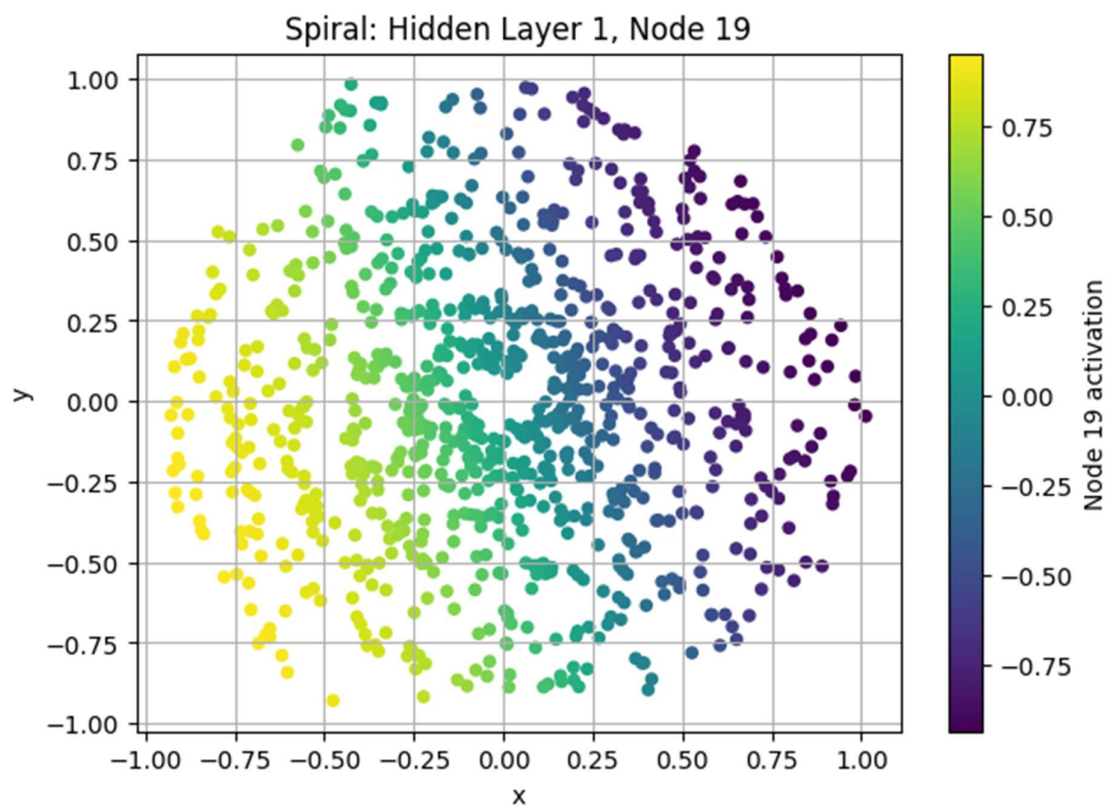




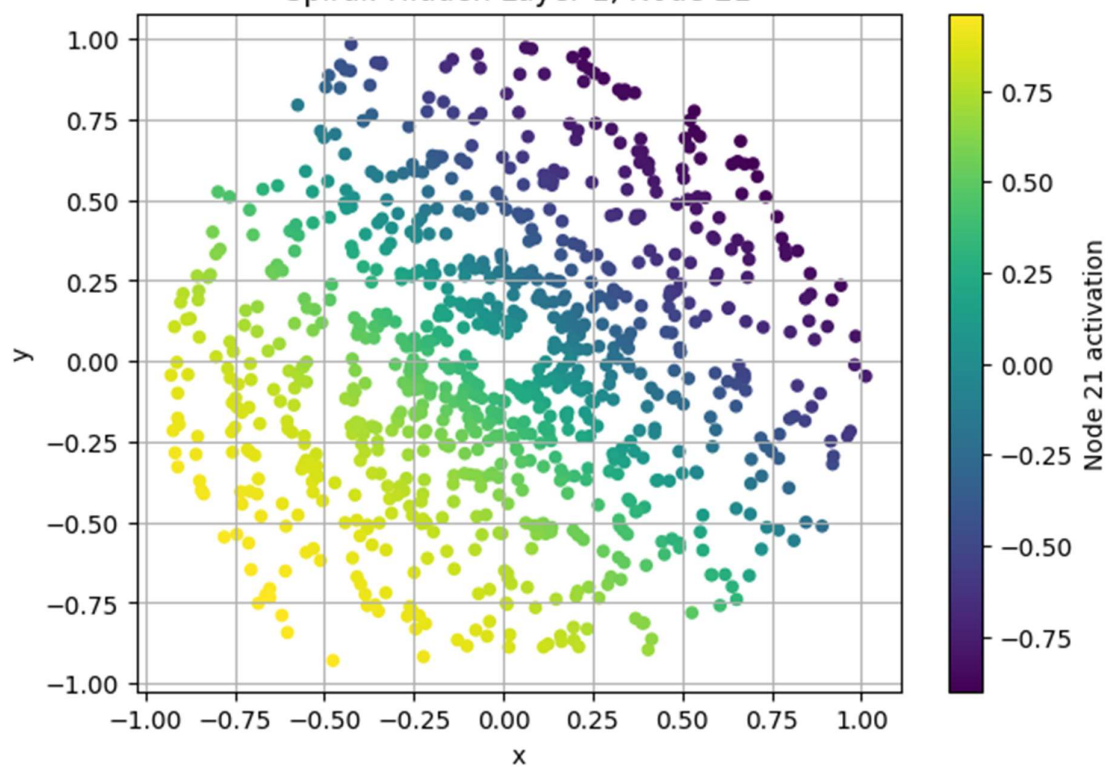




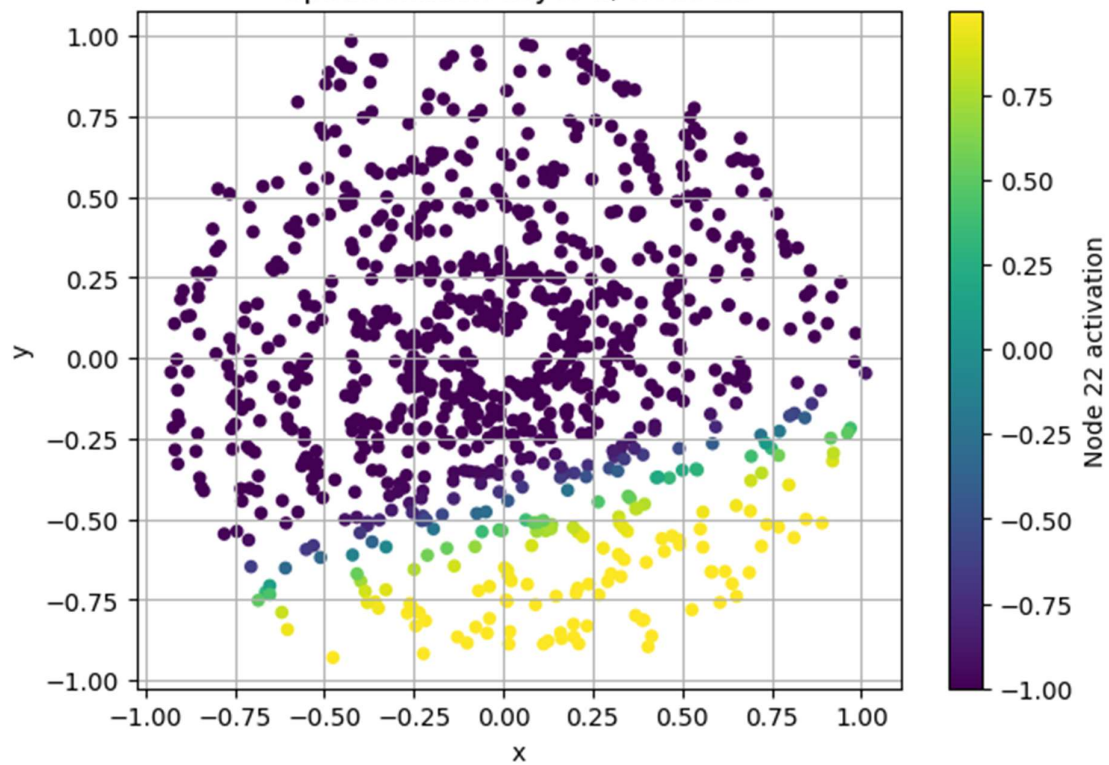


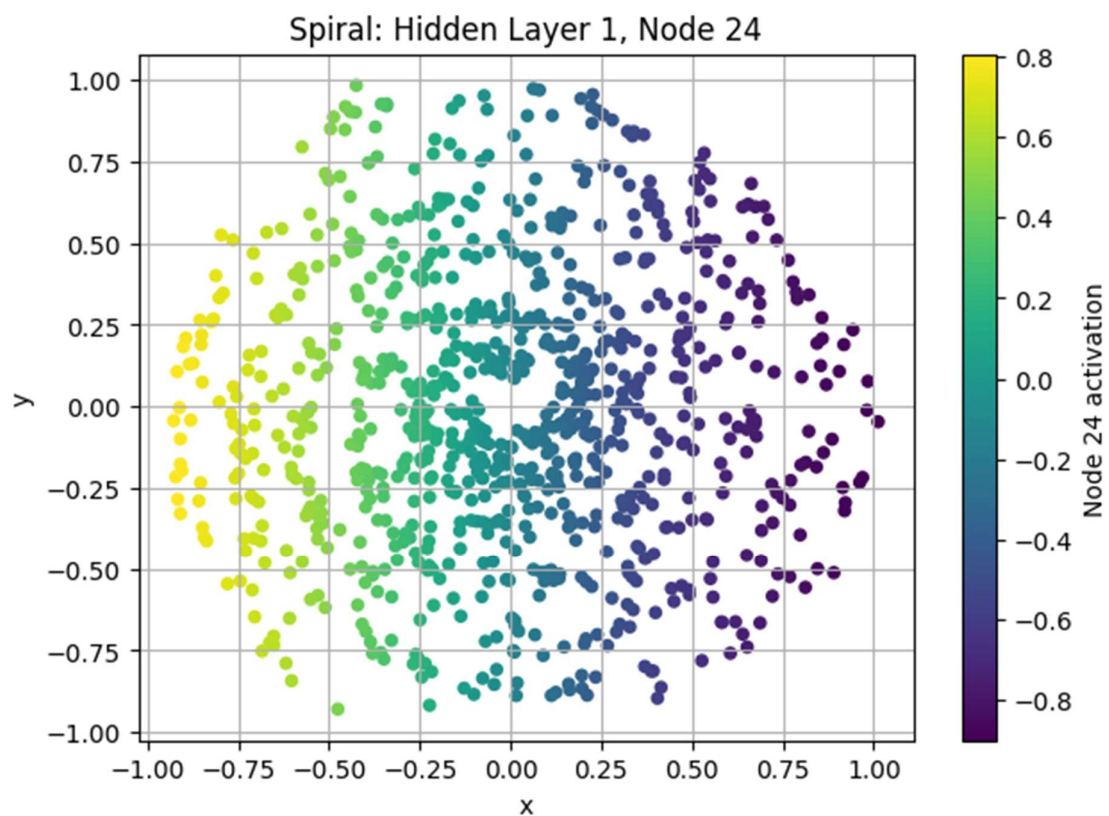
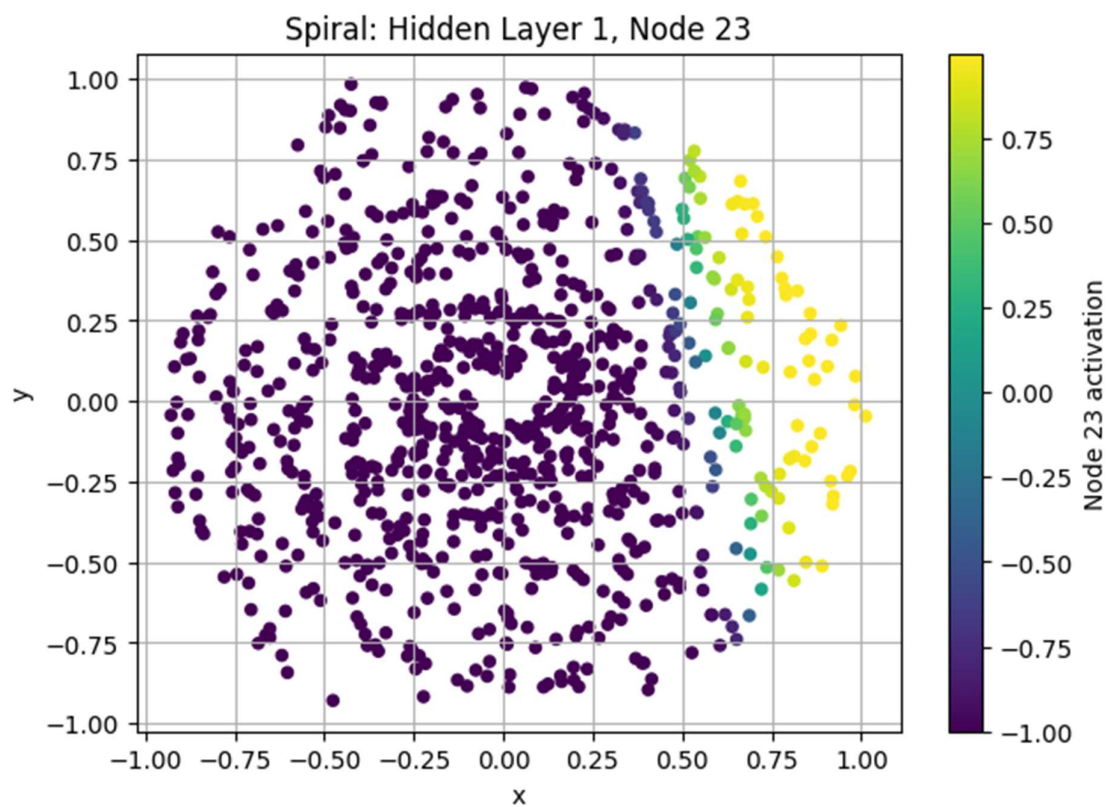


Spiral: Hidden Layer 1, Node 21

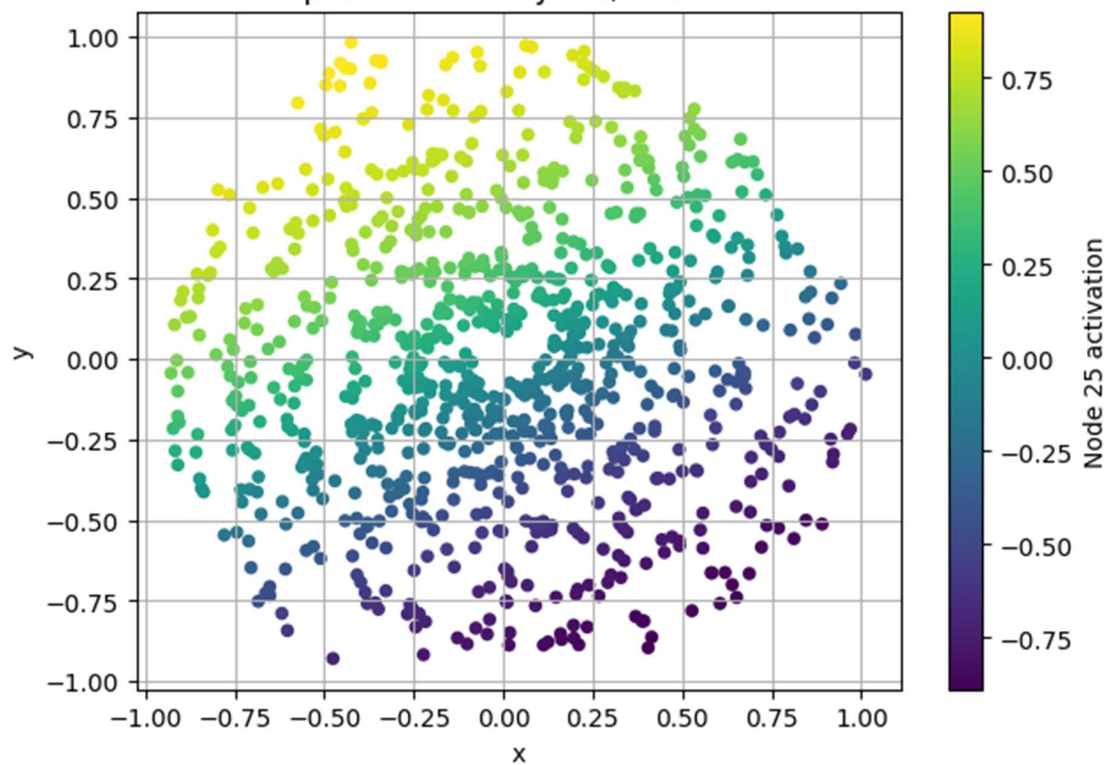


Spiral: Hidden Layer 1, Node 22

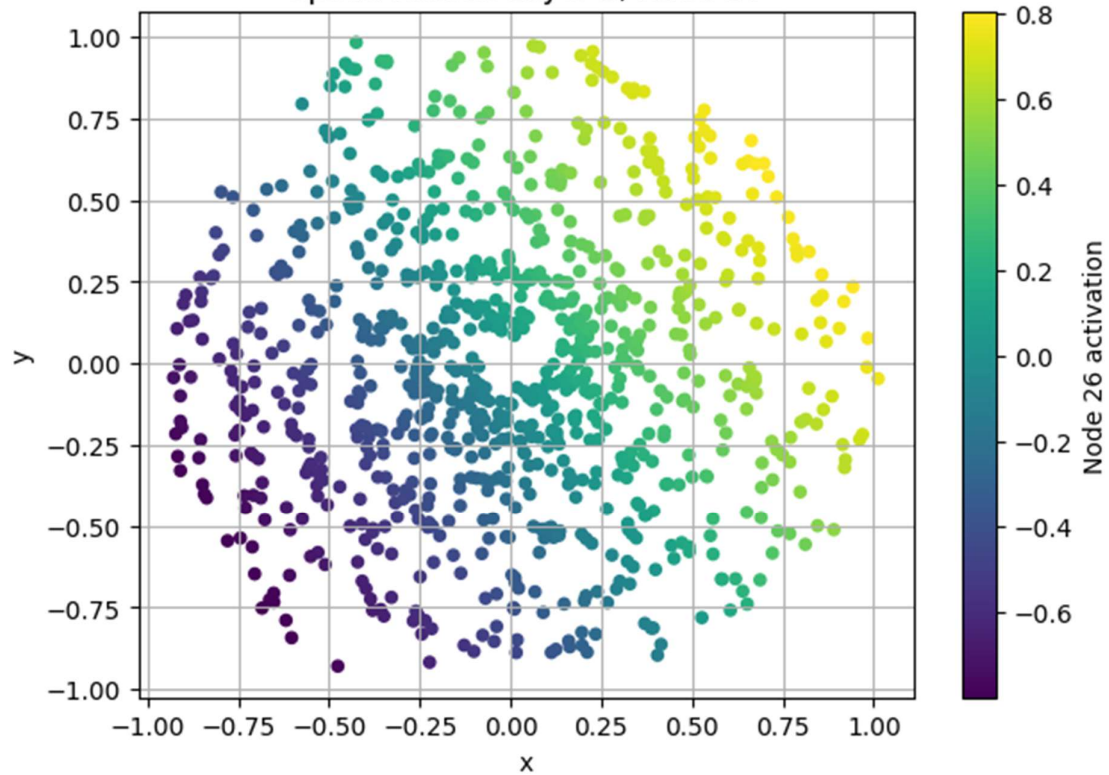


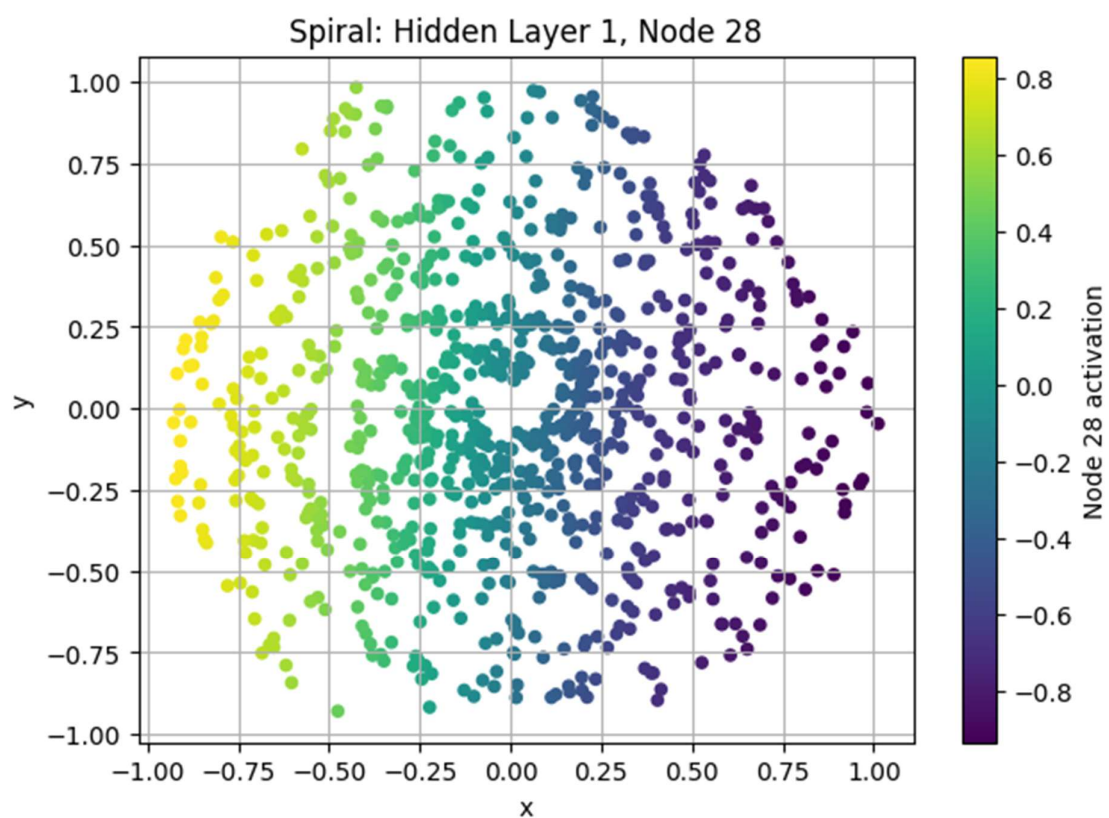
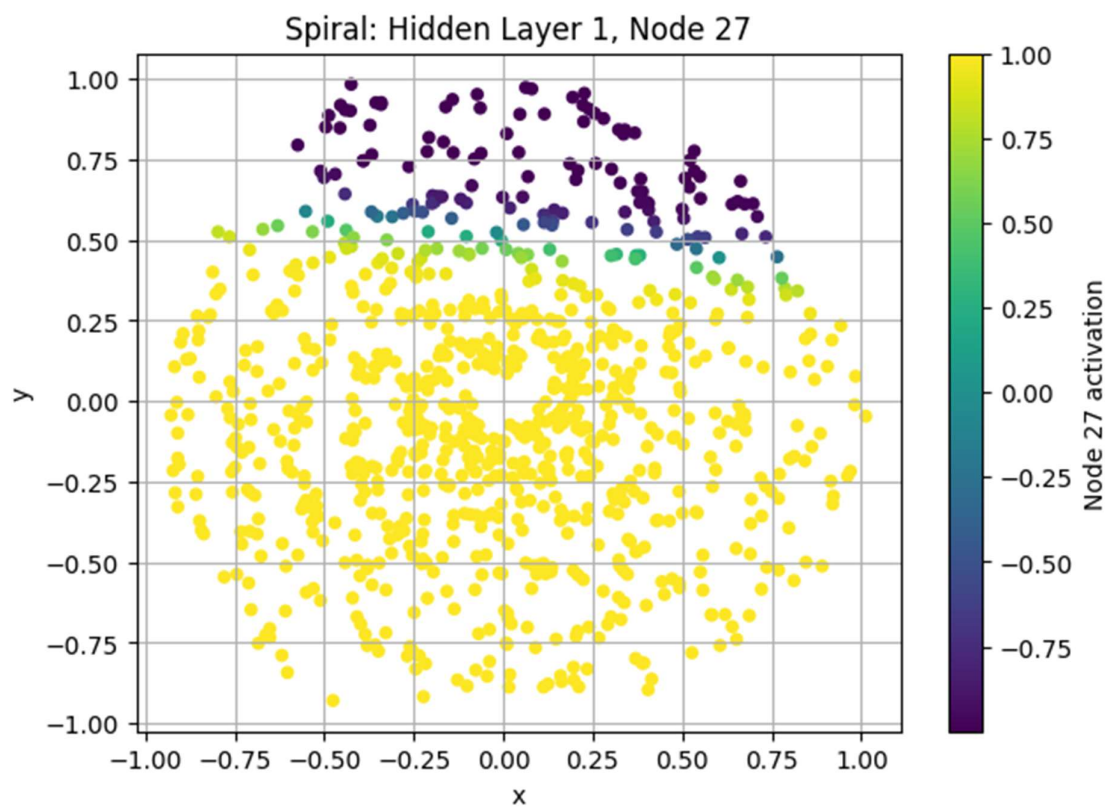


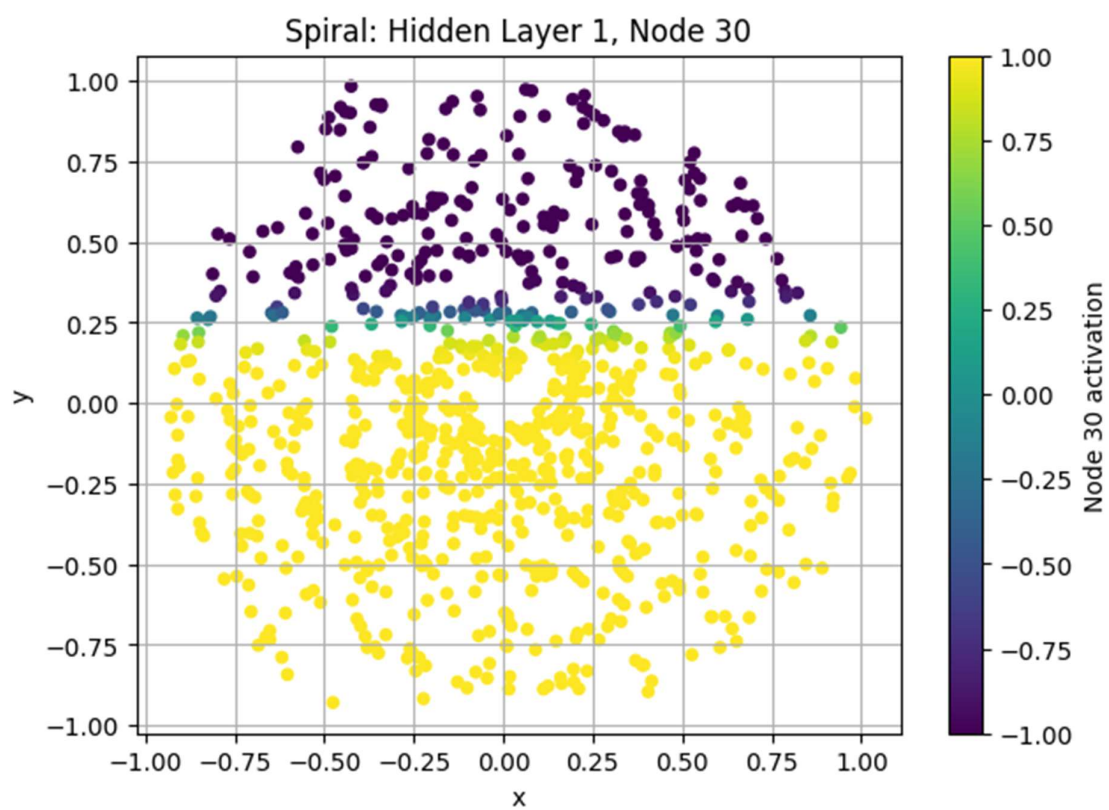
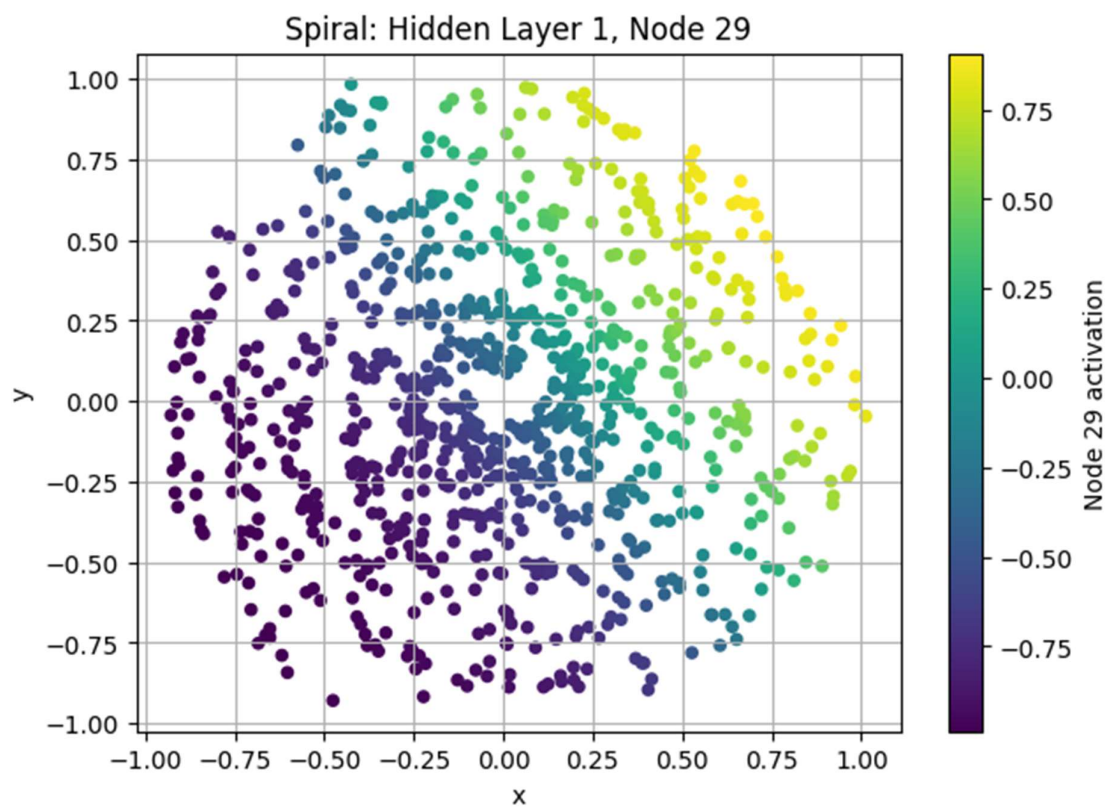
Spiral: Hidden Layer 1, Node 25

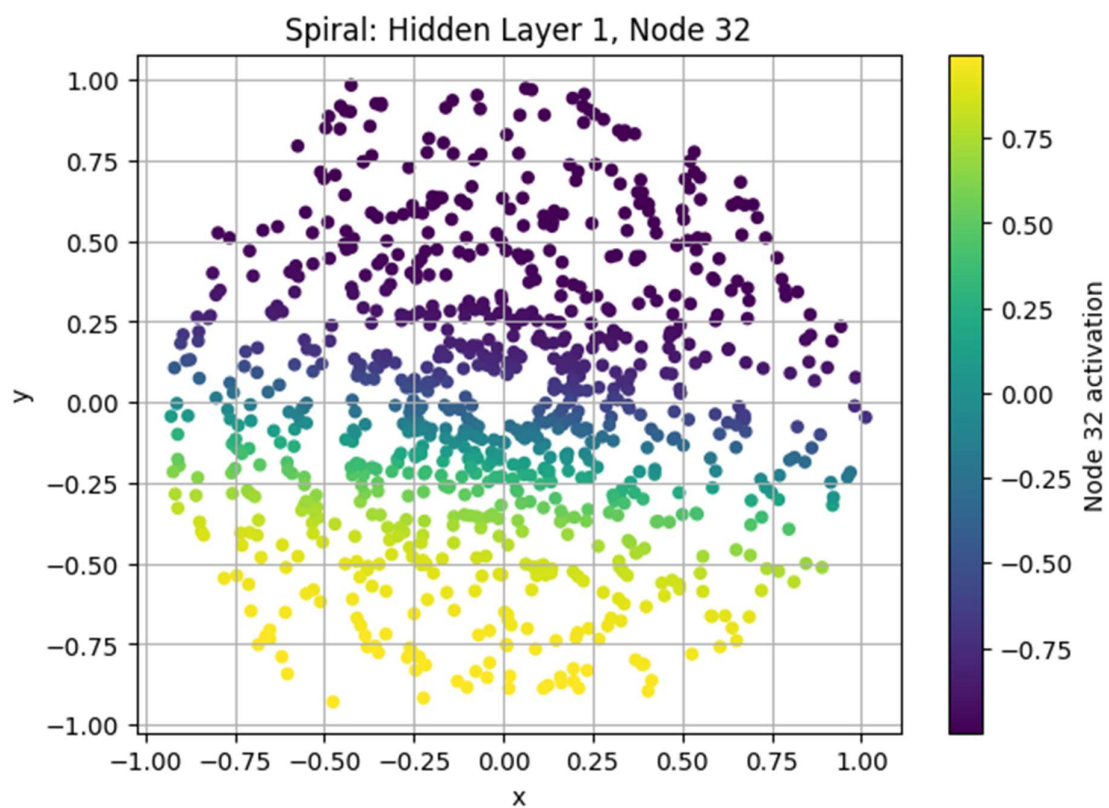
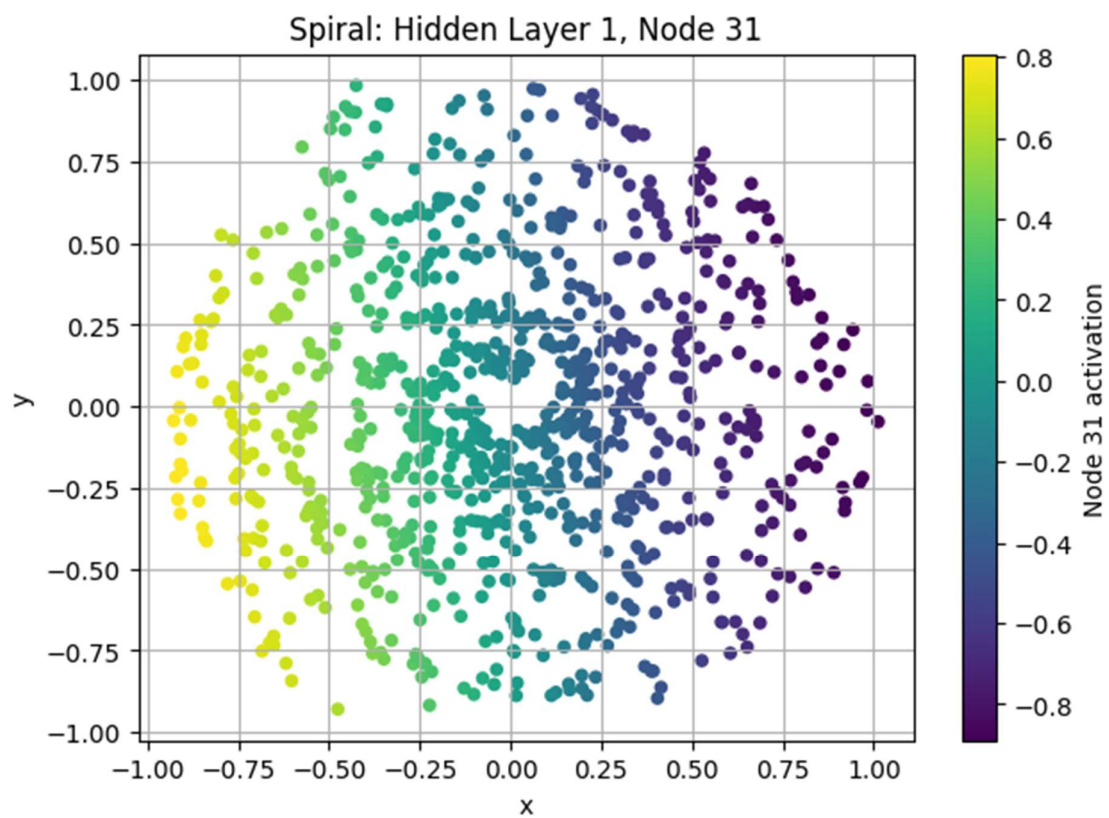


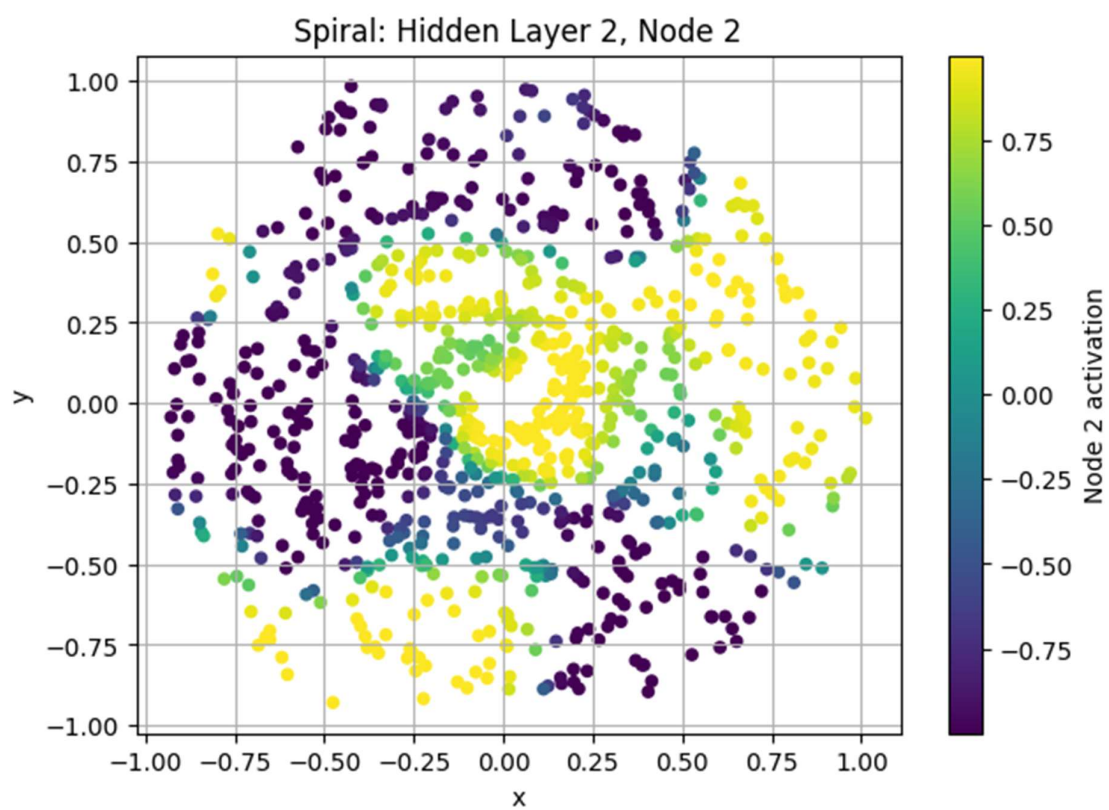
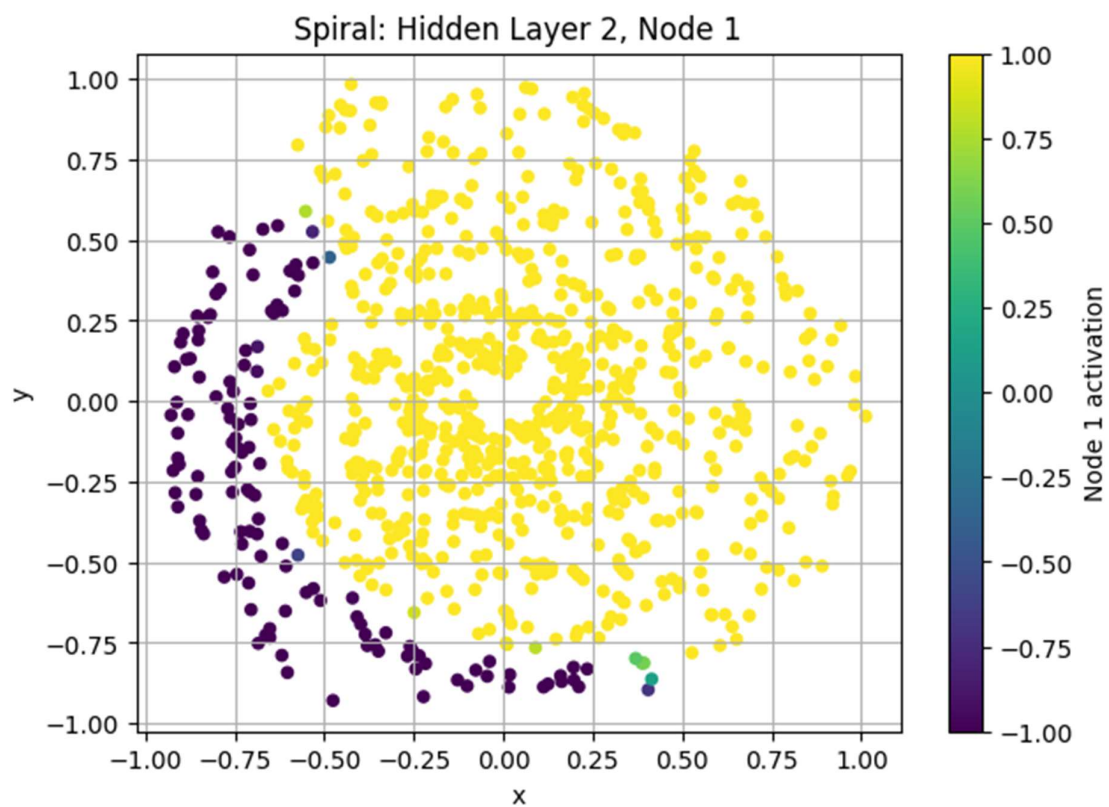
Spiral: Hidden Layer 1, Node 26



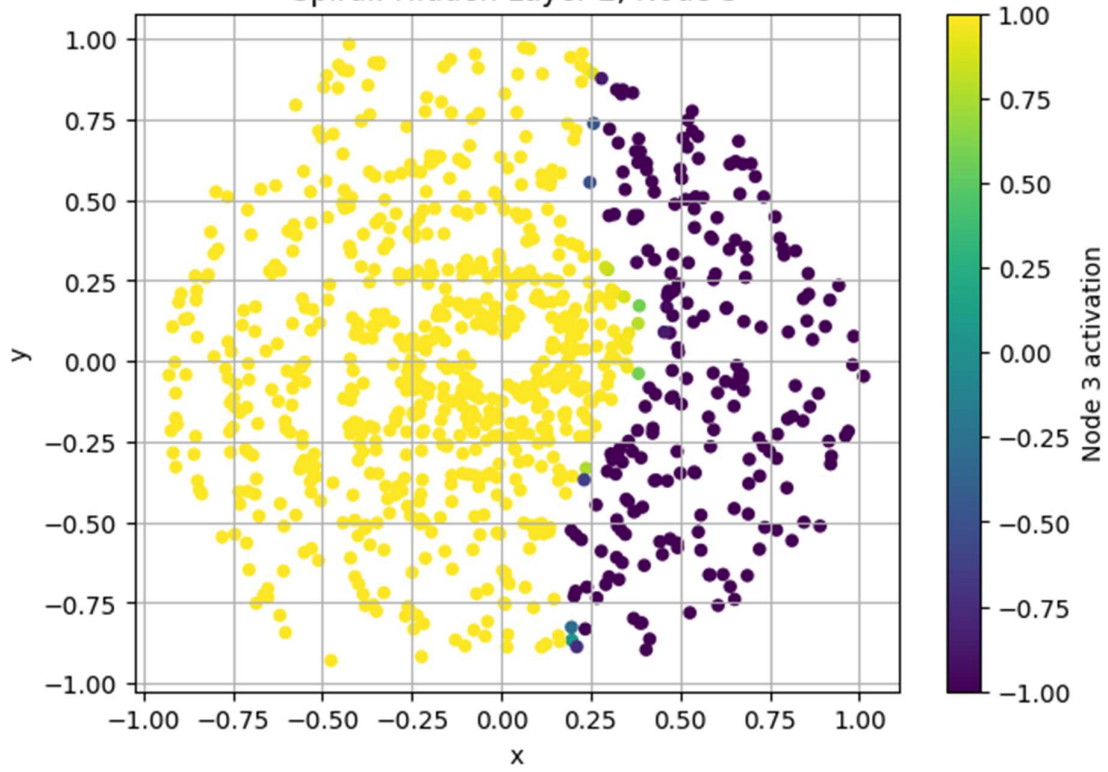




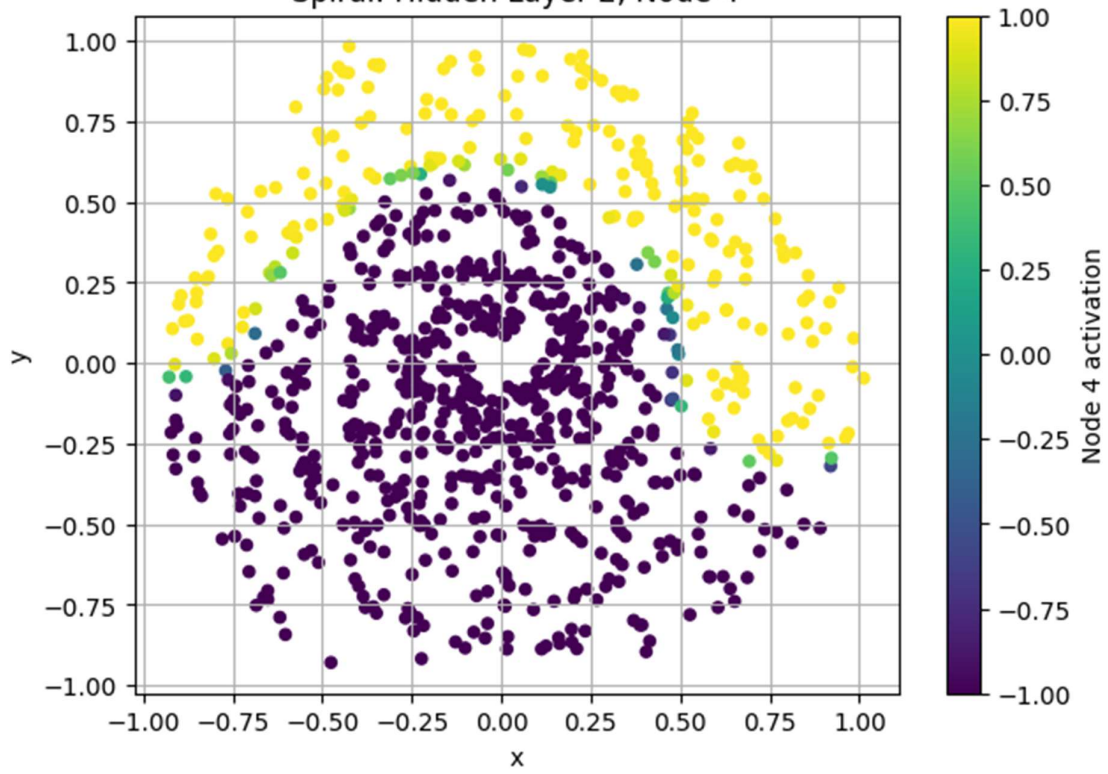




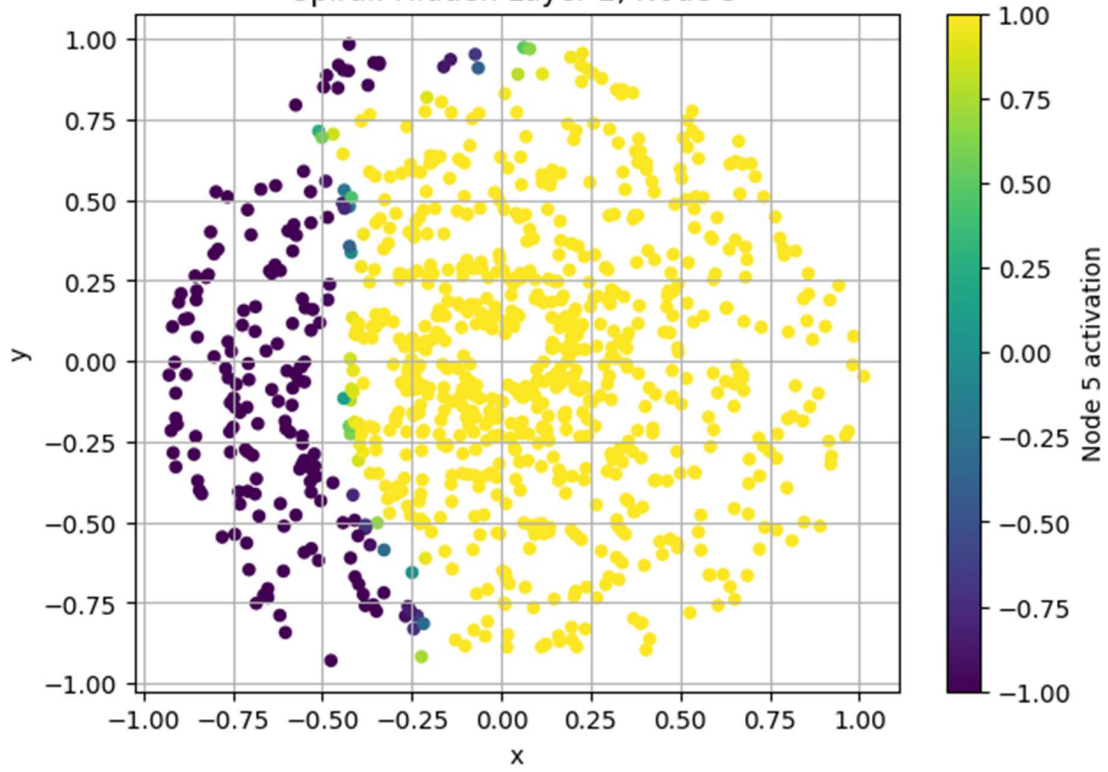
Spiral: Hidden Layer 2, Node 3



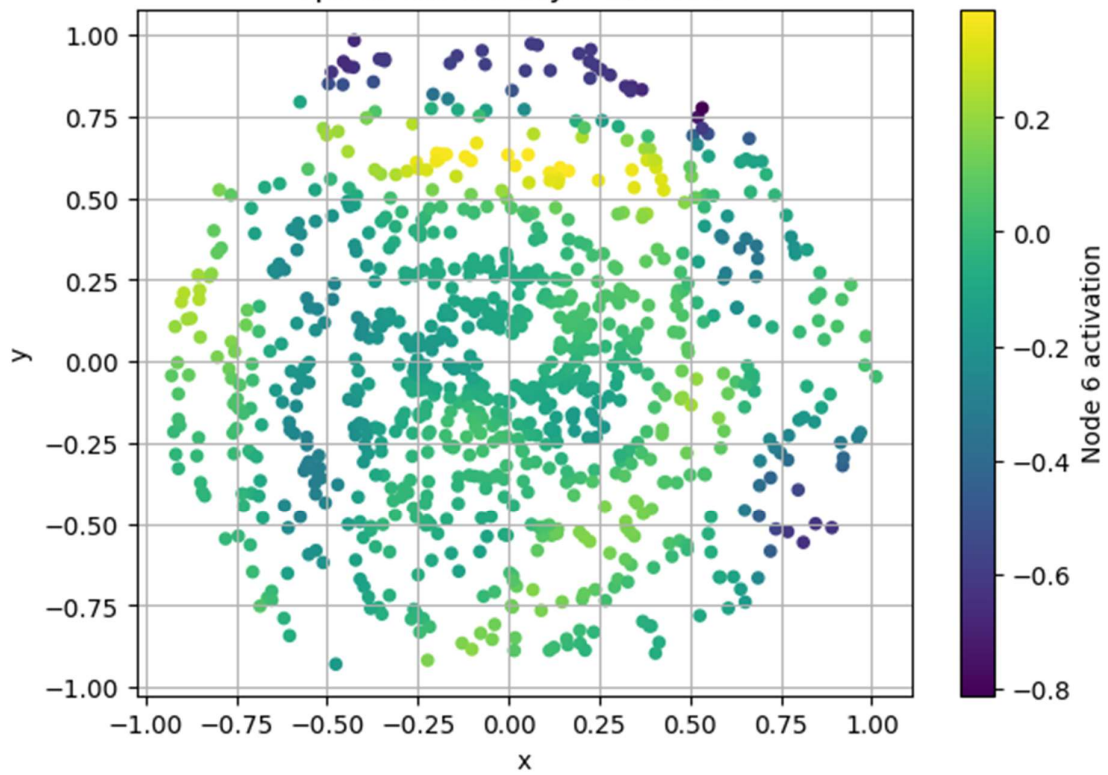
Spiral: Hidden Layer 2, Node 4

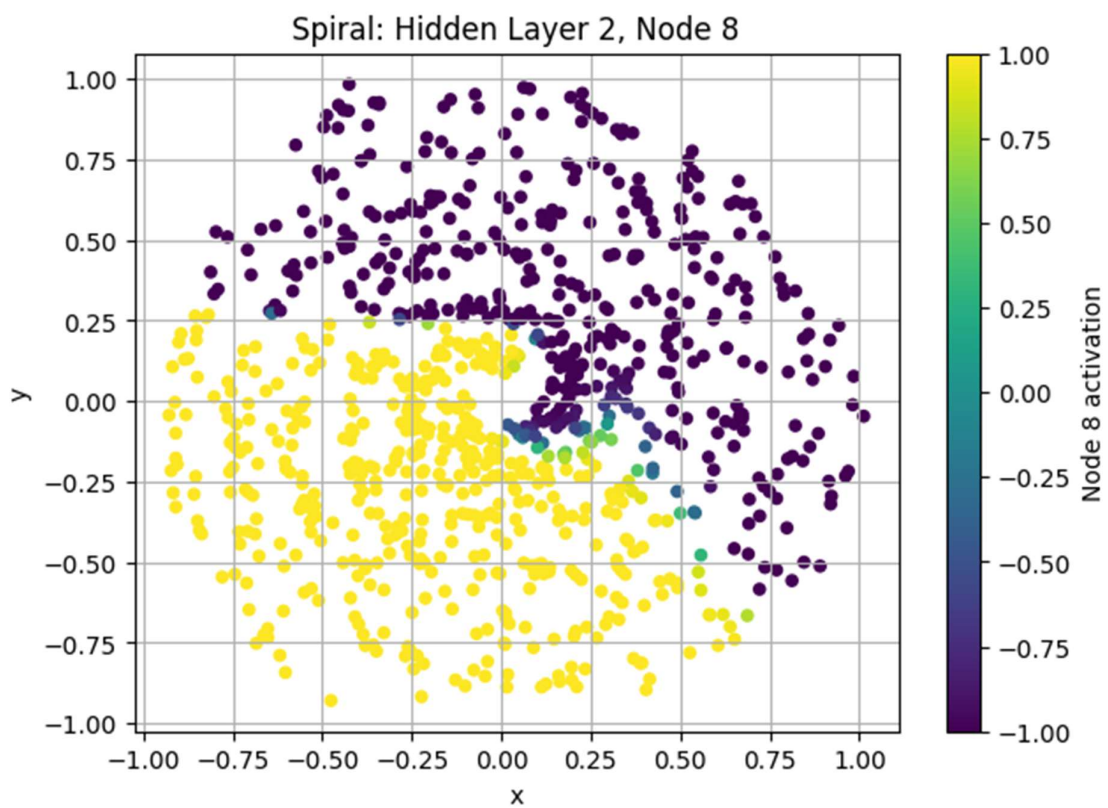
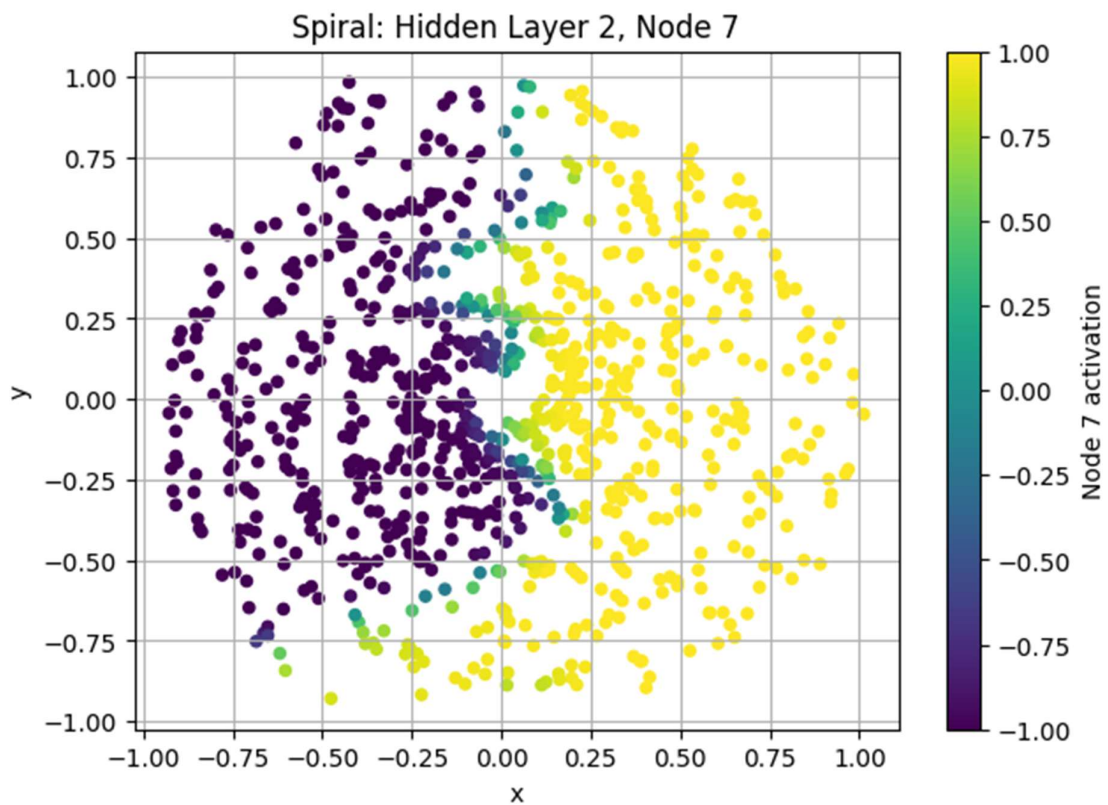


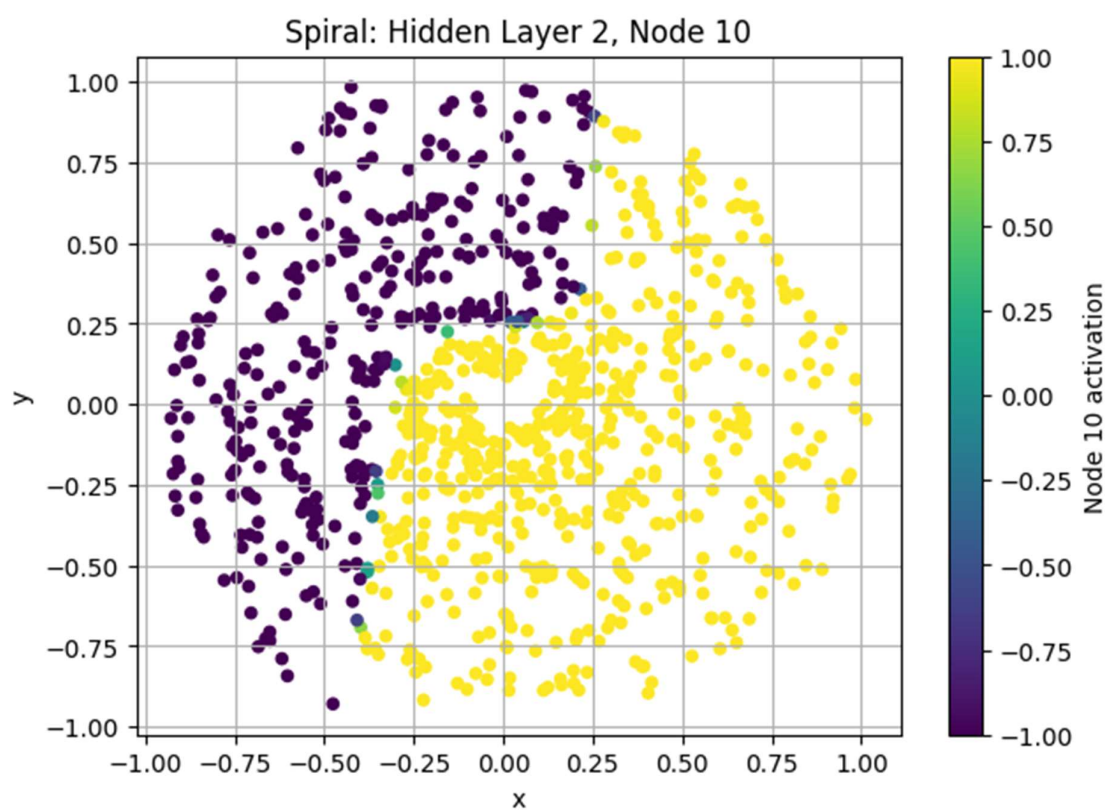
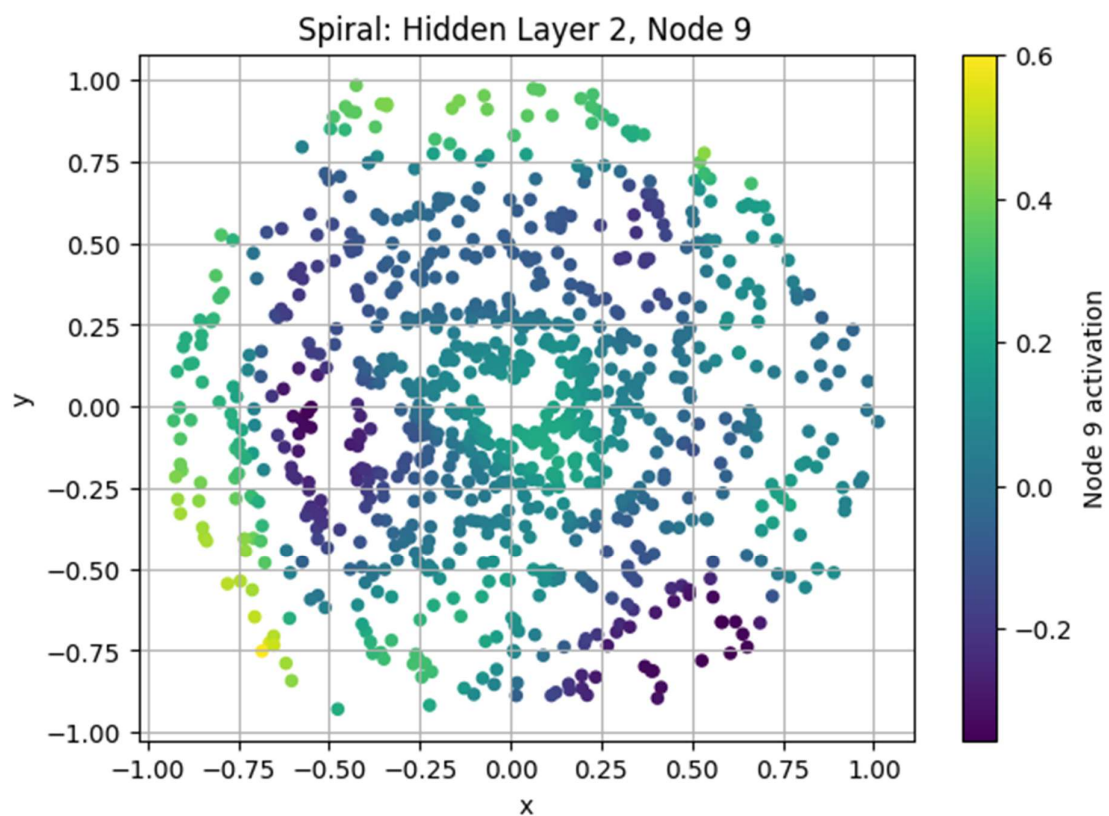
Spiral: Hidden Layer 2, Node 5



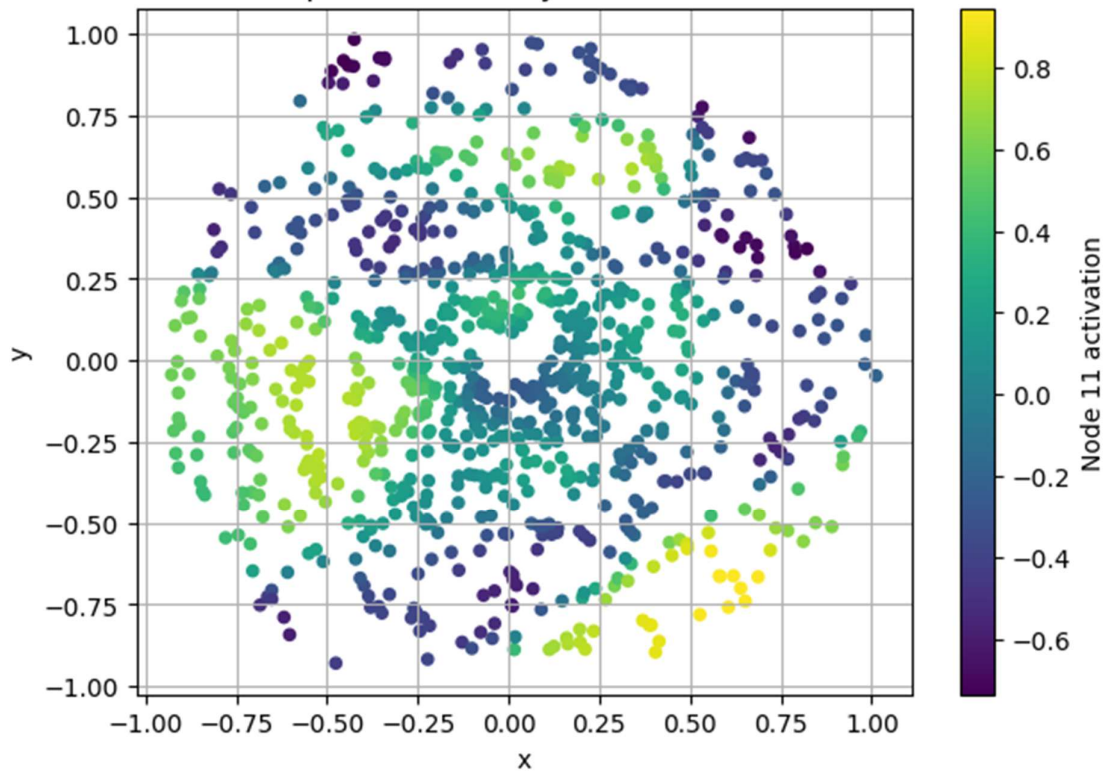
Spiral: Hidden Layer 2, Node 6



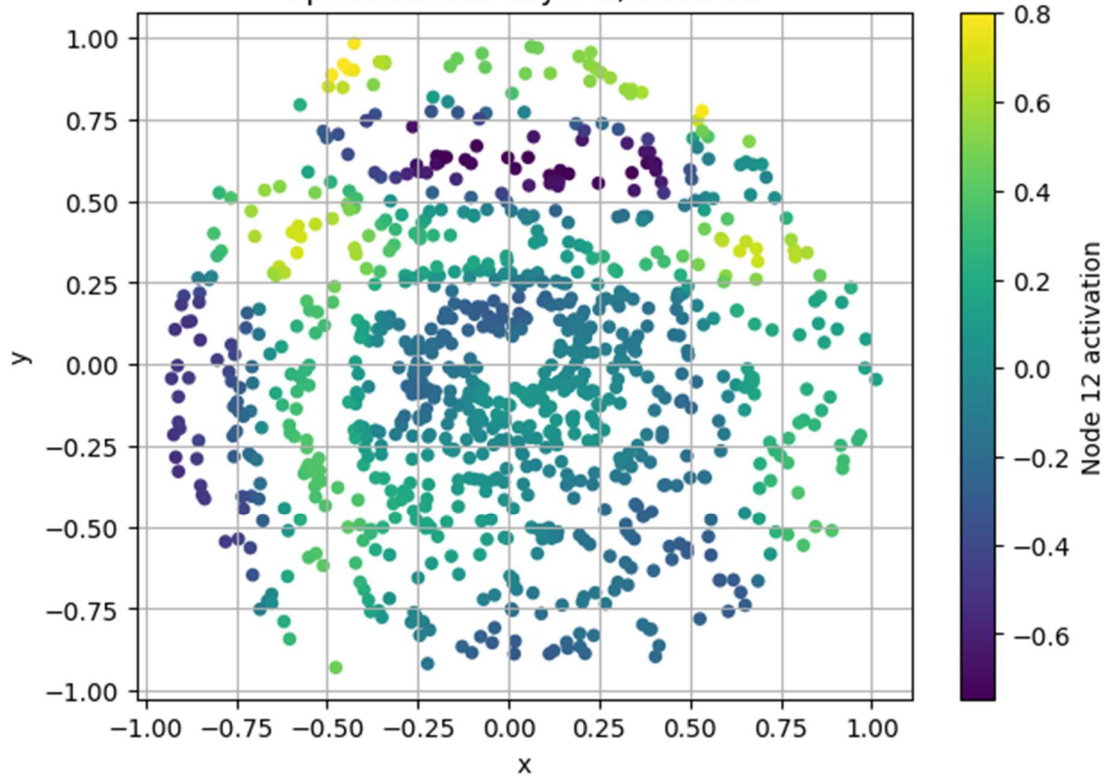


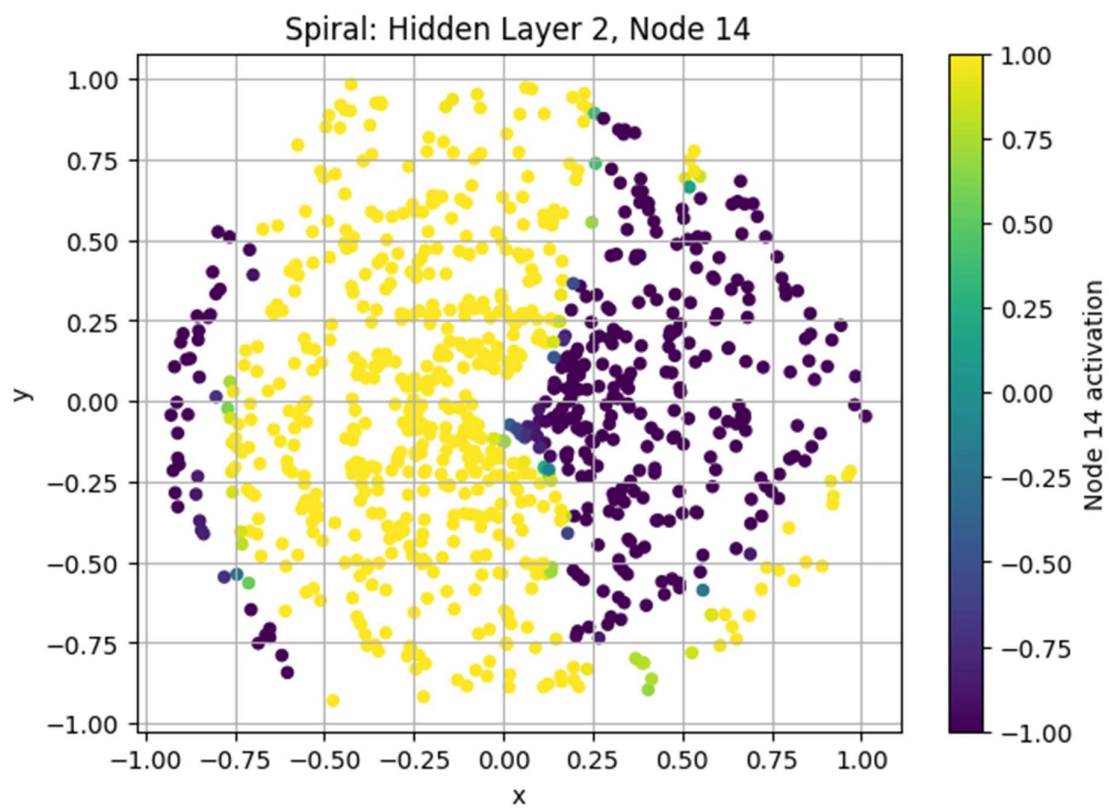
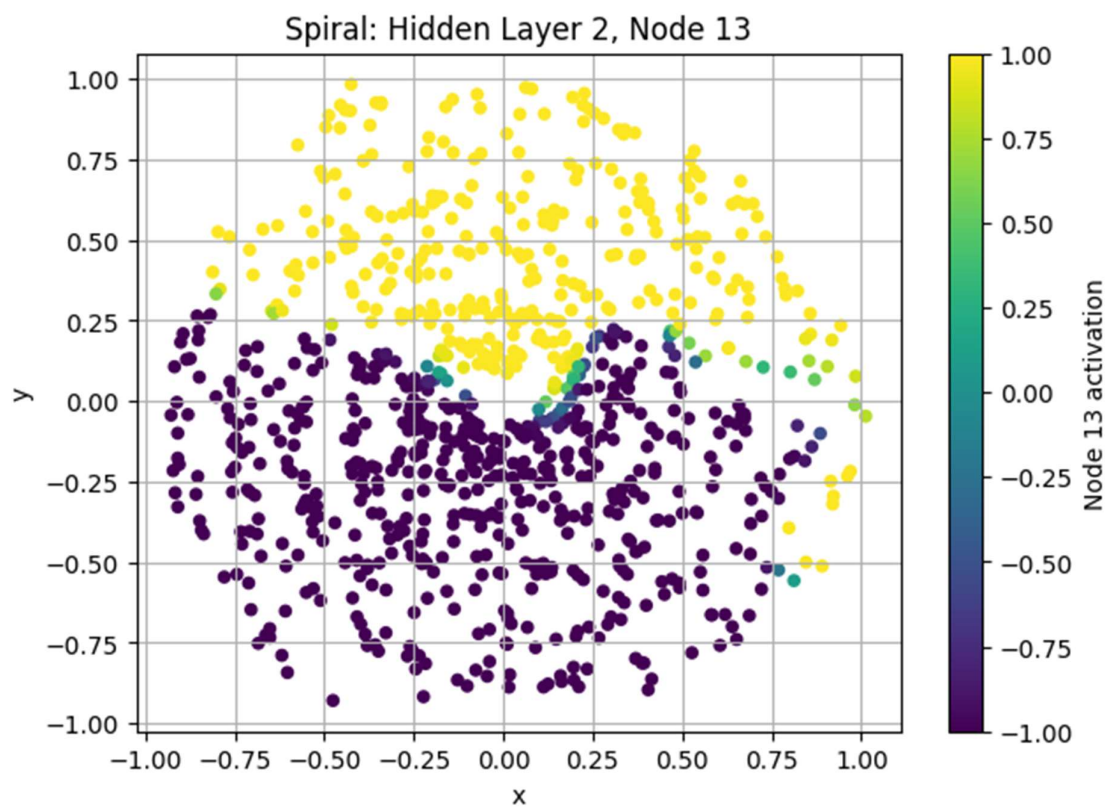


Spiral: Hidden Layer 2, Node 11

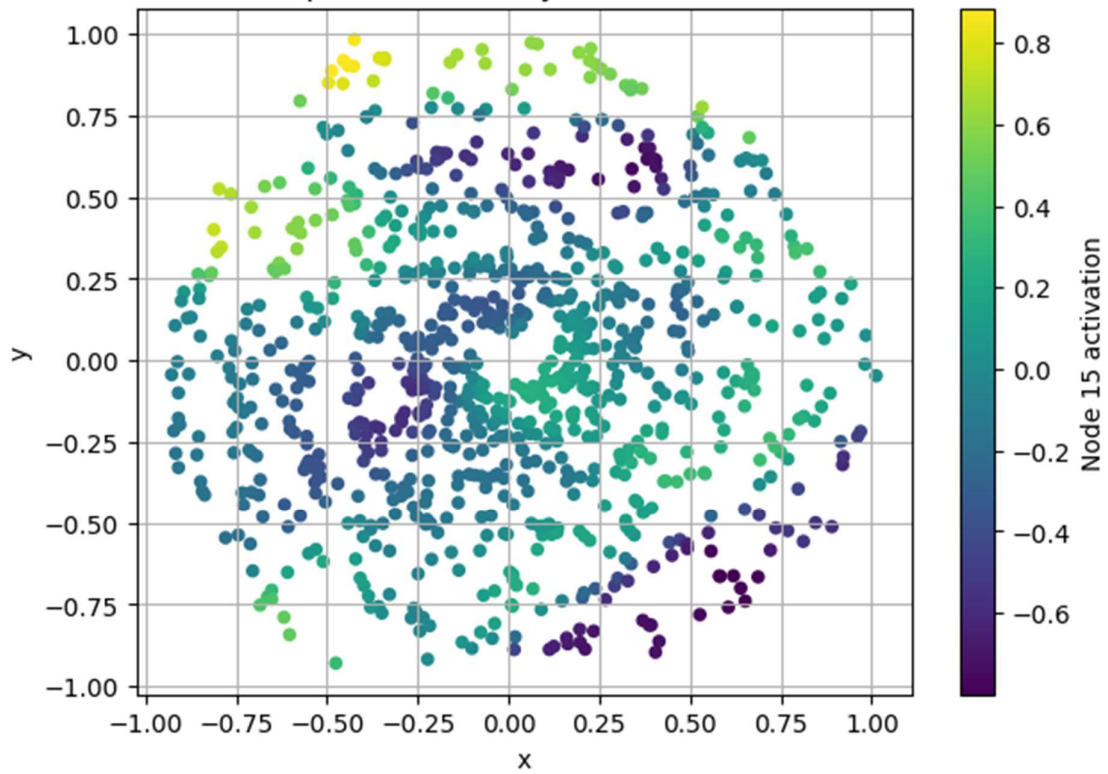


Spiral: Hidden Layer 2, Node 12

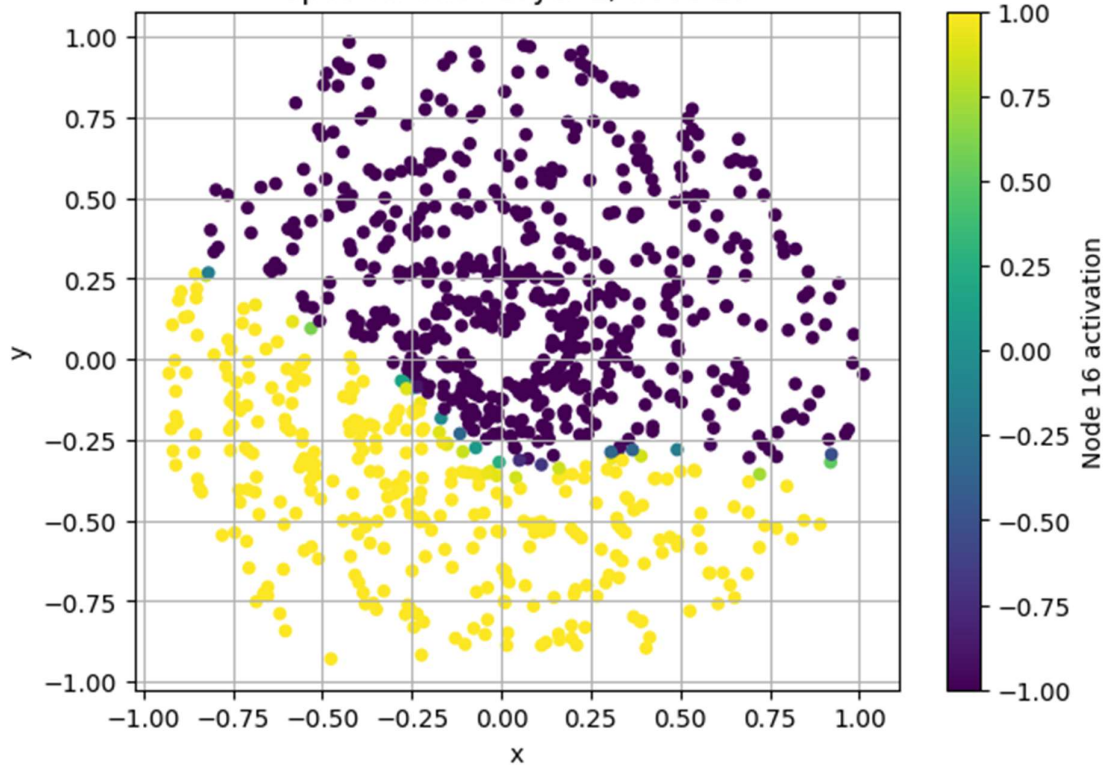


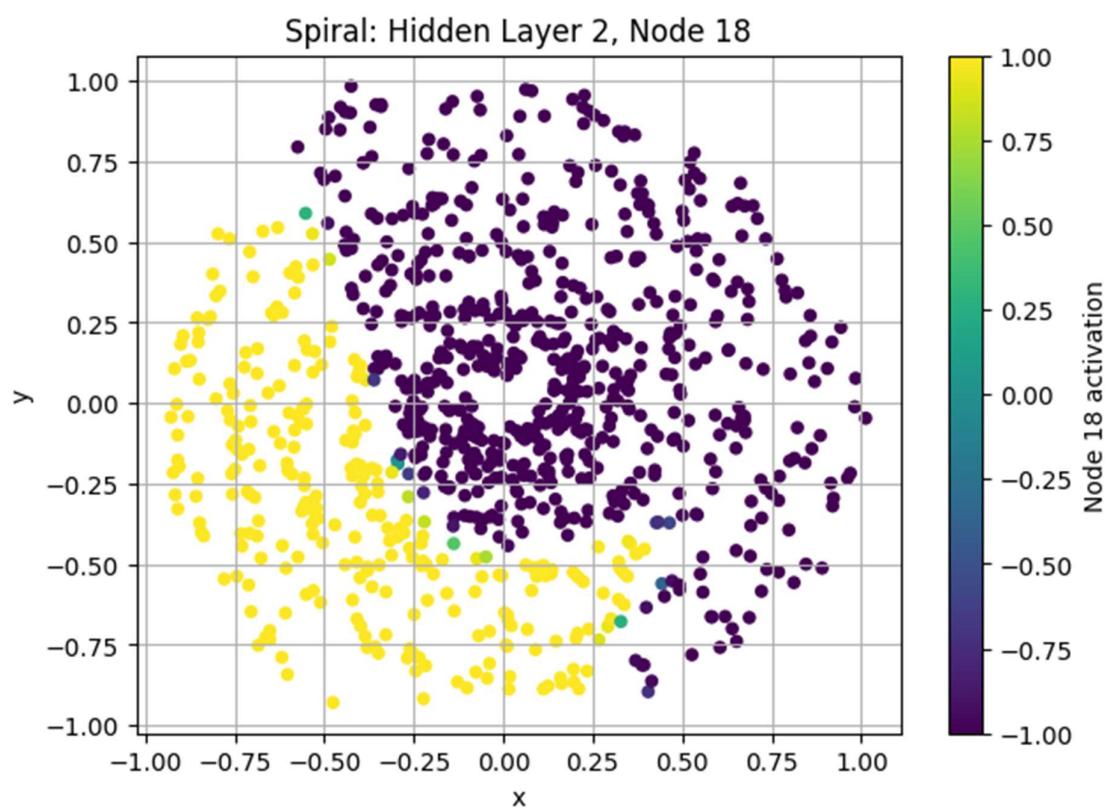
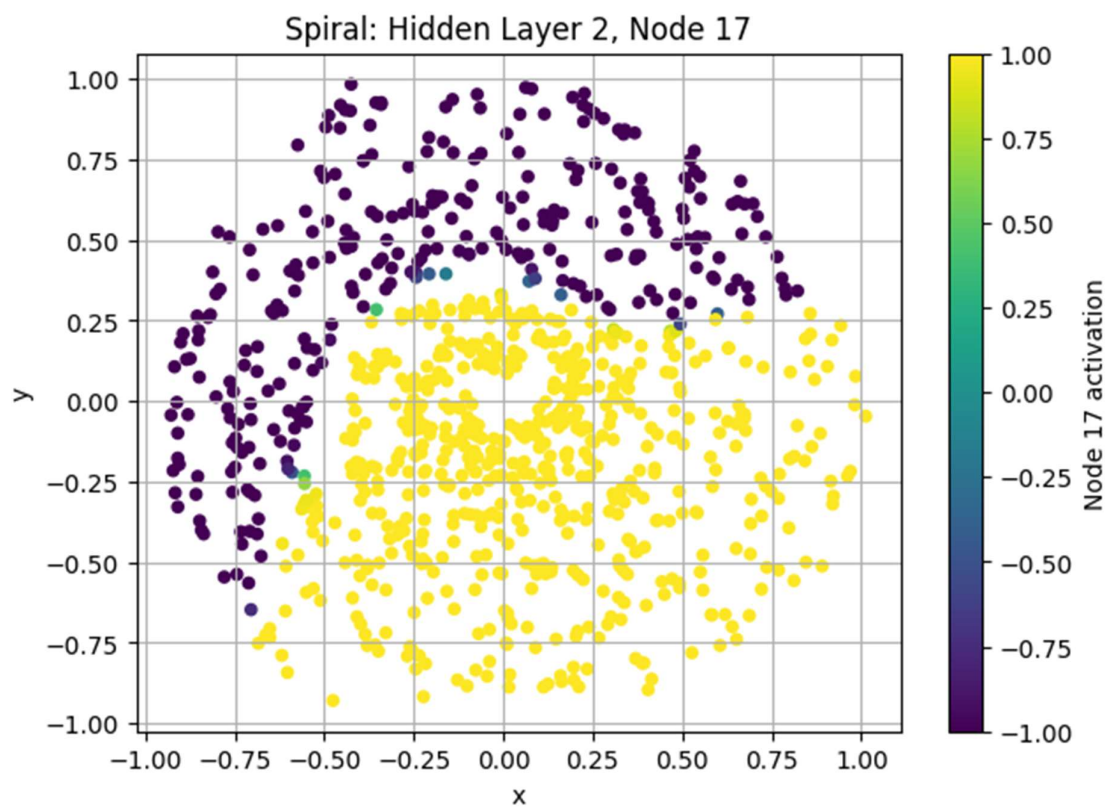


Spiral: Hidden Layer 2, Node 15

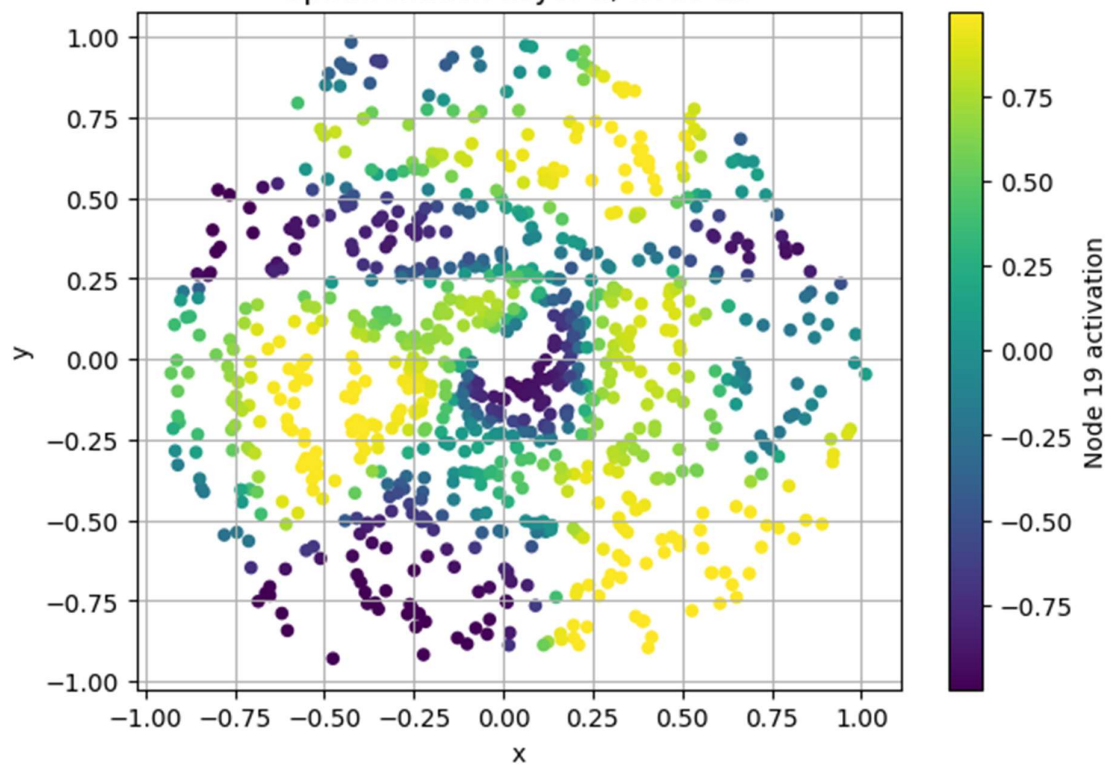


Spiral: Hidden Layer 2, Node 16

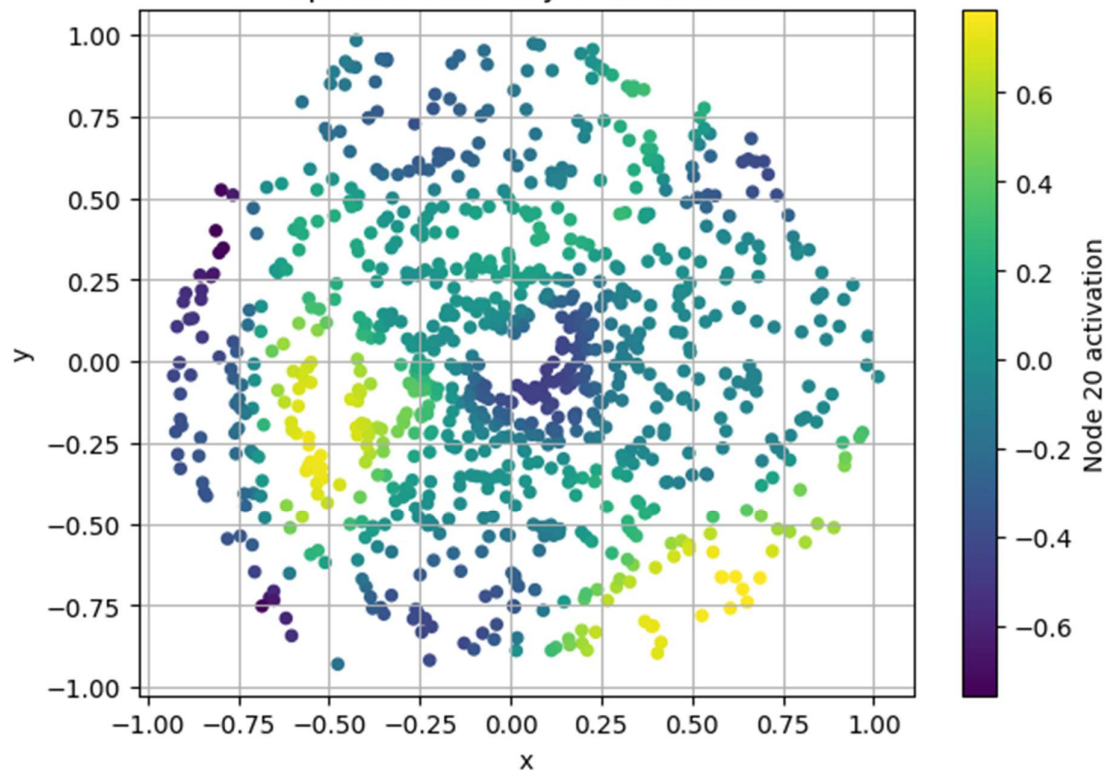


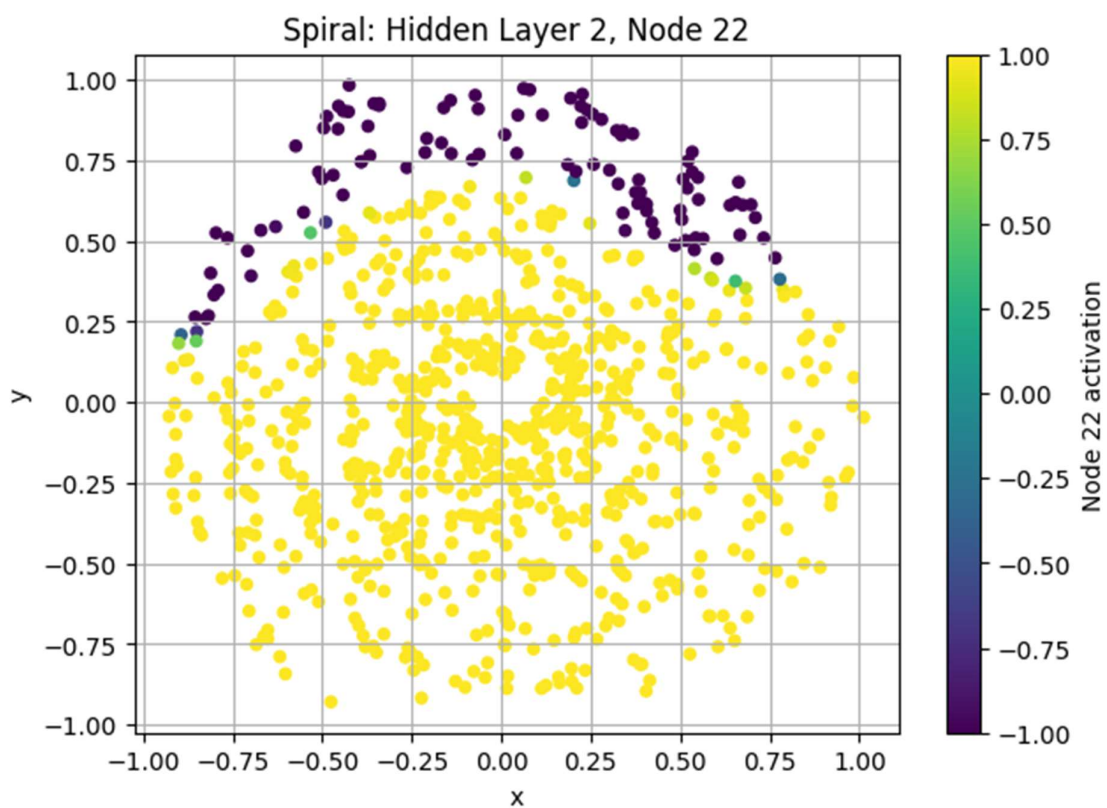
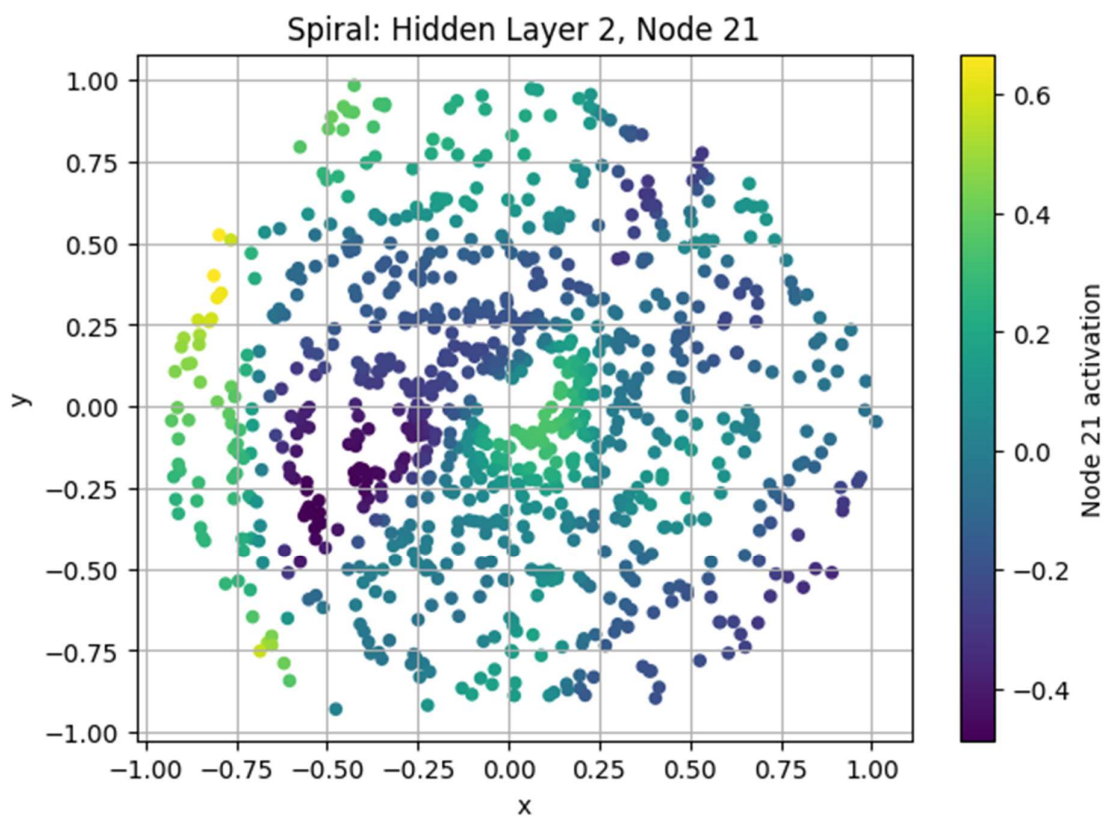


Spiral: Hidden Layer 2, Node 19

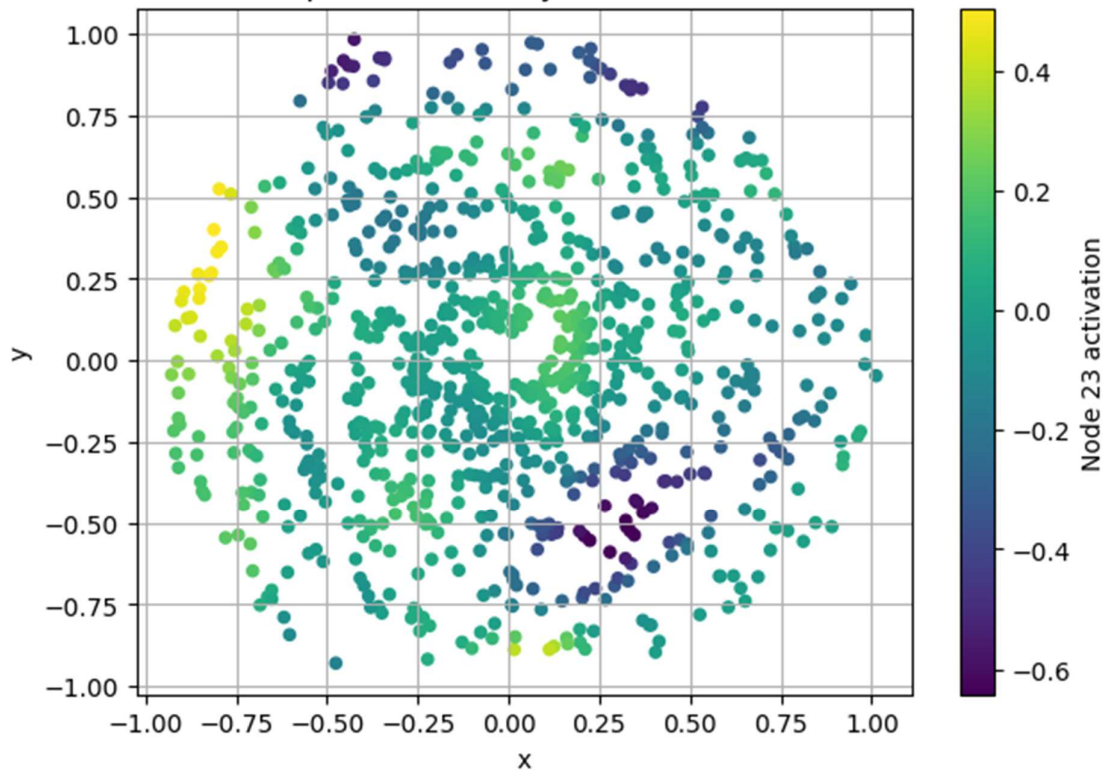


Spiral: Hidden Layer 2, Node 20

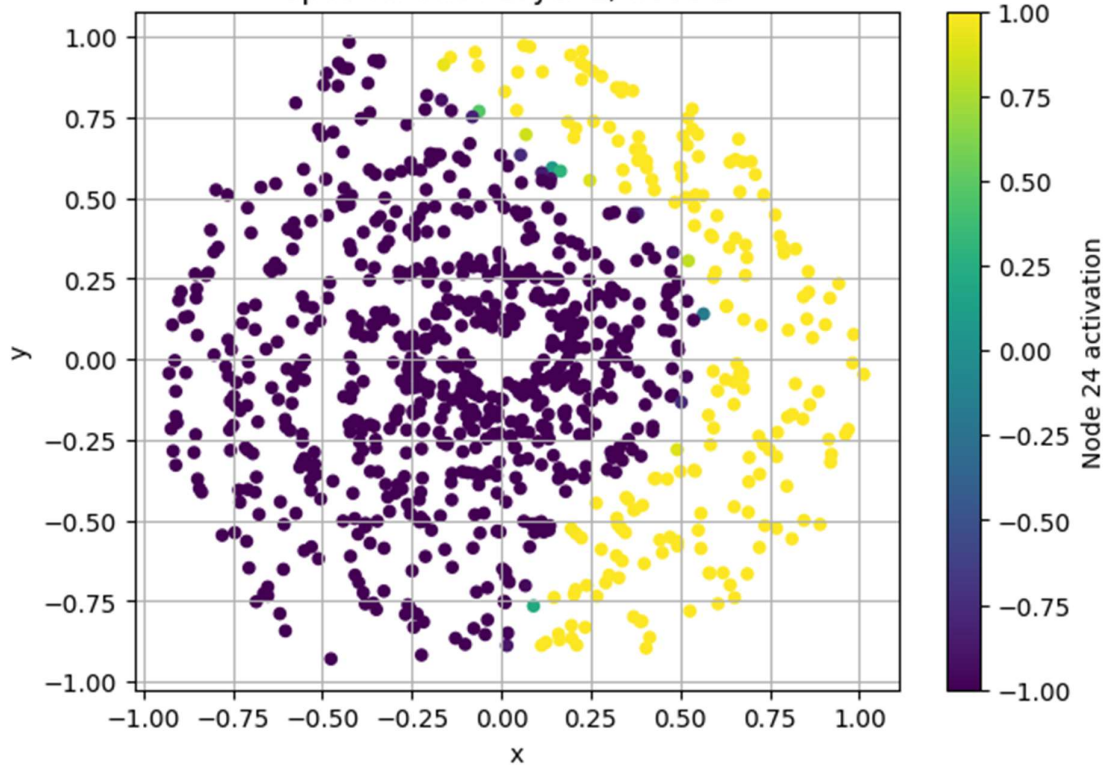


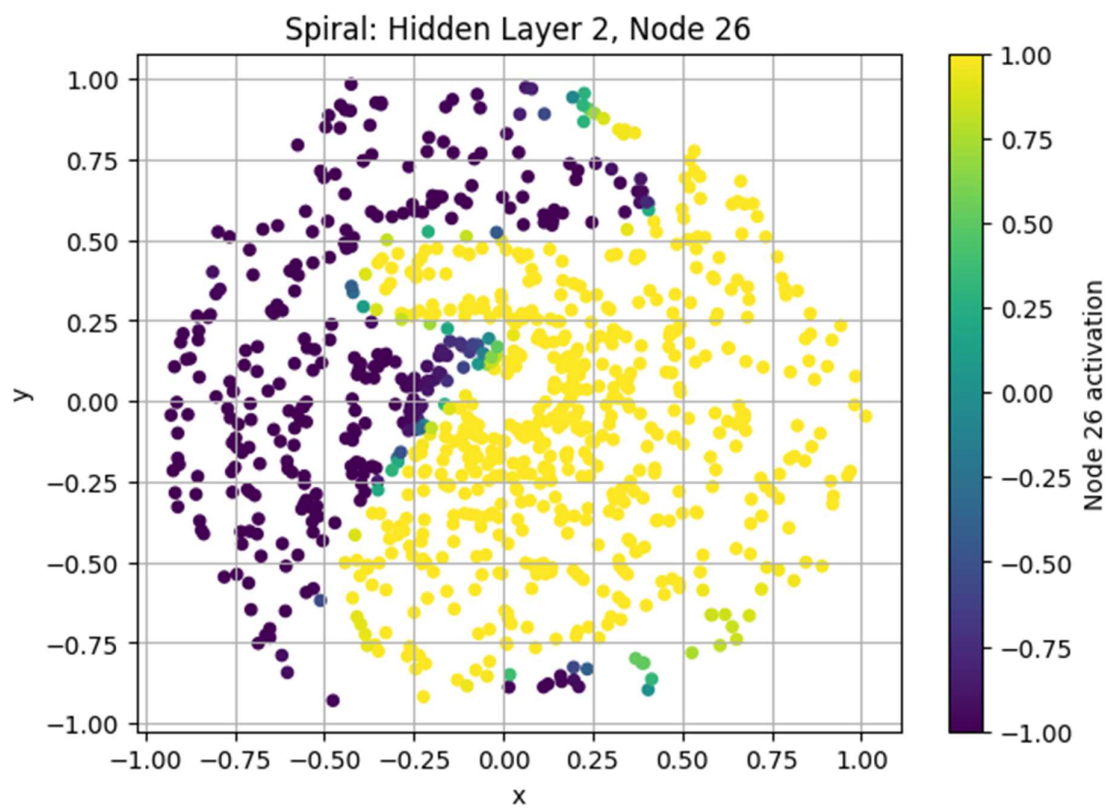
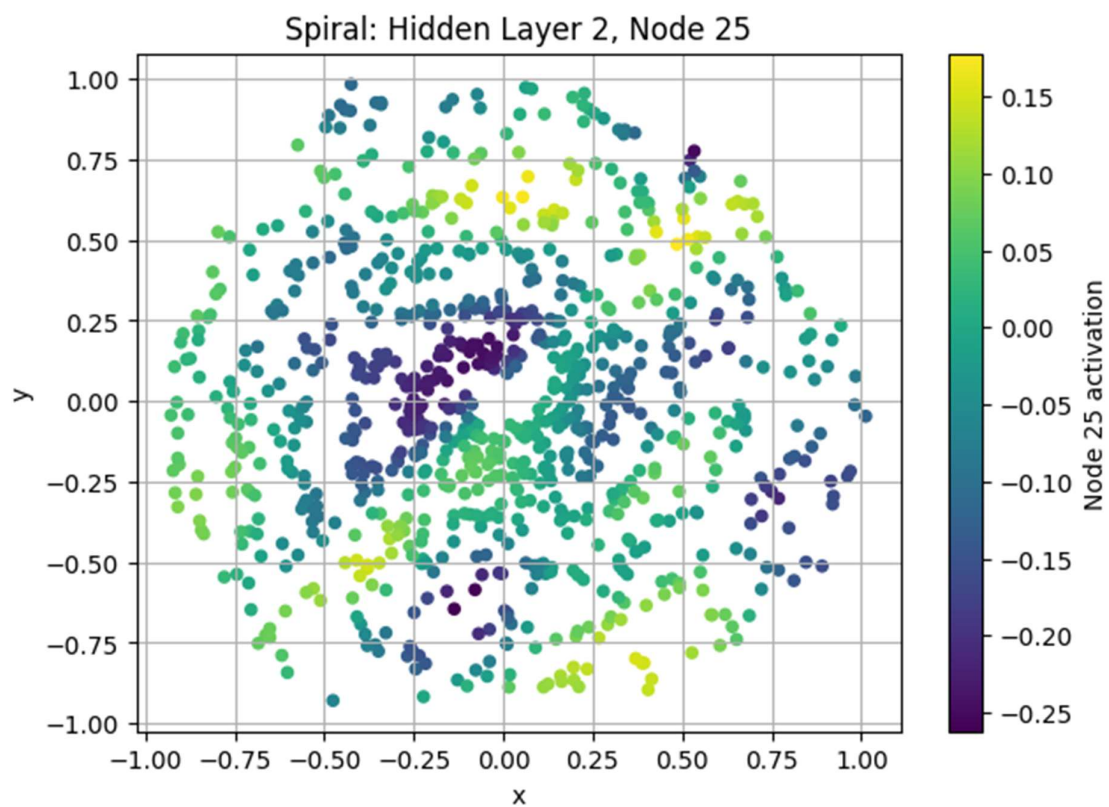


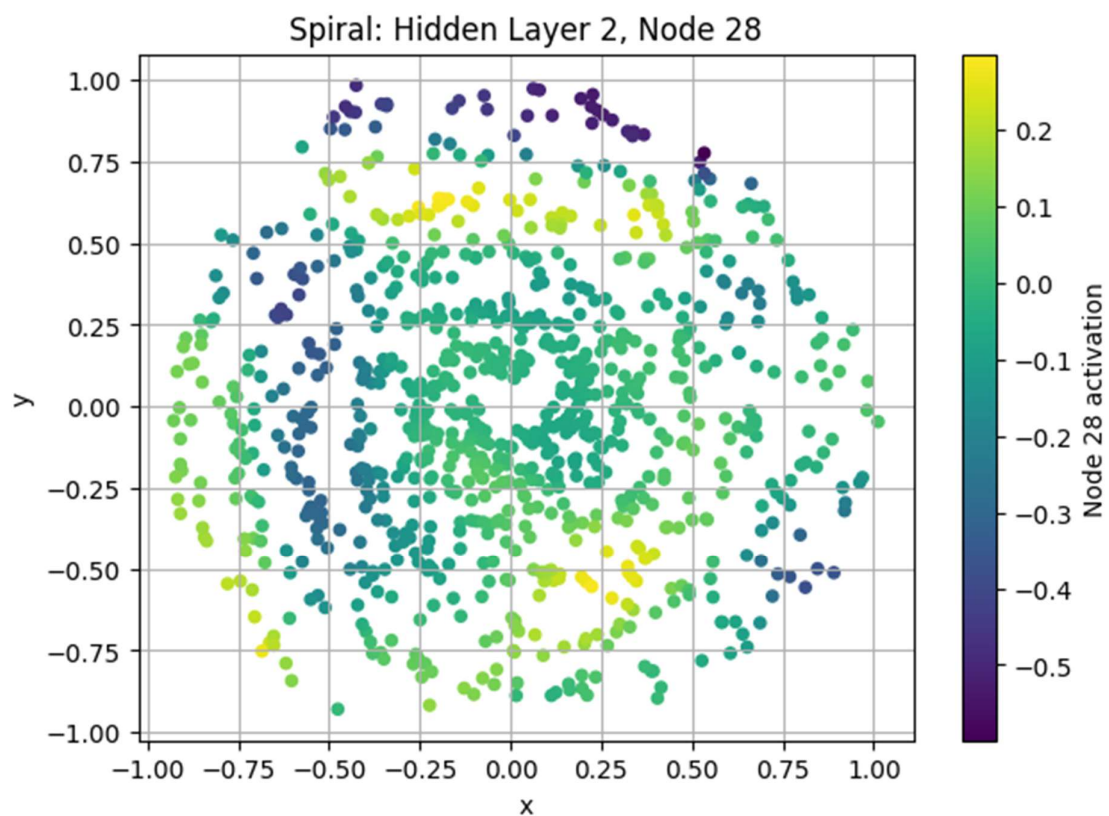
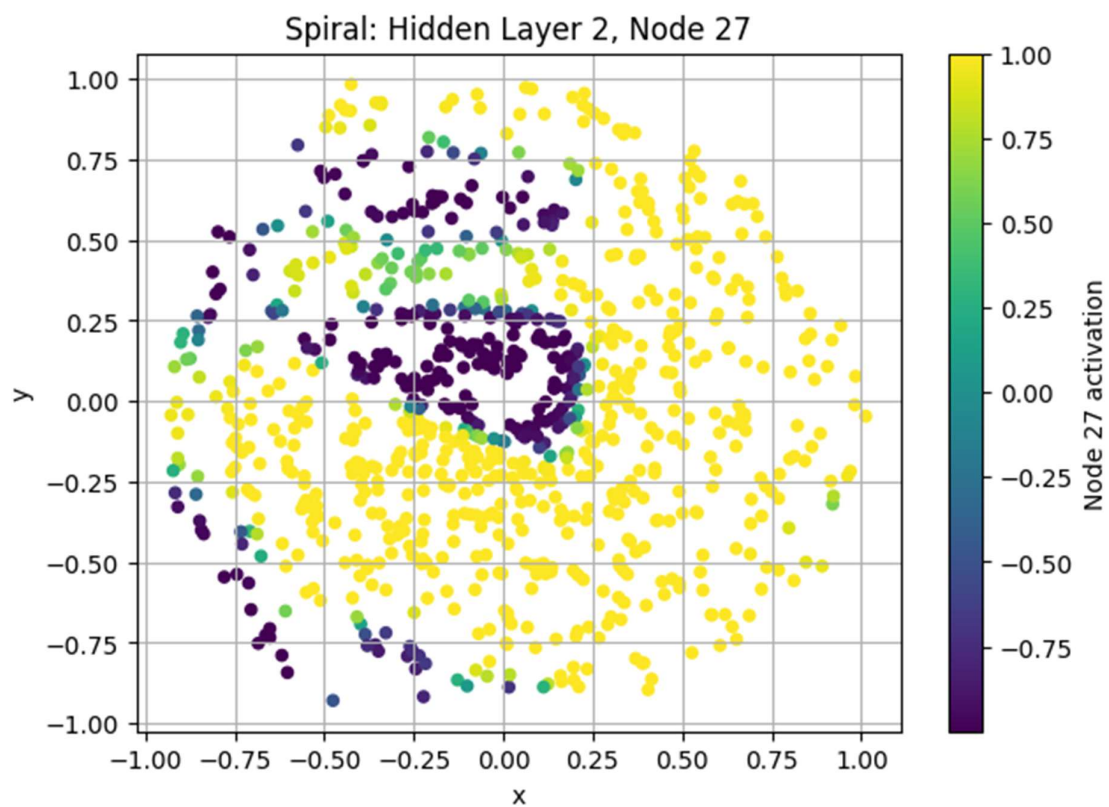
Spiral: Hidden Layer 2, Node 23



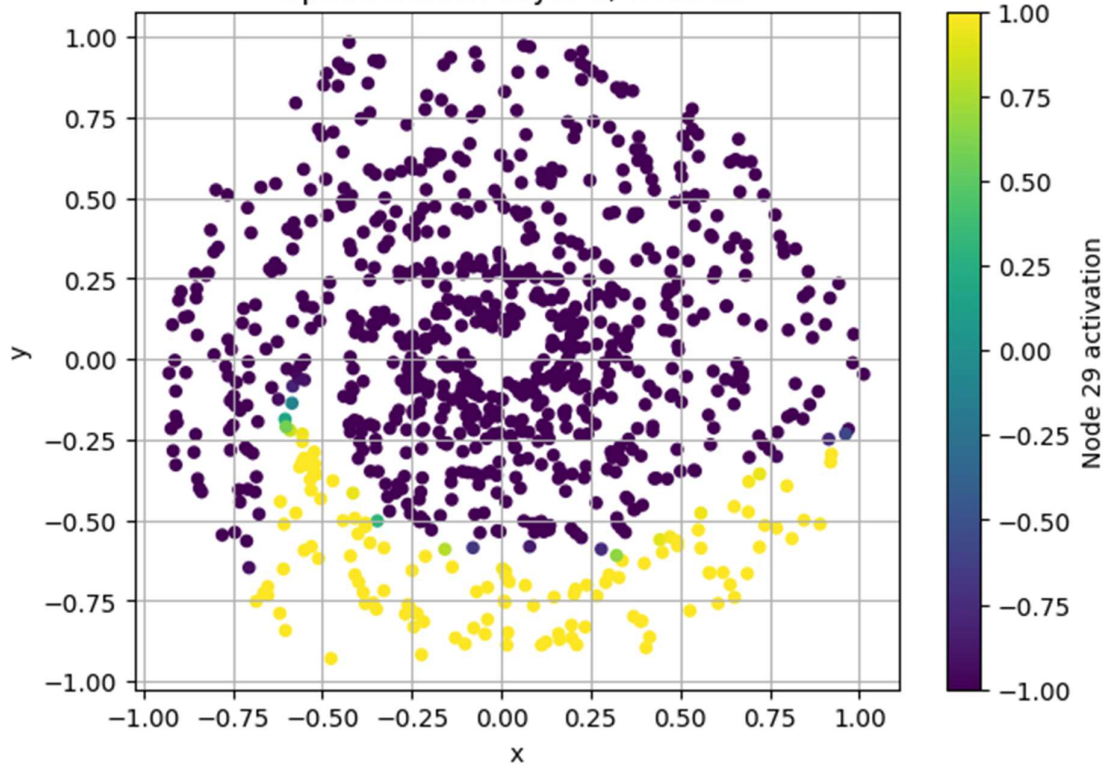
Spiral: Hidden Layer 2, Node 24







Spiral: Hidden Layer 2, Node 29



Spiral: Hidden Layer 2, Node 30

